FEMaster User Manual

Finite Elements in Mechanical Analysis and Simulation for Technical Engineering Research

Version 2.6.25

19 August 2026

Contents

1 Overview **1**

1.1 Two input styles . . . 1
1.2 The finite-element model . . . 1
1.3 Typical workflow . . . 2
1.4 Supported analysis families . . . 2
1.5 Degree-of-freedom convention . . . 3
1.6 Units . . . 3
1.7 Reading the keyword pages . . . 3

2 Build and Command Line **4**

2.1 Build . . . 4
2.2 CMake presets . . . 4
2.3 Running an input deck . . . 5
2.4 DSL documentation mode . . . 5

3 Input Format **7**

3.1 Command lines . . . 7
3.2 Data lines and multiline records . . . 7
3.3 Scopes . . . 8
3.4 Names, identifiers, and references . . . 8
3.5 FEMaster and Abaqus examples in this manual . . . 8
3.6 `*HEADING` . . . 9

4 Model Definition **10**

4.1 Model organization . . . 10
4.2 `*MODEL` . . . 10
4.3 `*PART` . . . 11
4.4 `*END PART` . . . 12
4.5 `*ASSEMBLY` . . . 13
4.6 `*END ASSEMBLY` . . . 14
4.7 `*INSTANCE` . . . 15
4.8 `*END INSTANCE` . . . 16

4.9 Mesh definition . . . 16
4.10 *NODE . . . 17
4.11 *ELEMENT . . . 18

5 Elements **19**
5.1 Supported element types . . . 19
5.2 Solid elements . . . 20
5.3 C3D4: 4-node tetrahedral solid . . . 21
5.4 C3D5: 5-node pyramid input . . . 22
5.5 C3D6: 6-node wedge solid . . . 23
5.6 C3D8: 8-node hexahedral solid . . . 24
5.7 C3D8R: reduced-integration 8-node hexahedral solid . . . 25
5.8 C3D10: 10-node quadratic tetrahedral solid . . . 26
5.9 C3D15: 15-node quadratic wedge solid . . . 27
5.10 C3D20: 20-node quadratic hexahedral solid . . . 28
5.11 C3D20R: 20-node reduced-integration hexahedral solid . . . 29
5.12 Shell elements . . . 29
5.13 S3: legacy 3-node triangular shell . . . 31
5.14 MITC3FRT: finite-rotation 3-node shell . . . 32
5.15 S4: legacy 4-node quadrilateral shell . . . 33
5.16 MITC4: legacy 4-node shell with tied shear interpolation . . . 34
5.17 MITC4FRT: finite-rotation 4-node MITC shell . . . 35
5.18 S6: legacy 6-node triangular shell . . . 36
5.19 MITC6FRT: finite-rotation 6-node shell . . . 37
5.20 S8: legacy 8-node quadrilateral shell . . . 38
5.21 MITC8: legacy 8-node MITC shell . . . 39
5.22 MITC8FRT: finite-rotation 8-node MITC shell . . . 40
5.23 QSPT: quadrilateral shear-panel element . . . 41
5.24 Beam and truss elements . . . 41
5.25 B33: 3D beam element . . . 42
5.26 T3: 2-node truss element . . . 43
5.27 Element formulation comparison . . . 43

6 Regions and Selections **45**
6.1 *NSET . . . 46
6.2 *ELSET . . . 49
6.3 *SURFACE . . . 50
6.4 *SFSET . . . 51

7 Materials **52**
7.1 *MATERIAL . . . 53

7.2 *ELASTIC . . . 54
7.3 *HYPERELASTIC . . . 56
7.4 *DENSITY . . . 57
7.5 *THERMALEXPANSION . . . 58
7.6 *EXPANSION . . . 59

8 Sections and Element Properties 60
8.1 *PROFILE . . . 61
8.2 *SOLID SECTION . . . 62
8.3 *SHELL SECTION . . . 63
8.4 *BEAM SECTION . . . 65
8.5 *TRUSS SECTION . . . 66

9 Coordinate Systems and Orientations 67
9.1 *ORIENTATION . . . 68
9.2 *TRANSFORM . . . 70

10 Fields 71
10.1 *FIELD . . . 72
10.2 *NORMAL . . . 74

11 Loads and Boundary Conditions 75
11.1 *AMPLITUDE . . . 76
11.2 *SUPPORT . . . 77
11.3 *BOUNDARY . . . 78
11.4 *CLOAD . . . 79
11.5 *DLOAD . . . 80
11.6 *PLOAD . . . 81
11.7 *DSLOAD . . . 82
11.8 *VLOAD . . . 83
11.9 *TLOAD . . . 84
11.10 *INERTIALOAD . . . 85

12 Constraints 86
12.1 *COUPLING . . . 87
12.2 *TIE . . . 88
12.3 *CONNECTOR . . . 89
12.4 *RBM . . . 90
12.5 Contact . . . 90
12.6 *CONTACT . . . 91

13 Analysis Procedures 93

13.1 Analysis families . . . 93
13.2 *LOADCASE . . . 94
13.3 *SUPPORTS . . . 95
13.4 *LOADS . . . 96
13.5 *END . . . 97
13.6 *STEP . . . 98
13.7 *STATIC . . . 99
13.8 *FREQUENCY . . . 100
13.9 *BUCKLE . . . 101
13.10 *DYNAMIC . . . 102
13.11 *STEADY STATE DYNAMICS . . . 103
13.12 *END STEP . . . 104

14 Solver and Numerical Controls **105**
14.1 *SOLVER . . . 106
14.2 *CONSTRAINTMETHOD . . . 107
14.3 *NONLINEAR . . . 109
14.4 *TIME . . . 111
14.5 *NEWMARK . . . 112
14.6 *DAMPING . . . 113
14.7 *FREQUENCIES . . . 114
14.8 *NUMEIGENVALUES . . . 115
14.9 *SIGMA . . . 116
14.10 *WRITEEVERY . . . 117
14.11 *INITIALVELOCITY . . . 118
14.12 *INERTIARELIEF . . . 119
14.13 *REBALANCELOADS . . . 120

15 Output, Diagnostics, and Results **121**
15.1 *OVERVIEW . . . 122
15.2 *REQUESTSTIFFNESS . . . 123
15.3 *REQUESTSTGEOM . . . 124
15.4 *CONSTRAINTSUMMARY . . . 125
15.5 Abaqus output-request keywords . . . 125
15.6 *NODE FILE . . . 126
15.7 *EL FILE . . . 127
15.8 *NODE PRINT . . . 128
15.9 *EL PRINT . . . 129
15.10 Native .res files . . . 129
15.11 FRD visualization output . . . 131

16 Advanced Model Features **132**
16.1 `*POINTMASS` . . . 133
16.2 Topology-weighted linear statics . . . 134
16.3 `*TOPODENSITY` . . . 135
16.4 `*TOPOORIENT` . . . 136
16.5 `*TOPOEXPONENT` . . . 137

17 Complete Examples **138**
17.1 Linear static truss . . . 138
17.2 Reusable Part and translated Instance . . . 139
17.3 Homogeneous shell cantilever . . . 140
17.4 Solid cube under pressure . . . 141
17.5 Eigenfrequency analysis . . . 142
17.6 Linear buckling of a compressed member . . . 143
17.7 Arc-length snap-through . . . 144
17.8 Linear transient ramp load . . . 145
17.9 Linear harmonic frequency sweep . . . 146
17.10 Using the examples as templates . . . 147

18 Keyword Index **148**
18.1 Supported element formulations . . . 150
18.2 Coverage notes . . . 150

A Mathematical Derivations **151**
A.1 Element stiffness . . . 151
A.2 Compliance . . . 151
A.3 Density penalization . . . 152
A.4 Orientation-dependent stiffness . . . 152
A.5 Voigt transformation . . . 152
A.6 Orientation sensitivity . . . 153
A.7 Rotation derivatives . . . 153

1 Overview

FEMaster is a finite-element solver for structural mechanics. Models can be described either with the native FEMaster input syntax or with the Abaqus-style syntax supported by FEMaster. This manual documents both forms together: each modelling concept is explained once, and the corresponding FEMaster and Abaqus input is shown side by side wherever both representations are available.

The documentation is organized by the way a finite-element model is built and solved. After the general input rules, the manual describes model organization and mesh definition, the available element formulations, regions and surfaces, materials, sections, coordinate systems, Fields, loads and boundary conditions, constraints including contact, analysis procedures, numerical controls, result output, and complete examples. A keyword index at the end provides a direct route from an input keyword to its detailed description.

1.1 Two input styles

Native FEMaster input exposes the solver's own modelling concepts directly. Loads and supports can be collected under names and selected by a loadcase, Fields can be defined explicitly, and analysis-specific numerical controls are available through dedicated FEMaster keywords.

The supported Abaqus input style uses familiar Abaqus concepts such as `*PART`, `*ASSEMBLY`, `*INSTANCE`, `*STEP`, `*BOUNDARY`, and procedure keywords such as `*STATIC` or `*FREQUENCY`. FEMaster interprets the supported subset in terms of the same structural model and analysis capabilities described throughout this manual.

Neither syntax forces the user to introduce Parts and Instances for a simple model. In native FEMaster input, nodes and elements can be defined directly. The structured Part/Assembly representation supports reused components, several placed copies of one component, independent local ID spaces, and Abaqus-style assemblies.

Optional Part/Assembly hierarchy

A native FEMaster model does not require `*PART`, `*ASSEMBLY`, or `*INSTANCE`. A flat model is a fully supported workflow, while the hierarchy adds explicit organization for reusable and placed components.

1.2 The finite-element model

A structural finite-element model consists of several layers of information that work together.

Nodes define the reference geometry and carry the mechanical degrees of freedom. **Elements** connect those nodes and define the interpolation and kinematic assumptions used over lines, surfaces, or volumes. The element formulation determines which deformation modes can be represented and how strains, curvatures, internal forces, mass, and other element quantities are evaluated.

Node sets, element sets, and surfaces give reusable names to parts of the mesh. They are used to assign Sections, supports, loads, couplings, ties, contact pairs, and other model features without repeatedly listing individual IDs.

Materials define constitutive behaviour and density. **Sections** connect element regions to their physical properties. A solid Section assigns a material; a shell Section additionally defines thickness or generalized shell stiffness; a truss Section defines area; and a beam Section combines a material with a profile and local cross-section direction.

Coordinate systems and orientations define local directions for anisotropic material behaviour, beam and shell properties, loads, supports, nodal transformations, and other directional quantities.

Fields store quantities that vary spatially over nodes, elements, element-local nodes, integration points, or material points. They are used when a model feature requires more information than can be expressed by one scalar keyword parameter, for example topology variables or prescribed shell directors.

Loads and boundary conditions define the external forcing and prescribed motion. **Constraints** introduce relations between different degrees of freedom, such as couplings, ties, idealized connectors, rigid-body-mode suppression, and unilateral contact.

Finally, an **analysis procedure** determines which equilibrium or eigenvalue problem is solved. Numerical controls select the solution strategy, constraint treatment, time or load stepping, convergence criteria, damping, and related settings.

1.3 Typical workflow

A practical FEMaster model is usually built in the following order:

1. define the geometry and mesh using nodes, elements, and optionally Parts and Instances;
2. create named regions and surfaces that will be used by later definitions;
3. define materials, profiles, Sections, and coordinate systems;
4. define Fields or other spatial model data where needed;
5. apply loads, supports, and kinematic constraints;
6. define the analysis procedure and its numerical controls;
7. resulting displacement, force, stress, strain, mode-shape, buckling, transient, or harmonic Fields complete the analysis output.

The same order exposes dependencies during diagnosis: geometry and element connectivity precede Section and material assignments, which precede region definitions and coordinate directions, supports and constraints, and finally the numerical analysis settings.

1.4 Supported analysis families

FEMaster analysis	Purpose
`LINEARSTATIC`	Small-displacement static equilibrium for a linear structural model.
`LINEARSTATICTOPO`	Linear static analysis with element-wise topology density, penalization, and optional orientation Fields.
`NONLINEARSTATIC`	Incremental nonlinear equilibrium with load control or arc-length path following, nonlinear element formulations, and contact where applicable.
`EIGENFREQ`	Constrained free-vibration eigenvalue analysis based on the structural stiffness and mass matrices.
`LINEARBUCKLING`	Linearized eigenvalue buckling analysis about a preloaded state using geometric stiffness.

FEMaster analysis	Purpose
`LINEARTRANSIENT`	Implicit linear time integration with Newmark parameters, optional Rayleigh damping, initial velocity, and configurable result cadence.
`LINEARHARMONIC`	Direct steady-state harmonic response over one or more excitation frequencies with optional Rayleigh damping.

The supported Abaqus procedures `*STATIC`, `*FREQUENCY`, `*BUCKLE`, `*DYNAMIC`, and `*STEADY STATE DYNAMICS` are documented next to the corresponding FEMaster analysis procedures in Chapter 13.

1.5 Degree-of-freedom convention

Generalized nodal quantities use the order

`Ux, Uy, Uz, Rx, Ry, Rz`.

The first three entries are translations and the last three are rotations. This convention is used by generalized concentrated loads, supports, coupling selectors, initial velocities, and several result Fields. Individual element formulations may use only a subset: continuum solids generally use translations, whereas beams and shells also use rotational degrees of freedom.

This nodal six-component order is distinct from the six-component stress/strain order. Stress and strain tensors use the engineering component convention documented in the result chapter.

1.6 Units

FEMaster does not impose a built-in unit system. All input quantities must form one consistent system. For example, a common structural choice is millimetres, newtons, tonnes, seconds, and megapascals, provided density and all dynamic quantities are converted consistently. The solver only combines the numerical values according to the finite-element equations; it cannot detect that a modulus was entered in pascals while the geometry was entered in millimetres.

Unit consistency is especially important for density, inertia, dynamic analyses, contact penalty stiffness, thermal expansion, and any problem in which length, force, mass, and time appear simultaneously.

1.7 Reading the keyword pages

Each input keyword has one canonical location in this manual and starts on a new page. The page first explains the modelling purpose and physical meaning of the command. Parameter and data-line tables then define the accepted syntax. Finally, a side-by-side example shows native FEMaster input on the left and supported Abaqus input on the right.

When both columns contain the same syntax, that is intentional. When the corresponding Abaqus keyword is not currently supported, the Abaqus column states that explicitly instead of showing syntax that FEMaster would reject. The aim is to make the manual a reliable description of what can actually be used in a FEMaster calculation.

2 Build and Command Line

FEMaster is built with CMake. A typical build produces an executable named `FEMaster` in the build output directory. The executable reads an input deck and writes result files using a common job base name. If no explicit output path is supplied, the job base is derived from the input file name. FEMaster then writes both the native text result file `.res` and the CalculiX/CGX result file `.frd`.

2.1 Build

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

A release build provides optimized execution for analysis work. A debug build retains assertions and development diagnostics at substantially higher runtime cost for larger finite-element models.

2.2 CMake presets

The repository provides CMake presets for the common supported toolchain combinations. Presets place their build directory under `build/<preset-name>` and set the normal release-oriented defaults: tests disabled, double precision enabled, Eigen compile-time reduction flags enabled, and OpenMP disabled unless the preset explicitly enables it.

```
cmake --preset linux
cmake --build --preset linux

cmake --preset linux-mkl-omp
cmake --build --preset linux-mkl-omp
```

The Linux presets cover CPU-only builds, OpenMP builds, oneMKL builds, CUDA builds, cuDSS builds, and combined CUDA/oneMKL configurations. Windows presets are split between MSVC and IntelLLVM toolchains. macOS currently provides CPU presets. `CMakePresets.json` contains the exact preset list for the checked-out version.

CMake option	Meaning
`FEMASTER_BUILD_TESTS`	Build and register the GoogleTest executable.
`FEMASTER_ENABLE_CUDA`	Build CUDA solver backends.
`FEMASTER_ENABLE_CUDSS`	Enables NVIDIA cuDSS for CUDA sparse direct solves. Requires CUDA.
`FEMASTER_ENABLE_MKL`	Enables Intel oneMKL for Eigen operations and PARDISO.
`FEMASTER_MKL_SEQUENTIAL`	Selects the sequential oneMKL threading layer.

CMake option	Meaning
FEMASTER_MKL_STATIC	Link oneMKL component libraries statically.
FEMASTER_ENABLE_OPENMP	Enable OpenMP for FEMaster CPU loops.
FEMASTER_DOUBLE_PRECISION	Enables double-precision arithmetic.
FEMASTER_FAST_EIGEN_COMPILE	Enables Eigen compile-time reduction flags.
FEMASTER_STATIC_RUNTIME	Link compiler C/C++ runtimes statically where supported.
FEMASTER_FULLY_STATIC	Produce a fully static Linux executable. Incompatible with CUDA shared libraries.
FEMASTER_TIME_REPORT	Enable compiler timing diagnostics where supported.
FEMASTER_DEPS_INCLUDE_DIR	Include root containing header-only dependencies such as Eigen, Spectra, and argparse.
FEMASTER_CUDSS_ROOT	Root of an NVIDIA cuDSS installation. Defaults to the CUDSS_DIR environment variable.

2.3 Running an input deck

```
FEMaster model.inp
FEMaster model.inp --output model.res
FEMaster model.inp --ncpus 8
FEMaster model.inp --no-frd
FEMaster model.inp --no-res
```

Argument	Meaning
input_file	Path to the input deck. If no .inp suffix is supplied, it is added automatically.
-output FILE	Explicit output base or result file path. The suffix .res, .frd, or .inp is stripped before writers are opened, so -output run.res produces run.res and run.frd.
-ncpus N	Number of threads allowed for parallel assembly and compute sections.
-no-res	Disables native FEMaster .res result output, leaving .frd output enabled by default.
-no-frd	Disables CalculiX/CGX .frd result output, leaving native .res output enabled by default.

By default both result writers are enabled. Combining -no-res and -no-frd leaves parsing, solution, and console diagnostics enabled without writing either result format.

At the end of a regular solver run, FEMaster prints a process timing summary in hours, minutes, seconds, milliseconds, and total milliseconds. This timing covers the full executable run, including parsing, solving, result writing, and shutdown.

2.4 DSL documentation mode

The executable can print DSL registration information containing the commands, keywords, and variants accepted by the installed build.

```
FEMaster --document --doc-list
FEMaster --document --doc-show CLOAD
FEMaster --document --doc-variants ELASTIC
```

Option	Meaning
`-doc-list`	Print the available DSL commands.
`-doc-show CMD`	Show details for one command.
`-doc-tokens CMD`	Show expected tokens for a command.
`-doc-variants CMD`	Show accepted data-line variants.
`-doc-search TEXT`	Search command names and descriptions.
`-doc-where-token TOKEN`	Find commands that use a token.
`-doc-all`	Print the full command overview.
`-doc-format text|md|json`	Select the documentation output format. Currently the implementation falls back to text for non-text formats.
`-doc-verbosity index|compact|full`	Select the detail level for command documentation.
`-doc-width N`	Set the wrapping width for documentation output; `0` disables wrapping.
`-doc-no-wrap`	Disable wrapping. Equivalent to `-doc-width 0`.
`-doc-regex`	Treat `-doc-search` as a regular expression.

3 Input Format

FEMaster reads keyword-based input decks. Both the native FEMaster language and the supported Abaqus input format use the familiar structure in which a command starts with an asterisk, optional parameters follow after commas, and command-specific data are supplied on one or more following lines.

The two syntaxes describe the same types of finite-element objects but do not always expose them in the same way. Native FEMaster input provides direct access to FEMaster concepts such as load and support collectors, explicit loadcases, generic Fields, and numerical solver controls. Abaqus input uses Abaqus-style Parts, Assemblies, Steps, boundary conditions, and procedure keywords. The remainder of this manual explains both forms together and shows their accepted syntax side by side.

3.1 Command lines

A command line begins with `*`. The command name follows immediately and may be followed by comma-separated parameters. Parameters can be key/value pairs or flags. Data lines continue until the next command line begins.

Syntax

```
*COMMAND, KEY=value, FLAG
value, value, value
value, value, value
```

Command and parameter names are case insensitive. Whitespace inside command names is ignored, so compact names such as `*SOLIDSECTION` and spaced names such as `*SOLID SECTION` refer to the same command where that keyword is supported. The manual normally uses the spaced form when it improves readability and the compact form where it is customary in native FEMaster decks.

Parameter values retain their meaning. Numerical values are read as integer or floating-point quantities according to the keyword definition. Boolean-style options may appear as flags or as explicit values where documented. Names are used to reference Parts, Instances, sets, surfaces, materials, sections, coordinate systems, Fields, amplitudes, collectors, and other model entities.

3.2 Data lines and multiline records

The number and meaning of data-line entries depend on the keyword. Some commands accept one record per line, such as `*NODE`. Others read a fixed-size group of values, a matrix, a list of identifiers, or multiple records. The keyword pages describe the required ordering explicitly.

Comma-separated values preserve the visibility of optional empty entries. Whitespace around values is ignored. Several native FEMaster commands use the literal `NAN` to mean that a generalized component is not prescribed. This is different from zero: zero is a physical numerical value, whereas an empty or NaN component may mean “leave this degree of freedom unspecified” for the corresponding keyword.

Long connectivity or data records may continue over more than one physical line where that command

permits multiline input. This representation accommodates higher-order elements and large fixed-size material or Section definitions.

3.3 Scopes

Several keywords open scopes. A Part contains reusable mesh topology and part-level regions. An Assembly contains Instance placement and assembly-level region definitions. Material subcommands apply to the active material. Native FEMaster `*LOADCASE` opens an analysis definition. Abaqus `*STEP` opens an Abaqus analysis step.

Scopes are defined by keywords, not by indentation. Indentation may be used for readability but has no syntactic meaning. Closing commands such as `*END PART`, `*END ASSEMBLY`, `*END INSTANCE`, `*END STEP`, and native `*END` make the end of the corresponding scope explicit.

Parts, Assemblies, and Instances are optional for simple native FEMaster models. Nodes and elements may be defined directly without constructing an explicit Assembly. The structured form represents reusable Parts, repeated Instances, independent local ID spaces, and assembly-level selections.

3.4 Names, identifiers, and references

Node and element IDs are user-defined non-negative integer labels. They may start at **0**; they do not need to start at 1 and do not need to be contiguous. IDs only need to be unique within the topology scope in which they are defined. For example, `0`, `10`, and `1000` are valid node IDs.

Inside separate Parts, the same numerical ID may be reused because each Part has its own local topology. Instances preserve these local labels from the user's point of view. When an assembly-level command needs to identify an entity from a particular Instance, the documented Instance-qualified reference syntax is used.

Named resources are used throughout the rest of the deck. Node sets, element sets, surfaces, materials, profiles, orientations, Fields, amplitudes, collectors, and other definitions are referenced by name. Stable names expose the model structure without repeatedly addressing individual numerical IDs.

User IDs and *local positional indices* represent different concepts. User node and element IDs may be sparse and may include zero. By contrast, indices such as a Field's `local_node`, `local_ip`, or `local_mp` identify a position inside an element and are explicitly zero based where documented.

3.5 FEMaster and Abaqus examples in this manual

Each keyword reference shows native FEMaster input on the left and the corresponding supported Abaqus input on the right. When the syntax is identical, both columns intentionally show the same form. When the modelling concepts differ, the two examples show the respective accepted representation. If no equivalent is currently read from Abaqus input, the Abaqus column states that directly rather than showing syntax that FEMaster would reject.

The comparison shows how the same model operation is expressed in either accepted input style without exposing parser implementation details.

3.6 *HEADING

`*HEADING` adds descriptive text to an input deck. It does not change the finite-element equations or the numerical result. The keyword keeps exported, archived, and regression decks self-describing and is accepted in both native FEMaster and Abaqus input.

A heading is metadata conventionally placed near the beginning of a file. Later model commands do not reference it by name, and it does not substitute for a Part name, material name, loadcase name, or other actual model identifier.

Native FEMaster also accepts `*MODEL, NAME=...` as an optional root-level model marker. `*HEADING` is free descriptive text, while `*MODEL` belongs to the model-definition syntax described in the next chapter.

Syntax and semantics

Syntax

```
*HEADING
Descriptive model title
```

FEMaster and Abaqus example

FEMaster

```
*HEADING
Cantilever verification model
```

Abaqus input

```
*HEADING
Cantilever verification model
```

4 Model Definition

Model definition describes the geometry and topology on which the analysis is built. A simple FEMaster model may be completely flat: nodes and elements can be defined directly without an explicit Part, Assembly, or Instance. The optional structured Part/Assembly form represents reused components, several placements of the same Part, separate local ID spaces, and imported Abaqus-style assemblies.

From the user's point of view, the two forms describe the same kind of finite-element model. In a flat model there is one direct topology. In a structured model, each Part owns its local nodes, elements, sets, surfaces, and section assignments, while Instances place those Parts into the final assembly. Parts therefore describe *what a component is*; Instances describe *where that component is*.

Node and element IDs are user labels. They are non-negative integers, may start at **0**, and may be sparse. Separate Parts may reuse the same numerical IDs because their topologies are independent. When an assembly-level definition needs to distinguish entities belonging to different Instances, the Instance name provides the required qualification.

4.1 Model organization

4.2 `*MODEL`

`*MODEL` is an optional native FEMaster root marker. It can attach a descriptive model name to a native deck and makes the top-level structure explicit, but it is not required. A file may begin directly with nodes, materials, Parts, or other root-level definitions.

`NAME` is optional. The command does not create a nested second model and is not used to select between several models in one input file. `*HEADING` provides free descriptive text, whereas `*MODEL` provides an explicit native model marker.

Parameters

Parameter	Status	Meaning
`NAME`	optional	Descriptive name of the native FEMaster model.

FEMaster and Abaqus example

FEMaster

```
*MODEL, NAME=BRACKET_MODEL
*NODE
0, 0., 0., 0.
1, 10., 0., 0.
```

Abaqus input

```
** No MODEL wrapper is required.
*NODE
0, 0., 0., 0.
1, 10., 0., 0.
```

4.3 *PART

*PART begins a reusable finite-element component definition. Nodes, elements, part-level sets, surfaces, and section assignments that follow belong to the active Part until *END PART closes it.

A Part is defined in its own local coordinate system. Its node coordinates describe the undeformed component geometry before any Instance placement. The Part itself has no global translation or rotation. If the same component is needed in several locations, define its topology once and create several *INSTANCE entries.

Part-local IDs are independent. Node 0 in Part LEFT and node 0 in Part RIGHT are different nodes even though they use the same numerical label. The same applies to element IDs. This is one of the main practical advantages of the Part representation: imported or reusable components do not need to be renumbered merely because another Part uses the same IDs.

Explicit Parts are optional in native FEMaster input. If nodes and elements are written directly at root level, they belong to the flat model topology and can be used without writing *PART, *ASSEMBLY, or *INSTANCE.

Part-level sets, surfaces, and Section assignments follow the Part when it is instantiated. Part-level definitions therefore describe properties intrinsic to every placed copy, whereas Assembly-level definitions can select or combine particular Instances.

Parameters

Parameter	Status	Meaning
NAME	required	Unique Part name. The name is referenced by *INSTANCE.

FEMaster and Abaqus example

FEMaster

```
*PART, NAME=ROD
*NODE, NSET=PNODES
0,   0., 0., 0.
1, 100., 0., 0.
*ELEMENT, TYPE=T3, ELSET=BAR
0, 0, 1
*END PART
```

Abaqus input

```
*PART, NAME=ROD
*NODE, NSET=PNODES
0,   0., 0., 0.
1, 100., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
0, 0, 1
*END PART
```

4.4 *END PART

*END PART closes the active Part scope. Definitions that follow are no longer part-local unless another *PART is opened. The command is distinct from native *END, which terminates a native loadcase.

Syntax and semantics

Syntax

```
*END PART
```

FEMaster and Abaqus example

FEMaster

```
*PART, NAME=PLATE
...
*END PART
```

Abaqus input

```
*PART, NAME=PLATE
...
*END PART
```

4.5 *ASSEMBLY

*ASSEMBLY begins the optional Assembly scope. The Assembly is where placed Instances and regions that refer to the assembled model are defined. A model that does not need explicit Instances may omit this scope entirely.

The Assembly combines several Parts or repeated placements of one Part. Part-level topology remains expressed with local IDs, while Assembly-level sets and surfaces select entities from particular Instances. This preserves local numbering inside each component without forcing a single manually renumbered global mesh.

The Assembly organizes the placed model but is not required by the solver. A flat native model and a structured Assembly represent alternative organizational forms for the same finite-element topology.

NAME is optional and defaults to ASSEMBLY. The name is descriptive for the supported current workflow; Instances and assembly-level regions are the referenced objects inside the Assembly.

Parameters

Parameter	Status	Meaning
NAME	optional	Assembly name; default ASSEMBLY.

FEMaster and Abaqus example

FEMaster

```
*ASSEMBLY, NAME=ASSEMBLY
*INSTANCE, NAME=ROD_A, PART=ROD
0., 0., 0.
*END INSTANCE
*END ASSEMBLY
```

Abaqus input

```
*ASSEMBLY, NAME=ASSEMBLY
*INSTANCE, NAME=ROD_A, PART=ROD
0., 0., 0.
*END INSTANCE
*END ASSEMBLY
```

4.6 *END ASSEMBLY

*END ASSEMBLY closes the Assembly scope.

Syntax and semantics

Syntax

```
*END ASSEMBLY
```

FEMaster and Abaqus example

FEMaster

```
*ASSEMBLY
...
*END ASSEMBLY
```

Abaqus input

```
*ASSEMBLY, NAME=ASSEMBLY
...
*END ASSEMBLY
```

4.7 *INSTANCE

*INSTANCE places an existing Part in the Assembly. Native FEMaster intentionally accepts the same placement representation used by the supported Abaqus reader. NAME gives the Instance a unique assembly name and PART selects the reusable Part.

An Instance may be placed by translation, rotation, both, or neither. A three-value data line gives a translation

$$\boldsymbol{t} = (t_x, t_y, t_z).$$

A seven-value line defines a rotation by two points on the rotation axis and an angle in degrees:

$$(x_1, y_1, z_1, x_2, y_2, z_2, \varphi).$$

The two axis points must be distinct. If both translation and rotation are supplied, the translation is applied first and the axis-angle rotation second. With no placement data the Part is instantiated in its original position.

Instance placement changes the position and orientation of the component in the assembled model; it does not require the Part coordinates or Part IDs to be rewritten. Several Instances may therefore reference the same Part with different placements.

The Part-local node and element labels remain associated with every Instance. Assembly-level sets use the Instance qualification to distinguish the same local ID in several placed copies.

Parameters

Parameter	Status	Meaning
NAME	required	Unique Instance name used for assembly-level references.
PART	required	Name of the Part to place.

Data lines

Entry	Meaning
tx, ty, tz	Optional translation vector.
x1,y1,z1,x2,y2,z2,angle	Optional rotation axis through two points and rotation angle in degrees.

FEMaster and Abaqus example

FEMaster

```
*INSTANCE, NAME=RIGHT, PART=BRACKET
100., 0., 0.
0., 0., 0., 0., 0., 1., 90.
*END INSTANCE
```

Abaqus input

```
*INSTANCE, NAME=RIGHT, PART=BRACKET
100., 0., 0.
0., 0., 0., 0., 0., 1., 90.
*END INSTANCE
```

4.8 *END INSTANCE

*END INSTANCE closes the current Instance definition.

FEMaster and Abaqus example

FEMaster

```
*INSTANCE, NAME=A, PART=BLOCK
20., 0., 0.
*END INSTANCE
```

Abaqus input

```
*INSTANCE, NAME=A, PART=BLOCK
20., 0., 0.
*END INSTANCE
```

4.9 Mesh definition

4.10 *NODE

*NODE defines finite-element nodes by user ID and Cartesian reference coordinates. In a Part scope the nodes belong to that Part. In a flat native model they belong directly to the model topology.

Node IDs are non-negative integers. **ID 0 is valid.** IDs need not be contiguous and may be chosen to preserve meaningful numbering from a mesh generator or external model. They must only be unique within the topology scope in which they are defined.

The optional NSET parameter adds every node in the block to a node set. Native FEMaster defaults this set to NALL. Coordinates are Cartesian reference coordinates. Missing trailing coordinate components default to zero; explicit three-component coordinates expose the complete position in shared examples.

Node order has no mechanical meaning by itself, but the order in which node IDs appear in an element connectivity is crucial because it determines the element's local topology and orientation.

The coordinates define the undeformed geometry. Loads, prescribed displacements, initial velocities, and Instance placement do not change the meaning of the node's Part-local reference coordinates.

Parameters

Parameter	Status	Meaning
NSET	optional	Node set receiving all nodes in the block; native default NALL.

Data lines

Entry	Meaning
id	Non-negative user node ID; 0 is valid.
x, y, z	Cartesian reference coordinates.

FEMaster and Abaqus example

FEMaster

```
*NODE, NSET=EDGE
0,  0., 0., 0.
10, 25., 0., 0.
20, 50., 0., 0.
```

Abaqus input

```
*NODE, NSET=EDGE
0,  0., 0., 0.
10, 25., 0., 0.
20, 50., 0., 0.
```

4.11 *ELEMENT

*ELEMENT defines element connectivity. Every data record begins with a user element ID followed by the node IDs in the formulation-specific connectivity order. TYPE selects the finite-element formulation. ELSET optionally adds all elements in the block to an element set and defaults to EALL in native FEMaster input.

Element IDs are non-negative user labels and may also begin at **0**. They may be sparse. Connectivity is written using node IDs from the same topology scope as the element. The connectivity order determines local edges and faces, shell normal direction, solid face labels, element orientation, and the mapping from the reference element to the physical geometry.

The complete formulation reference is given in Chapter 5. That chapter documents the supported solid, shell, beam, and truss elements individually, including node ordering, numerical behaviour, applicability, and the existing element schematics.

The native and Abaqus input names are not always identical. The solid labels C3D4, C3D5, C3D6, C3D8, C3D8R, C3D10, C3D15, C3D20, and C3D20R are shared. Native T3 corresponds to Abaqus T3D2. Abaqus shell labels S3/S3R, S4/S4R, S6/S6R, and S8/S8R are read into the corresponding finite-rotation shell formulations. Native FEMaster additionally exposes the legacy shell types, MITC variants, FRT names, and QSPT directly.

An element definition provides topology and kinematics, but not the complete physical properties. Solids, shells, beams, and trusses require the corresponding Section definitions to obtain their intended material and geometric stiffness; Chapter 8 documents these assignments.

Parameters

Parameter	Status	Meaning
TYPE	required	Element formulation/type identifier.
ELSET	optional	Element set receiving all elements in the block; native default EALL.

Data lines

Entry	Meaning
id	Non-negative user element ID; 0 is valid.
n1, n2, ...	Node connectivity in the formulation-specific order.

FEMaster and Abaqus example

FEMaster

```
*ELEMENT, TYPE=T3, ELSET=BARS
0, 0, 10
1, 10, 20

*ELEMENT, TYPE=MITC4FRT, ELSET=PANEL
10, 100, 110, 120, 130
```

Abaqus input

```
*ELEMENT, TYPE=T3D2, ELSET=BARS
0, 0, 10
1, 10, 20

*ELEMENT, TYPE=S4, ELSET=PANEL
10, 100, 110, 120, 130
```

5 Elements

Elements define how the displacement field is interpolated between nodes and therefore determine the kinematic approximation used by the finite-element model. The element formulation controls which deformation modes can be represented, how strains and curvatures are evaluated, how numerical integration is performed, and which result quantities can be recovered. Element family and mesh resolution jointly determine the discrete approximation.

This chapter documents the element formulations that are currently accepted by FEMaster. It is intentionally more detailed than the `*ELEMENT` keyword page in Chapter 4: the keyword page explains how elements are entered, while this chapter explains what the individual formulations represent, how they behave numerically, and which applications match their kinematic assumptions.

Element IDs are non-negative user labels and may start at `0`. Connectivity order is significant. It determines the reference orientation, element faces, shell normals, and the mapping between the reference element and the physical geometry. A connectivity order different from the documented order defines different local topology and orientation.

5.1 Supported element types

FEMaster type	Nodes	Formulation
`C3D4`	4	Linear tetrahedral solid.
`C3D5`	5	Five-node pyramid input represented by a degenerated hexahedral topology.
`C3D6`	6	Linear wedge solid.
`C3D8`	8	Linear fully integrated hexahedral solid.
`C3D8R`	8	Reduced-integration hexahedral solid with hourglass stabilization.
`C3D10`	10	Quadratic tetrahedral solid.
`C3D15`	15	Quadratic wedge solid.
`C3D20`	20	Quadratic hexahedral solid with full stiffness integration.
`C3D20R`	20	Quadratic hexahedral solid with reduced stiffness integration.
`S3`	3	Legacy three-node triangular shell.
`S4`	4	Legacy four-node quadrilateral shell.
`MITC4`	4	Legacy four-node MITC shell with tied transverse shear interpolation.
`S6`	6	Legacy six-node quadratic triangular shell.
`S8`	8	Legacy eight-node quadratic quadrilateral shell.
`MITC8`	8	Legacy eight-node MITC shell with tied transverse shear interpolation.
`MITC3FRT`	3	Three-node finite-rotation shell.
`MITC4FRT`	4	Four-node finite-rotation MITC shell.
`MITC6FRT`	6	Six-node finite-rotation MITC shell.
`MITC8FRT`	8	Eight-node finite-rotation MITC shell.
`QSPT`	4	Specialized quadrilateral shear-panel element.

FEMaster type	Nodes	Formulation
B33	2	Three-dimensional beam element.
T3	2	Three-dimensional truss element.

The Abaqus reader accepts the same solid names C3D4, C3D5, C3D6, C3D8, C3D8R, C3D10, C3D15, C3D20, and C3D20R. Abaqus T3D2 is mapped to FEMaster T3. Abaqus B33 is mapped to FEMaster B33. Abaqus shell names are translated as shown below.

Abaqus input	FEMaster formulation	Comment
S3, S3R	MITC3FRT	Both imported labels use the finite-rotation triangular shell.
S4, S4R	MITC4FRT	Both imported labels use the finite-rotation four-node MITC shell.
S6, S6R	MITC6FRT	Both imported labels use the finite-rotation six-node shell.
S8, S8R	MITC8FRT	Both imported labels use the finite-rotation eight-node shell.

Table 5.2: Current Abaqus shell-type mapping.

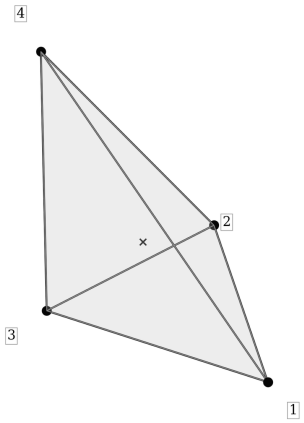
This mapping is important: an Abaqus S4 deck does *not* select the native legacy S4 formulation. It is translated to MITC4FRT. Native FEMaster input can still select the legacy shell names explicitly for comparison and regression work.

5.2 Solid elements

Solid elements model a three-dimensional continuum. They use translational nodal degrees of freedom and obtain their constitutive response from a *SOLID SECTION. They represent the full three-dimensional stress state, through-thickness stresses, and local geometry or load introduction that shell and line elements omit.

Low-order solids are inexpensive but may require significant refinement in bending-dominated problems. Higher-order solids generally represent curvature and stress gradients more efficiently, but they contain more degrees of freedom per element and are more sensitive to poor midside-node placement and distorted mappings. Mesh quality remains essential for every formulation.

5.3 C3D4: 4-node tetrahedral solid



Element	C3D4
Type	solid
Nodes	4
Faces	4
Edges	6
DOF / node	3
Total DOF	12
Shape functions	linear tetrahedral Lagrange
Integration points	1

Figure 5.1: C3D4 element topology and integration scheme.

Syntax

```
*ELEMENT, TYPE=C3D4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

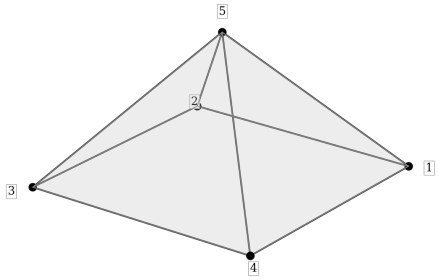
C3D4 is the simplest three-dimensional continuum element in FEMaster. It uses four corner nodes and linear displacement interpolation over a tetrahedral reference domain. The resulting strain field is constant within one element, which makes the element inexpensive and geometrically easy to mesh but limits the deformation and stress gradients that a single element can reproduce.

The principal advantage of C3D4 is meshing flexibility. Tetrahedra can fill complex CAD volumes, narrow transitions, and irregular local details with little manual topology design. This supports early stiffness checks, coarse load-path studies, and robust automatic meshing at the cost of stress accuracy per degree of freedom.

The limitation is stiffness and stress resolution. Linear tetrahedra tend to be overly stiff in bending-dominated problems unless the mesh is fine. Since the strain state is constant in each element, stress fields can appear piecewise constant or patchy and steep gradients near holes, fillets, notches, and concentrated loads require substantial refinement.

Compared with C3D4, C3D10 provides higher-order stress and bending resolution in tetrahedral meshes. The sensitivity of C3D4 results to refinement appears in displacements, reactions, strain energy, and averaged stresses, while a single coarse local peak is not a convergence measure.

5.4 C3D5: 5-node pyramid input



Element	C3D5
Type	solid
Nodes	5
Faces	-
Edges	8
DOF / node	3
Total DOF	15
Shape functions	pyramid input mapped to C3D8
Integration points	-

Figure 5.2: C3D5 pyramid input topology.

Syntax

```
*ELEMENT, TYPE=C3D5, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <apex>
```

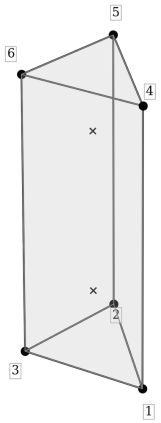
C3D5 provides a five-node pyramid connectivity for transition meshes. FEMaster represents this input with a degenerated eight-node hexahedral topology in which the four upper hexahedral positions coincide at the pyramid apex. The user nevertheless writes the natural five-node pyramid connectivity shown above.

The element primarily acts as a transition cell between hexahedral and tetrahedral regions. In automatically generated mixed meshes, it connects a quadrilateral face to tetrahedral topology without forcing a large surrounding region to change element family.

A degenerated mapping is generally less predictable than a regular hexahedron, wedge, or tetrahedron. Flat pyramids, badly skewed bases, and apex positions close to the base plane can produce poor Jacobians and unreliable stiffness. These properties limit C3D5 primarily to transition regions rather than large stress-critical volumes.

In models with many pyramids in the primary load path, reactions and global stiffness can remain plausible even when local stress quality differs significantly from an alternative regular-element mesh.

5.5 C3D6: 6-node wedge solid



Element	C3D6
Type	solid
Nodes	6
Faces	5
Edges	9
DOF / node	3
Total DOF	18
Shape functions	linear wedge / prism
Integration points	2

Figure 5.3: C3D6 wedge element.

Syntax

```
*ELEMENT, TYPE=C3D6, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>
```

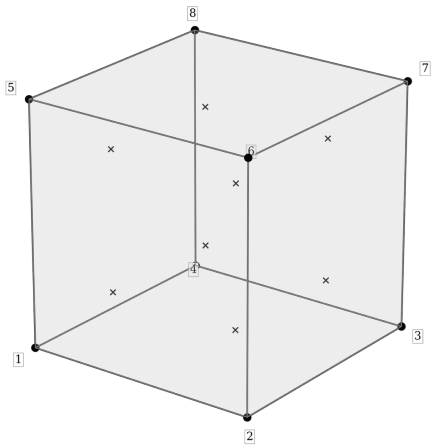
C3D6 is a linear wedge or prism element. It combines triangular end faces with quadrilateral side faces and represents swept meshes, extruded triangular surface meshes, layered regions, and transitions where a purely hexahedral topology is not possible.

A wedge mesh can align naturally with a dominant through-thickness or extrusion direction. In locally layered solids and prismatic regions, its element rows can follow the physical geometry more consistently than an unstructured tetrahedral mesh.

Like other low-order elements, C3D6 has limited bending and gradient resolution. One or two linear wedges through the thickness of a bending-dominated region provide limited stress recovery. Resolution increases with the number of elements in the gradient direction, while collapsed triangular faces, excessive twisting between end faces, and highly skewed side faces degrade the mapping.

C3D6 matches geometries with a natural wedge topology. C3D15 provides the corresponding higher-order interpolation for quadratic swept meshes.

5.6 C3D8: 8-node hexahedral solid



Element	C3D8
Type	solid
Nodes	8
Faces	6
Edges	12
DOF / node	3
Total DOF	24
Shape functions	trilinear Lagrange
Integration points	8

Figure 5.4: C3D8 hexahedral solid element.

Syntax

```
*ELEMENT, TYPE=C3D8, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

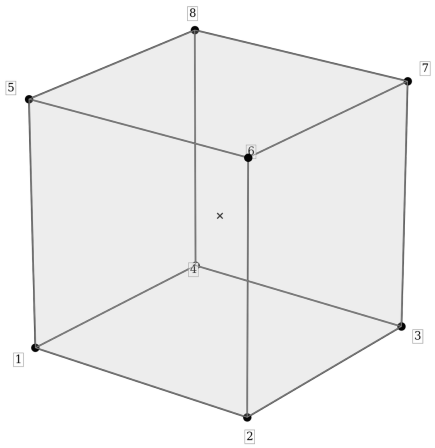
C3D8 is the standard low-order hexahedral continuum element. It uses trilinear displacement interpolation and full integration. For regular block-like, swept, or layered meshes it is often an efficient general-purpose solid because element directions can follow geometry and material orientation naturally.

Hexahedral topology provides direct control of the number of elements through thickness and along structural load paths. A regular C3D8 mesh therefore represents plates modelled with solids, bonded layers, spars, ribs, extruded components, and regions whose orthotropic material axes follow the mesh.

The element remains low order. A coarse mesh can be too stiff in bending, and straight element edges only approximate curved boundaries. Through-thickness resolution and refinement of curved or stress-critical geometry directly control the corresponding bending-stress and geometry errors.

Warping, skew, negative Jacobians, and extreme aspect ratios can remove the numerical advantages of the hexahedral topology. A clean quadratic tetrahedral mesh can provide a more accurate mapping than a badly distorted hexahedral mesh; topology alone does not determine mesh quality.

5.7 C3D8R: reduced-integration 8-node hexahedral solid



Element	C3D8R
Type	solid
Nodes	8
Faces	6
Edges	12
DOF / node	3
Total DOF	24
Shape functions	trilinear Lagrange with hourglass stabilization
Integration points	1

Figure 5.5: C3D8R reduced-integration hexahedral solid element.

Syntax

```
*ELEMENT, TYPE=C3D8R, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

C3D8R uses the same eight-node topology as C3D8 but evaluates the continuum contribution with one integration point at the element centre. Reduced integration lowers assembly cost and avoids some of the excessive stiffness that a fully integrated low-order hexahedron can exhibit in bending-dominated states.

One-point integration also introduces zero-energy hourglass deformation modes. FEMaster therefore adds hourglass stabilization. The primitive scalar hourglass patterns are based on the four non-affine modes

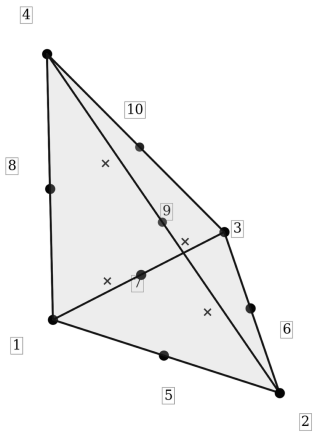
$$\gamma_1 = s\,t, \qquad \gamma_2 = t\,r, \qquad \gamma_3 = r\,s, \qquad \gamma_4 = r\,s\,t.$$

These modes are projected so that affine displacement fields are not penalized. The stabilization scale is based on the element reference volume, the reference geometry gradients, and an effective shear stiffness derived from the constitutive tangent.

Newton convergence alone does not characterize C3D8R; the deformation pattern and stabilization sensitivity are independent numerical properties. A coarse or badly distorted mesh can converge while exhibiting nonphysical hourglass deformation. Comparison with C3D8 and mesh refinement expose response components associated with hourglass-sensitive deformation.

Severe finite rotations and highly distorted elements increase sensitivity to reduced-integration stabilization. Relative to C3D8, C3D8R exchanges lower integration cost and locking tendency for dependence on hourglass control.

5.8 C3D10: 10-node quadratic tetrahedral solid



Element	C3D10
Type	solid
Nodes	10
Faces	4
Edges	6
DOF / node	3
Total DOF	30
Shape functions	quadratic tetrahedral Lagrange
Integration points	4

Figure 5.6: C3D10 quadratic tetrahedral element.

Syntax

```
*ELEMENT, TYPE=C3D10, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>, <n9>, <n10>
```

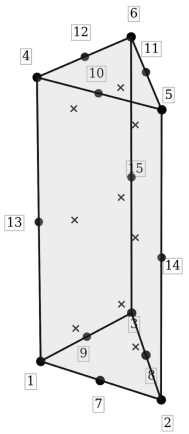
C3D10 is the quadratic tetrahedral solid. Six midside nodes augment the four corner nodes and allow quadratic displacement interpolation. Compared with C3D4, the element can represent curved deformation, bending, and stress gradients far more efficiently.

For automatic tetrahedral meshing, C3D10 retains the geometric flexibility of tetrahedra while reducing the excessive stiffness and piecewise-constant strain behaviour associated with linear tetrahedra.

The quadratic geometry mapping also represents curved boundaries more accurately when midside nodes follow the CAD geometry. Badly displaced midside nodes can create a distorted mapping even if the corner tetrahedron appears acceptable.

The element costs more per cell than C3D4, but substantially fewer cells may be required for comparable displacement and stress accuracy. Refinement is still required near singularities, sharp re-entrant corners, and concentrated loads where no polynomial element can make a mathematical singularity disappear.

5.9 C3D15: 15-node quadratic wedge solid



Element	C3D15
Type	solid
Nodes	15
Faces	5
Edges	9
DOF / node	3
Total DOF	45
Shape functions	quadratic wedge / prism
Integration points	9

Figure 5.7: C3D15 quadratic wedge element.

Syntax

```
*ELEMENT, TYPE=C3D15, ELSET=<set>
<id>, <n1>, ..., <n15>
```

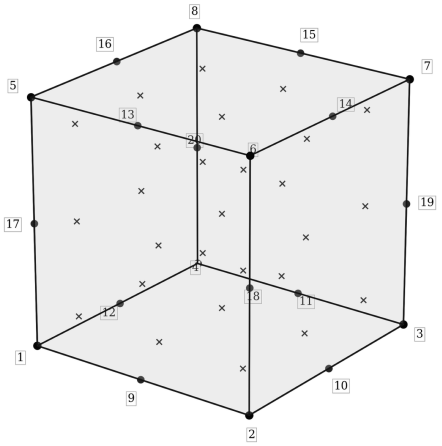
C3D15 is the quadratic counterpart of the six-node wedge. It adds midside nodes and represents curvature, bending, and through-thickness gradients at higher order than C3D6. Its topology matches higher-order swept meshes and prismatic regions created by extruding a quadratic triangular surface mesh.

The element combines wedge topology with higher-order through-thickness interpolation. Curved swept solids, local layered regions, and transition zones can therefore require fewer elements than with C3D6.

Higher-order mapping increases sensitivity to midside-node placement. Poorly shaped triangular end faces, excessive twisting of the side faces, and midside nodes that depart from the intended geometry degrade the mapping. A high-order wedge with a poor mapping is not automatically more accurate than a clean low-order mesh.

C3D15 represents quadratic wedge meshes. A structured quadratic hexahedral mesh provides the more regular C3D20 topology where geometry permits it, whereas C3D10 supports fully unstructured complex geometry.

5.10 C3D20: 20-node quadratic hexahedral solid



Element	C3D20
Type	solid
Nodes	20
Faces	6
Edges	12
DOF / node	3
Total DOF	60
Shape functions	quadratic serendipity hexahedral
Integration points	27

Figure 5.8: C3D20 quadratic hexahedral element.

Syntax

```
*ELEMENT, TYPE=C3D20, ELSET=<set>
<id>, <n1>, ..., <n20>
```

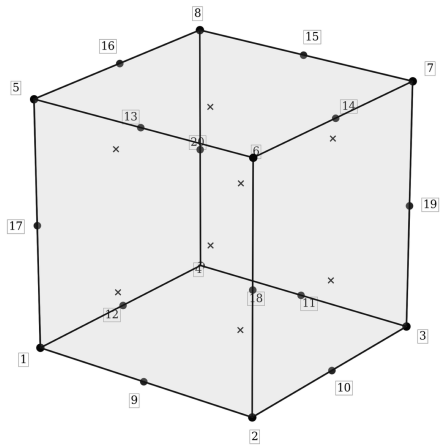
C3D20 is the full-integration quadratic hexahedral solid. The eight corner nodes are supplemented by twelve midside nodes, giving the element higher-order bending and stress-gradient representation than C3D8. Curved geometry is also represented at higher order when midside nodes follow the physical boundary.

For smooth structural fields and high-quality structured meshes, C3D20 provides high-order solid accuracy. Its quadratic hexahedral interpolation represents curved blocks, swept volumes, thick structural regions, and smooth stress gradients.

The main costs are larger element matrices and stricter geometric quality requirements. Distortion of a quadratic mapping can be more harmful than distortion of a linear one because the midside nodes participate directly in the geometry interpolation. Jacobian quality therefore depends on both corner and midside-node positions.

Relative to C3D8, C3D20 exchanges larger element matrices for higher-order accuracy. A fine regular C3D8 mesh can remain less expensive for models dominated by linear global stiffness.

5.11 C3D20R: 20-node reduced-integration hexahedral solid



Element	C3D20R
Type	solid
Nodes	20
Faces	6
Edges	12
DOF / node	3
Total DOF	60
Shape functions	quadratic serendipity hexahedral
Integration points	8

Figure 5.9: C3D20R reduced-integration quadratic hexahedral element.

Syntax

```
*ELEMENT, TYPE=C3D20R, ELSET=<set>
<id>, <n1>, ..., <n20>
```

C3D20R uses the same twenty-node topology and quadratic interpolation as C3D20, but the stiffness contribution is evaluated with reduced integration. The reduced rule lowers cost and can improve behaviour in states where full integration introduces excessive constraint or stiffness.

Reduced integration changes the numerical character of the element; it is not merely a faster switch. Nonphysical low-energy deformation modes and sensitivity to mesh distortion can appear in deformed shapes, reaction balance, strain-energy distribution, and comparison with C3D20.

The formulation combines a quadratic hexahedral mesh with reduced integration. C3D20 provides the corresponding fully integrated comparison formulation.

5.12 Shell elements

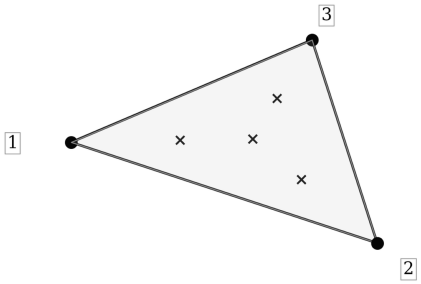
Shell elements reduce a three-dimensional thin structure to a two-dimensional midsurface with membrane, bending, and transverse-shear behaviour. Their kinematics represent structures whose thickness is small relative to the in-plane dimensions and whose result quantities are surface displacements, membrane forces, bending moments, transverse shear resultants, and top/bottom shell stresses rather than detailed three-dimensional through-thickness stress Fields.

Shell quality depends strongly on surface mesh quality, normal consistency, aspect ratio, element warping, and thickness definition. The shell normal determines the top/bottom convention and affects pressure direction and result signs. In curved shell models, inconsistent normals therefore change the interpretation of bending results.

Current and legacy shell families

The native shell elements `S3`, `S4`, `MITC4`, `S6`, `S8`, and `MITC8` are retained for legacy decks, verification, and comparison. The finite-rotation family consists of `MITC3FRT`, `MITC4FRT`, `MITC6FRT`, and `MITC8FRT`. The supported Abaqus shell labels are translated to this FRT family automatically.

5.13 S3: legacy 3-node triangular shell



Element	S3
Type	shell
Nodes	3
Faces	-
Edges	3
DOF / node	6
Total DOF	18
Shape functions	linear triangular shell
Integration points	4

Figure 5.10: S3 triangular shell topology.

Syntax

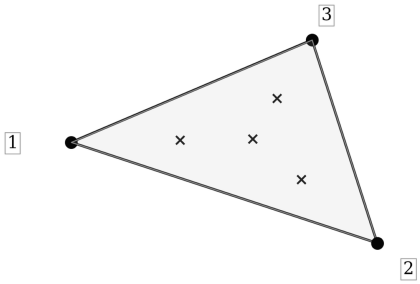
```
*ELEMENT, TYPE=S3, ELSET=<set>
<id>, <n1>, <n2>, <n3>
```

Native S3 is the legacy three-node triangular shell. Triangular topology fills irregular surface meshes and transition regions, but a three-node triangle provides only low-order interpolation and typically needs a finer mesh than a regular quadrilateral discretization for smooth bending Fields.

The element remains accepted so older FEMaster decks continue to run and legacy formulations can be compared with newer shells. MITC3FRT provides the corresponding current finite-rotation triangular formulation for geometrically nonlinear analyses and models with finite rotations.

Extremely acute angles, abrupt size changes, and inconsistent normal orientation degrade triangular shell meshes. A dense mesh of poor triangles can still give inferior bending behaviour compared with a smaller set of well-shaped elements.

5.14 MITC3FRT: finite-rotation 3-node shell



Element	MITC3FRT
Type	shell
Nodes	3
Faces	-
Edges	3
DOF / node	6
Total DOF	18
Shape functions	finite-rotation MITC3 shell
Integration points	4

Figure 5.11: MITC3FRT finite-rotation triangular shell.

Syntax

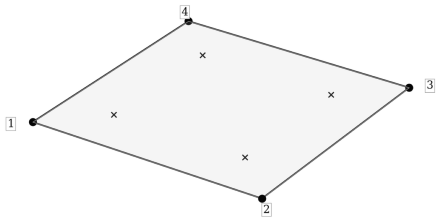
```
*ELEMENT, TYPE=MITC3FRT, ELSET=<set>
<id>, <n1>, <n2>, <n3>
```

MITC3FRT is the current three-node finite-rotation shell formulation. It retains the geometric flexibility of triangular shell meshes while using the FRT shell kinematics intended for geometrically nonlinear analysis.

Its triangular topology represents irregular shell boundaries, local mesh transitions, complex curved mid-surfaces, and imported surface meshes. As with every three-node shell, convergence in bending-dominated regions generally requires more elements than a high-quality quadrilateral mesh.

For imported Abaqus input, both S3 and S3R are translated to this formulation. The Abaqus suffix therefore does not select a separate reduced-integration implementation inside FEMaster.

5.15 S4: legacy 4-node quadrilateral shell



Element	S4
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	6
Total DOF	24
Shape functions	bilinear quadrilateral shell
Integration points	4

Figure 5.12: S4 quadrilateral shell topology.

Syntax

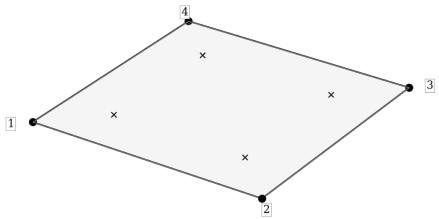
```
*ELEMENT, TYPE=S4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

Native S4 is the legacy four-node quadrilateral shell. Quadrilateral topology represents plates, webs, skins, and other surfaces with two natural in-plane directions, while this specific formulation retains legacy shell kinematics.

The low-order formulation can suffer severe transverse-shear locking in thin bending-dominated problems. A coarse S4 mesh may therefore appear dramatically too stiff while still satisfying equilibrium. Mesh refinement reduces this problem, while MITC interpolation addresses the shear constraint at formulation level.

MITC4FRT is the corresponding current finite-rotation four-node formulation. S4 remains available for compatibility, regression comparisons, and controlled studies of the legacy formulation.

5.16 MITC4: legacy 4-node shell with tied shear interpolation



Element	MITC4
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	6
Total DOF	24
Shape functions	bilinear MITC quadrilateral shell
Integration points	4

Figure 5.13: MITC4 shell with MITC transverse-shear interpolation.

Syntax

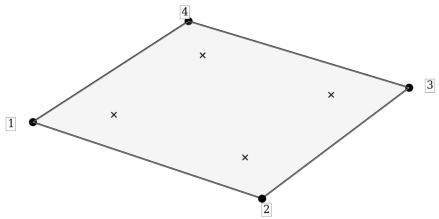
```
*ELEMENT, TYPE=MITC4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

MITC4 uses the same four-node topology as S4 but replaces the direct low-order transverse-shear interpolation with MITC-style tied shear interpolation. The purpose is to suppress the artificial shear constraint responsible for locking in thin bending problems.

This element historically provided substantially less shear locking than S4 on coarse thin-shell bending meshes. It remains available for regression and linear-formulation comparison. MITC4FRT combines the corresponding low-order MITC interpolation with the current finite-rotation shell formulation.

MITC interpolation does not make a quadrilateral insensitive to mesh quality. Excessive warping, skew, poor aspect ratio, and inconsistent normals can still degrade both stiffness and stress recovery.

5.17 MITC4FRT: finite-rotation 4-node MITC shell



Element	MITC4FRT
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	6
Total DOF	24
Shape functions	finite-rotation MITC4 shell
Integration points	4

Figure 5.14: MITC4FRT finite-rotation quadrilateral shell.

Syntax

```
*ELEMENT, TYPE=MITC4FRT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

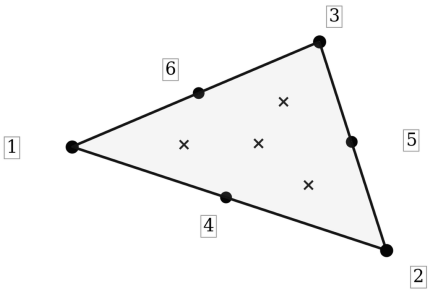
MITC4FRT is the current low-order quadrilateral shell formulation. It uses finite-rotation shell kinematics and MITC transverse-shear interpolation for thin-shell bending and geometrically nonlinear deformation with large rotations.

The four-node topology keeps mesh generation and element cost modest. It represents panels, webs, skins, shells of moderate curvature, and large shell assemblies on structured or mostly quadrilateral midsurface meshes.

Tied shear interpolation addresses shear locking but does not eliminate geometric discretization error. Curved-surface resolution depends on mesh density, strongly warped quadrilaterals degrade the mapping, and nonlinear load paths remain sensitive to deformation shape and refinement.

The supported Abaqus labels S4 and S4R are both translated to MITC4FRT.

5.18 S6: legacy 6-node triangular shell



Element	S6
Type	shell
Nodes	6
Faces	-
Edges	3
DOF / node	6
Total DOF	36
Shape functions	quadratic triangular shell
Integration points	4

Figure 5.15: S6 quadratic triangular shell.

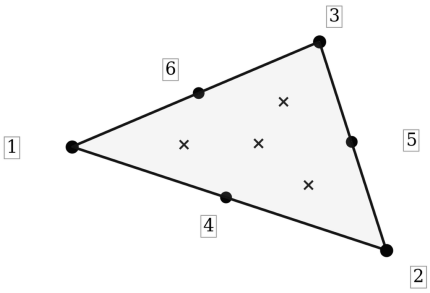
Syntax

```
*ELEMENT, TYPE=S6, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>
```

Native S6 is the legacy six-node quadratic triangular shell. The midside nodes improve geometric representation and allow a smoother in-plane and bending interpolation than S3. It remains available for older curved triangular shell meshes and comparisons with the corresponding FRT formulation.

MITC6FRT provides the corresponding current finite-rotation formulation. The higher-order triangle can represent curved boundaries effectively, but badly curved or highly skewed elements can create poor mappings despite the higher interpolation order.

5.19 MITC6FRT: finite-rotation 6-node shell



Element	MITC6FRT
Type	shell
Nodes	6
Faces	-
Edges	3
DOF / node	6
Total DOF	36
Shape functions	finite-rotation MITC6-b shell
Integration points	4

Figure 5.16: MITC6FRT finite-rotation six-node triangular shell.

Syntax

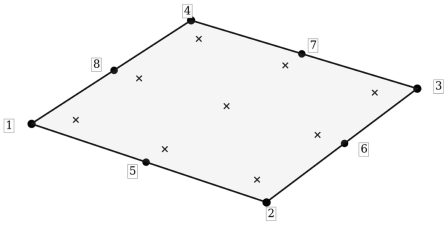
```
*ELEMENT, TYPE=MITC6FRT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>
```

MITC6FRT is the current six-node triangular finite-rotation shell. It combines quadratic triangular geometry with the FRT shell formulation and provides a higher-order triangular midsurface description.

The element represents curved shell regions with triangular topology at higher order than a three-node mesh. Midside nodes allow curved edges and smoother deformation fields when positioned consistently with the intended geometry.

Both Abaqus S6 and S6R are translated to this formulation by the current reader.

5.20 S8: legacy 8-node quadrilateral shell



Element	S8
Type	shell
Nodes	8
Faces	-
Edges	4
DOF / node	6
Total DOF	48
Shape functions	quadratic serendipity shell
Integration points	9

Figure 5.17: S8 quadratic quadrilateral shell.

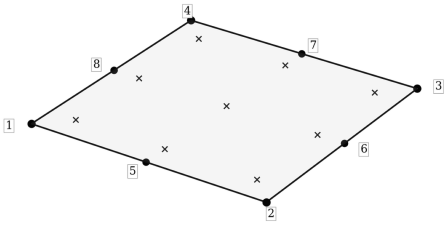
Syntax

```
*ELEMENT, TYPE=S8, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

Native S8 is the legacy eight-node quadratic quadrilateral shell. Four corner nodes and four midside nodes provide higher-order representation of curved geometry and smooth bending Fields than S4 when the element is well shaped.

The formulation remains available for older models and comparison studies. MITC8FRT provides the corresponding current finite-rotation formulation. Higher order does not remove sensitivity to midside-node placement, aspect ratio, and warping.

5.21 MITC8: legacy 8-node MITC shell



Element	MITC8
Type	shell
Nodes	8
Faces	-
Edges	4
DOF / node	6
Total DOF	48
Shape functions	quadratic MITC8 shell
Integration points	9

Figure 5.18: MITC8 quadratic MITC shell.

Syntax

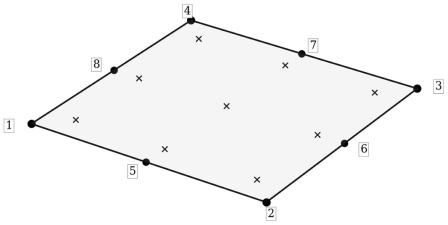
```
*ELEMENT, TYPE=MITC8, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

MITC8 combines the eight-node quadratic quadrilateral topology with MITC-style transverse-shear interpolation. It provides high-order thin-shell bending behaviour in the legacy linear shell family and serves as a reference formulation for comparison with lower-order shell results.

Because the interpolation is quadratic, a regular MITC8 mesh can reproduce smooth bending fields with relatively few elements. The cost per element is higher than for MITC4, and distorted midside geometry can reduce the expected advantage.

MITC8FRT provides the corresponding finite-rotation formulation for nonlinear and general-purpose shell models. Native MITC8 retains the legacy linear shell behaviour for comparison studies.

5.22 MITC8FRT: finite-rotation 8-node MITC shell



Element	MITC8FRT
Type	shell
Nodes	8
Faces	-
Edges	4
DOF / node	6
Total DOF	48
Shape functions	finite-rotation MITC8 shell
Integration points	9

Figure 5.19: MITC8FRT finite-rotation eight-node shell.

Syntax

```
*ELEMENT, TYPE=MITC8FRT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

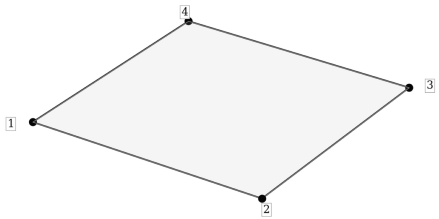
MITC8FRT is the current higher-order quadrilateral shell. It combines an eight-node midsurface interpolation with finite-rotation shell kinematics and MITC transverse-shear treatment. The additional interpolation order increases bending and curved-shell resolution together with element cost.

The formulation represents curved shell geometry with a quadratic quadrilateral mesh. Compared with a low-order shell, fewer elements may be required to describe smooth curvature and stress-resultant gradients.

Higher order does not make the formulation insensitive to distortion. Warped quadrilaterals, poorly placed midside nodes, inconsistent normals, and abrupt mesh transitions can still produce misleading stiffness and result recovery. Deformation shapes and refinement expose this sensitivity in large-rotation models.

Both Abaqus S8 and S8R are translated to MITC8FRT.

5.23 QSPT: quadrilateral shear-panel element



Element	QSPT
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	3
Total DOF	12
Shape functions	quadrilateral shear-panel formulation
Integration points	-

Figure 5.20: QSPT shear-panel element.

Syntax

```
*ELEMENT, TYPE=QSPT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

QSPT is a specialized four-node quadrilateral panel element developed for shear-flow-oriented structural idealizations. It is intended for simplified panel models where the principal purpose of the element is to transfer in-plane shear rather than reproduce the complete behaviour of a general shell.

Typical applications are preliminary thin-walled structures, webs, aircraft-style shear panels, and reduced-order structural models in which panel shear flow is the primary quantity of interest. In such models QSPT can provide a compact load-path representation without introducing a complete shell formulation for every panel.

QSPT is not a general replacement for MITC4FRT despite sharing a four-node quadrilateral topology. It does not provide the bending stiffness, local shell curvature, full shell resultants, or detailed top/bottom stresses of a shell formulation.

5.24 Beam and truss elements

Line elements idealize slender three-dimensional members by describing the cross-section through section properties instead of meshing it explicitly. This can reduce the number of degrees of freedom by orders of magnitude compared with shell or solid modelling. The trade-off is that local three-dimensional connection behaviour, detailed cross-section distortion, holes, welds, and local stress concentrations are not resolved.

5.25 B33: 3D beam element

Element	B33
Type	line
Nodes	2
Faces	-
Edges	1
DOF / node	6
Total DOF	12
Shape functions	linear 2-node beam
Integration points	2

Figure 5.21: B33 beam element.

Syntax

```
*ELEMENT, TYPE=B33, ELSET=<set>
<id>, <n1>, <n2>
```

B33 is a two-node three-dimensional beam element with translational and rotational nodal degrees of freedom. It represents axial deformation, bending, torsion, and the section response defined by *BEAM SECTION and the referenced *PROFILE.

The current *ELEMENT syntax uses exactly two connectivity nodes for B33. Beam cross-section orientation is supplied by the orientation vector in *BEAM SECTION; an additional orientation node is not part of the current connectivity grammar.

Beam elements represent slender frames, stiffeners, spars, ribs, idealized support members, and other structures through Section forces and global deformation. They include axial area, bending inertias, torsional stiffness, product of inertia, Section offsets, and reference-line offsets without meshing the full cross-section.

The beam idealization does not resolve local connection stresses, bolt or weld geometry, detailed cross-section deformation, or three-dimensional load introduction. Those effects require a corresponding shell or solid representation. The beam local axes determine the interpretation of bending moments, section forces, and profile properties.

The supported Abaqus reader accepts B33 with the same two-node connectivity.

5.26 T3: 2-node truss element

Element	T3
Type	line
Nodes	2
Faces	-
Edges	1
DOF / node	3
Total DOF	6
Shape functions	linear 2-node truss
Integration points	1

Figure 5.22: T3 truss element.

Syntax

```
*ELEMENT, TYPE=T3, ELSET=<set>
<id>, <n1>, <n2>
```

T3 is a two-node truss element. It carries axial force along its current line direction and uses translational nodal degrees of freedom. The material and cross-sectional area are assigned by *TRUSS SECTION.

The element represents pin-jointed members, rods, braces, links, and other components whose structural action is axial tension or compression. Its mechanical quantities are axial force, axial strain, and axial stress.

The limitations are deliberate. T3 has no bending stiffness, no torsional stiffness, and no rotational continuity. It cannot model fixed-end moments, eccentric frame action, or transverse stiffness of a physical beam; those behaviours require B33 or a shell/solid representation.

In Abaqus input the corresponding accepted type is T3D2. The two labels select the same FEMaster truss formulation through their respective input dialects.

5.27 Element formulation comparison

No element is universally optimal. The following relationships summarize the principal formulation differences:

- C3D10 provides higher-order tetrahedral stress resolution than C3D4;
- well-shaped hexahedra align naturally with structured and swept geometries;
- C3D5 provides transition topology between other solid families;
- reduced-integration solids introduce deformation modes and refinement sensitivity that solver convergence alone does not characterize;
- the FRT shell family provides finite-rotation kinematics, while the non-FRT shell types retain legacy formulations;
- shells omit detailed three-dimensional through-thickness stress in exchange for a midsurface representation of thin structures;

- beams and trusses impose one-dimensional kinematic assumptions on the represented member;
- reactions, deformed shapes, energy trends, and mesh convergence characterize the discretization error associated with element and mesh selection.

A solver can converge accurately to the discrete equations of a poor finite-element model. Algebraic convergence therefore remains distinct from discretization accuracy.

6 Regions and Selections

Regions give names to reusable parts of the mesh. They are used throughout a FEMaster model: Sections are assigned to element sets, supports and concentrated loads target nodes or node sets, pressure and traction loads act on surfaces, and constraints such as couplings, ties, and contact refer to named regions.

Region names can express modelling concepts rather than only lists of IDs. Names such as `ROOT`, `PRESSURE_FACE`, `DESIGN_DOMAIN`, or `CONTACT_SLAVE` expose the intent of later commands without repeating numerical identifiers throughout the deck.

Regions can be defined inside a Part or at Assembly level. A Part-level region belongs to that Part and is available for every Instance of the Part. An Assembly-level region refers to the placed model and represents a selection on one particular Instance or a combination of entities from several Instances. The optional `INSTANCE` parameter identifies the Instance whose local IDs are being referenced.

User node, element, and explicit surface IDs are non-negative labels and may start at `0`. Generated ranges therefore may also begin at zero.

6.1 *NSET

*NSET creates or extends a named node set. The same basic syntax is accepted by native FEMaster and by the supported Abaqus input format. NSET gives the set name; NAME is accepted as an alternative spelling. Data may contain explicit node references or generated arithmetic ranges.

Inside a Part, numerical entries refer to that Part's local node IDs. Separate Parts may therefore contain sets with the same numerical members without referring to the same physical nodes. If a Part is instantiated more than once, its Part-level node sets remain available for every copy.

At Assembly level, INSTANCE=<name> qualifies local IDs with one particular Instance. This is the most direct way to create a support or load region on one placed copy of a reusable Part. Assembly-level sets may also combine selections from different Instances by using the supported qualified references or by building larger sets from previously named regions.

With GENERATE, every data row contains

start, end, increment.

The end value is inclusive. The increment defaults to 1 and must not be zero. Positive and negative increments are supported, so generated sequences can run in either direction as long as the sign agrees with the requested range.

Part-level sets represent regions intrinsic to the component, such as a bolt row, root edge, or material domain, and are available on every Instance. Assembly-level sets represent placement-dependent selections, such as one support on a particular Instance or a region combining nodes from several components.

Parameters

Parameter	Status	Meaning
NSET / NAME	required	Name of the node set to create or extend.
INSTANCE	optional	Assembly-level qualifier for local node references.
GENERATE	optional flag	Interpret each data row as start, end, increment.

Data lines

Entry	Meaning
Explicit form	One or more node IDs or supported named/qualified node references.
Generated form	start, end, increment; the increment defaults to 1 and must be nonzero.

FEMaster and Abaqus example

FEMaster

```
*PART, NAME=BEAM
*NODE
0, 0., 0., 0.
1, 1., 0., 0.
2, 2., 0., 0.
3, 3., 0., 0.
*NSET, NSET=ENDS
0, 3
*NSET, NSET=ALLN, GENERATE
0, 3, 1
*END PART
```

Abaqus input

```
*PART, NAME=BEAM
*NODE
0, 0., 0., 0.
1, 1., 0., 0.
2, 2., 0., 0.
3, 3., 0., 0.
*NSET, NSET=ENDS
0, 3
*NSET, NSET=ALLN, GENERATE
0, 3, 1
*END PART
```

Assembly-level Instance selection

A typical Assembly selection looks like

Syntax

```
*ASSEMBLY
*INSTANCE, NAME=BEAM_A, PART=BEAM
0., 0., 0.
*END INSTANCE
*INSTANCE, NAME=BEAM_B, PART=BEAM
100., 0., 0.
*END INSTANCE

*NSET, NSET=SUPPORT_A, INSTANCE=BEAM_A
0
*NSET, NSET=SUPPORT_B, INSTANCE=BEAM_B
0
*END ASSEMBLY
```

Both sets contain a local node with ID `0`, but they refer to different placed Instances.

6.2 *ELSET

*ELSET is the element-domain counterpart of *NSET. It creates or extends a named element set from explicit element references or generated ranges. ELSET supplies the name and NAME is accepted as an alternative.

Element sets are central to property assignment. Solid, shell, beam, and truss Sections normally target an *ELSET, so the element-set structure can mirror the physical construction of the model through separate names for different materials, thicknesses, beam profiles, or design domains.

Inside a Part, entries refer to that Part's local element IDs. At Assembly level, INSTANCE qualifies local IDs with one placed Instance. This allows two Instances of the same Part to be selected independently even though they use identical local element numbering.

GENERATE follows the same inclusive start/end/increment rule as *NSET. ID 0 is valid, so for example 0, 99, 1 generates one hundred element IDs if they exist in the intended topology.

A Section assigned to a Part-level element set belongs to the component and follows every Instance of the Part. Assembly-level element sets instead identify loads, diagnostics, or other selections whose meaning depends on the particular placed copy.

Parameters

Parameter	Status	Meaning
ELSET / NAME	required	Name of the element set to create or extend.
INSTANCE	optional	Assembly-level qualifier for local element references.
GENERATE	optional flag	Generate an arithmetic sequence of element IDs.

Data lines

Entry	Meaning
Explicit form	One or more element IDs or supported named/qualified element references.
Generated form	start, end, increment; inclusive end, nonzero increment.

FEMaster and Abaqus example

FEMaster

```
*ELSET, ELSET=DESIGN_DOMAIN, GENERATE
0, 99, 1

*ELSET, ELSET=EVEN_ELEMENTS, GENERATE
0, 98, 2
```

Abaqus input

```
*ELSET, ELSET=DESIGN_DOMAIN, GENERATE
0, 99, 1

*ELSET, ELSET=EVEN_ELEMENTS, GENERATE
0, 98, 2
```

6.3 *SURFACE

*SURFACE defines a geometric boundary from element sides. A surface is more than a node set: it retains which element face is selected and therefore has a well-defined interpolation, area or line measure, and orientation. Surface information is required for pressure loads, distributed tractions, surface couplings, ties, and contact.

A side is selected by an element or element-set target together with a side label such as S1, S2, and so on. The actual face represented by a side label depends on the element type and its connectivity. Reversing or changing the element node order can therefore change the physical direction of a surface normal.

For shell elements the two sides of the midsurface can also be described by SPOS and SNEG. These represent the positive and negative shell sides that determine pressure direction and top/bottom interpretation.

Native FEMaster additionally supports an explicit surface-ID form in which a user surface ID is supplied together with the parent element and side. Explicit surface IDs are non-negative and may start at 0. A surface named directly from an element or element set avoids the separate numerical surface-ID space of the explicit form.

At Assembly level, INSTANCE can qualify the target element or element set. This allows a face of one particular Instance to be selected even when several copies use the same local element IDs and set names.

The Abaqus example uses the supported element-based *SURFACE, TYPE=ELEMENT syntax. Only the specialized surface features documented here are included in the supported Abaqus subset.

An unordered node set contains no element-side information and therefore cannot define a pressure or contact normal. The selected element side determines the surface orientation. For shell models, normal consistency additionally controls the positive and negative shell faces and the sign convention of bending-related quantities.

Parameters

Parameter	Status	Meaning
SFSET / NAME	optional	Name of the destination surface region; native default SFALL.
INSTANCE	optional	Assembly-level Instance qualifier.

Data lines

Entry	Meaning
Target form	element-or-elset, side.
Explicit native form	surface_id, element_id, side.

FEMaster and Abaqus example

FEMaster

```
*SURFACE, NAME=TOP
SOLIDS, S2

*SURFACE, NAME=ONE_FACE
0, 42, S1
```

Abaqus input

```
*SURFACE, NAME=TOP, TYPE=ELEMENT
SOLIDS, S2
```

6.4 *SFSET

*SFSET groups existing native FEMaster surface entities into a named surface region. It combines surfaces created with explicit user surface IDs or several previously defined surface groups into one target.

*SURFACE and *SFSET therefore serve different purposes: *SURFACE creates geometric boundary entities from elements, while *SFSET collects already existing surface entities. A direct named *SURFACE does not require the separate numerical surface-ID space used by *SFSET.

The destination name is supplied by SFSET or NAME; the native default is SFALL. Explicit surface references can be listed directly. With GENERATE, arithmetic ranges of surface IDs can be produced using the same inclusive start/end/increment convention as node and element sets. INSTANCE can be used for Assembly-level selection where applicable.

The supported Abaqus input format does not have a separate *SFSET keyword. A named Abaqus *SURFACE already serves as the reusable surface region, so the right-hand example expresses the same modelling intent directly through one combined surface definition.

Parameters

Parameter	Status	Meaning
SFSET / NAME	optional	Destination surface-region name; native default SFALL.
INSTANCE	optional	Assembly-level Instance qualifier.
GENERATE	optional flag	Generate an arithmetic sequence of surface IDs.

Data lines

Entry	Meaning
Explicit form	Existing surface IDs or supported named surface references.
Generated form	start, end, increment; inclusive end, nonzero increment.

FEMaster and Abaqus example

FEMaster

```
*SURFACE, NAME=PATCH_A
0, 20, S1
1, 21, S1
*SURFACE, NAME=PATCH_B
2, 22, S1

*SFSET, NAME=CONTACT_FACE
0, 1, 2
```

Abaqus input

```
*SURFACE, NAME=CONTACT_FACE, TYPE=ELEMENT
20, S1
21, S1
22, S1
```

7 Materials

Materials define constitutive and inertial properties independently of the mesh. A material is created once, given a name, and referenced by one or more Sections. The same material may therefore be used by several element regions, Parts, or Instances without duplicating its constants.

`*MATERIAL` begins a material definition. The material-property keywords that follow—for example `*ELASTIC`, `*HYPERELASTIC`, `*DENSITY`, and thermal expansion—belong to that material until the material definition ends with the next appropriate root-level model definition. A material may contain only the properties required by the intended analyses; for example, density is unnecessary for a purely elastic static stiffness calculation but is required when mass participates in the mechanics.

7.1 *MATERIAL

*MATERIAL creates a named material and makes it the current target for subsequent material-property keywords. Native FEMaster accepts the identifier through MATERIAL or NAME; Abaqus input uses NAME.

The command itself does not define a constitutive law. Elasticity, hyperelasticity, density, and thermal expansion are added with separate keywords. This makes the material definition modular and keeps the physical meaning of each property explicit.

Material names must be unique. Sections reference the material by name, so one definition can be reused throughout the model.

Materials are independent of Parts. The corresponding Section assignment connects a material to a particular mesh region.

Parameters

Parameter	Status	Meaning
MATERIAL / NAME	required	Unique material identifier.

FEMaster and Abaqus example

FEMaster

```
*MATERIAL, NAME=STEEL
*ELASTIC, TYPE=ISOTROPIC
210000., 0.3
*DENSITY
7.85e-9
```

Abaqus input

```
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*DENSITY
7.85e-9
```

7.2 *ELASTIC

*ELASTIC defines a linear-elastic constitutive law. Native FEMaster supports isotropic elasticity, generalized isotropic elasticity with an independent shear modulus, orthotropic engineering constants, and an orthotropic stiffness-coefficient form. The supported Abaqus input covers the corresponding documented Abaqus forms.

Isotropic elasticity

For TYPE=ISOTROPIC or ISO, the data are Young's modulus E and Poisson's ratio ν. This is the default form. The shear modulus is

$$G = \frac{E}{2(1+\nu)},$$

and the three-dimensional isotropic constitutive law follows from E and ν in the usual way.

For ordinary stable isotropic elasticity, $E > 0$ and $-1 < \nu < 0.5$. Values very close to $\nu = 0.5$ describe nearly incompressible behaviour and may require element formulations and meshes that can represent that regime without excessive volumetric locking.

Generalized isotropic elasticity

Native FEMaster also accepts GENERALISEDISOTROPIC, GENERALISED_ISOTROPIC, or GENISO. This form reads E, ν, and an independent G. It preserves the isotropic-style normal relation while allowing the shear response to differ from the classical identity $G = E/[2(1+\nu)]$.

This is a FEMaster-specific convenience for constitutive models with a physically independent shear modulus. It does not compensate for an inappropriate element formulation or mesh.

Orthotropic engineering constants

TYPE=ENGINEERINGCONSTANTS defines the nine constants

$$E_1, E_2, E_3, \nu_{12}, \nu_{13}, \nu_{23}, G_{12}, G_{13}, G_{23}.$$

These quantities are expressed in the material coordinate system. A Section orientation therefore determines which physical directions of the model correspond to material axes 1, 2, and 3.

The Poisson ratios are directional. For example, ν_{12} describes strain in direction 2 caused by stress in direction 1. The reciprocal values follow from elastic symmetry, e.g.

$$\frac{\nu_{12}}{E_1} = \frac{\nu_{21}}{E_2}.$$

Transferred orthotropic data are compatible only when the axis ordering and Poisson-ratio convention agree with this definition.

Orthotropic stiffness coefficients

TYPE=ORTHOTROPIC or ORTHO accepts the nine coefficients

$$D_{1111}, D_{1122}, D_{2222}, D_{1133}, D_{2233}, D_{3333}, D_{1212}, D_{1313}, D_{2323}.$$

The first six values form the symmetric normal-stiffness block

$$\begin{bmatrix} D_{1111} & D_{1122} & D_{1133} \\ D_{1122} & D_{2222} & D_{2233} \\ D_{1133} & D_{2233} & D_{3333} \end{bmatrix},$$

while D_{1212}, D_{1313}, and D_{2323} are the three shear stiffnesses. The normal block must be nonsingular. This form accepts constitutive stiffness coefficients directly rather than through engineering constants.

Elastic constants follow the unit system of the model. If stresses are measured in MPa, the elastic moduli and stiffness coefficients are also entered in MPa. FEMaster does not attach units to numerical values, so dimensional consistency is determined by the complete set of input quantities.

Parameters

Parameter	Status	Meaning
`TYPE`	optional	Elasticity form. Native default `ISOTROPIC`; supported forms and aliases are described above.

Data lines

Entry	Meaning
`E, nu`	Isotropic Young's modulus and Poisson ratio.
`E, nu, G`	Native generalized-isotropic form.
Engineering constants	$E_1, E_2, E_3, \nu_{12}, \nu_{13}, \nu_{23}, G_{12}, G_{13}, G_{23}$.
Orthotropic stiffness	$D_{1111}, D_{1122}, D_{2222}, D_{1133}, D_{2233}, D_{3333}, D_{1212}, D_{1313}, D_{2323}$.

FEMaster and Abaqus example

FEMaster

```
*MATERIAL, NAME=UD
*ELASTIC, TYPE=ENGINEERINGCONSTANTS
135000., 10000., 10000.,
0.30, 0.30, 0.40,
5000., 5000., 3500.
```

Abaqus input

```
*MATERIAL, NAME=UD
*ELASTIC, TYPE=ENGINEERING CONSTANTS
135000., 10000., 10000.,
0.30, 0.30, 0.40,
5000., 5000., 3500.
```

7.3 *HYPERELASTIC

*HYPERELASTIC defines the currently supported finite-strain Neo-Hooke material law. Native FEMaster accepts the Neo-Hooke selector through the documented spellings NEOHOOKE, NEO_HOOKE, and NEO-HOOKE. The supported Abaqus input uses the corresponding NEO HOOKE form.

The material is parameterized by C_{10} and D_1. C_{10} controls the deviatoric stiffness and D_1 controls compressibility. These parameters are distinct from Young's modulus and Poisson's ratio and do not accept E and ν as direct substitutes.

For the conventional compressible Neo-Hooke parameterization, the small-strain shear modulus is related to C_{10} by

$$\mu = 2C_{10}.$$

The precise relation between D_1 and bulk stiffness follows the Neo-Hooke energy convention used by the model. Converted test data or material cards are compatible only when they use the same parameter convention rather than merely matching parameter names.

Hyperelasticity is a finite-strain constitutive model and requires a nonlinear analysis and an element formulation capable of evaluating the finite-deformation response. Defining a hyperelastic material does not by itself make a linear static loadcase nonlinear.

For nearly incompressible rubber-like materials, the ratio between bulk and shear stiffness can become very large. Element formulation, mesh quality, nonlinear convergence, and contact penalty stiffness are then sensitive to the volumetric stiffness.

Parameters

Parameter	Status	Meaning
Neo-Hooke selector	required	Selects the supported Neo-Hooke constitutive model.

Data lines

Entry	Meaning
C10	Deviatoric Neo-Hooke coefficient.
D1	Compressibility coefficient.

FEMaster and Abaqus example

FEMaster

```
*MATERIAL, NAME=RUBBER
*HYPERELASTIC, NEOHOOKE
0.348, 5.0e-6
```

Abaqus input

```
*MATERIAL, NAME=RUBBER
*HYPERELASTIC, NEO HOOKE
0.348, 5.0e-6
```

7.4 *DENSITY

*DENSITY assigns mass density ρ to the current material. The keyword is shared by native FEMaster and the supported Abaqus input format.

Density contributes wherever the model requires mass. For an element occupying volume V, the total distributed mass scales with

$$m = \int_V \rho \, \mathrm{d}V.$$

The corresponding element mass matrix enters eigenfrequency and transient analyses, and density also affects gravity/body loads, inertial loading, and inertia-relief calculations where those features are used.

Density must be consistent with the chosen length, force, mass, and time units. A common source of dynamic errors is to enter geometry in millimetres and elastic modulus in MPa while leaving density in kg/m^3 without conversion. Such a model may have a plausible static stiffness but eigenfrequencies that are wrong by orders of magnitude.

A purely stiffness-based linear static analysis can omit density. Eigenfrequency, transient, inertia-relief, and mass-dependent loadcases require mass data from density, explicitly modelled concentrated masses, or both.

Data lines

Entry	Meaning
rho	Scalar mass density.

FEMaster and Abaqus example

FEMaster

```
*MATERIAL, NAME=STEEL
*DENSITY
7.85e-9
```

Abaqus input

```
*MATERIAL, NAME=STEEL
*DENSITY
7.85e-9
```

7.5 *THERMALEXPANSION

Native *THERMALEXPANSION defines one constant isotropic coefficient of thermal expansion α. The coefficient describes the free thermal strain produced by a temperature change. In the isotropic case,

$$\boldsymbol{\varepsilon}_{\mathrm{th}} = \alpha\,\Delta T\begin{bmatrix}1 & 1 & 1 & 0 & 0 & 0\end{bmatrix}^{T}.$$

Defining α alone does not load the structure; a temperature Field and reference temperature are required to create ΔT.

The coefficient has units of inverse temperature, for example K^{-1}. Temperature differences in kelvin and degrees Celsius are numerically identical, so either scale may be used for ΔT as long as the coefficient is consistent.

The current native property is scalar and isotropic. Direction-dependent or tabulated temperature-dependent expansion is not described by this keyword.

Thermal stress arises when free thermal expansion is restrained or when temperature varies spatially. A uniformly heated completely free body expands without developing stress in an ordinary thermoelastic model.

Data lines

Entry	Meaning
alpha	Constant isotropic coefficient of thermal expansion.

FEMaster and Abaqus example

FEMaster

```
*MATERIAL, NAME=ALUMINIUM
*THERMALEXPANSION
2.30e-5
```

Abaqus input

```
*MATERIAL, NAME=ALUMINIUM
*EXPANSION, TYPE=ISO
2.30e-5
```

7.6 *EXPANSION

*EXPANSION is the supported Abaqus-input spelling for the same constant isotropic thermal-expansion property described above. TYPE is optional and accepts ISO or ISOTROPIC; the default is ISO.

The single data value is the coefficient α. This separate keyword page reflects the accepted Abaqus spelling; for the supported constant isotropic case, native *THERMALEXPANSION and Abaqus-style *EXPANSION describe the same mechanical thermal strain law.

Parameters

Parameter	Status	Meaning
TYPE	optional	ISO or ISOTROPIC; default ISO.

Data lines

Entry	Meaning
alpha	Constant isotropic coefficient of thermal expansion.

FEMaster and Abaqus example

FEMaster

```
*MATERIAL, NAME=ALUMINIUM
*THERMALEXPANSION
2.30e-5
```

Abaqus input

```
*MATERIAL, NAME=ALUMINIUM
*EXPANSION
2.30e-5
```

8 Sections and Element Properties

Element connectivity defines interpolation and kinematics, but it does not by itself provide a complete structural model. Sections connect element regions to material and geometric properties. For solids the Section selects a material and optionally a material orientation. Shells additionally require thickness or a generalized shell stiffness. Trusses require cross-sectional area. Beams combine material data with cross-section constants and a local section direction.

A Section defined for an element set inside a Part belongs to that Part. If the Part is instantiated several times, every Instance carries the same Section assignment. Material, thickness, area, and profile data thereby remain properties of the physical component, while the Instance placement determines where that component is located.

8.1 *PROFILE

`*PROFILE` defines the cross-section constants used by native FEMaster beam Sections. A Profile is referenced by name from `*BEAM SECTION`. It stores area, bending second moments, torsional constant, optional product of inertia, shear-centre offsets, and reference-line offsets.

The data order is

$$A,\ I_y,\ I_z,\ J_t,\ I_{yz},\ e_y,\ e_z,\ \mathrm{ref}_y,\ \mathrm{ref}_z.$$

Only the first four values are required. The remaining quantities default to zero.

FEMaster uses the convention

$$I_{yz} = \int_A yz \, \mathrm{d}A$$

without a leading minus sign. Some texts and tools define the product of inertia with the opposite sign, so imported Section properties can differ by this convention.

The offsets e_y, e_z describe the relative position used for the section/shear-centre definition, while $\mathrm{ref}_y, \mathrm{ref}_z$ describe the location of the beam reference line relative to the section reference point. In unsymmetric or eccentric Sections these offsets determine the different points about which axial force, shear, bending, and torsion act.

Profile constants follow the unit system of the model. In a consistent N–mm model, A is in mm^2, I_y, I_z, J_t, and I_{yz} are in mm^4, and all offsets are in mm.

Parameters

Parameter	Status	Meaning
`PROFILE` / `NAME`	required	Unique Profile name referenced by `*BEAM SECTION`.

Data lines

Entry	Meaning
`A`	Cross-sectional area.
`Iy, Iz`	Second moments of area about the local section axes.
`Jt`	Torsional constant.
`Iyz`	Product of inertia using $I_{yz} = \int_A yz \, \mathrm{d}A$.
`ey, ez`	Optional section/shear-centre offsets; default zero.
`refy, refz`	Optional reference-line offsets; default zero.

FEMaster and Abaqus example

FEMaster

```
*PROFILE, NAME=RECT_EQUIV
200., 6666.67, 1666.67, 4577.,
0., 0., 0., 0., 0.
```

Abaqus input

The supported Abaqus reader currently has no standalone Profile mapping for the native FEMaster beam-profile definition.

8.2 *SOLID SECTION

*SOLID SECTION assigns material properties to a solid element region. Native FEMaster requires ELSET and MATERIAL; MAT is accepted as an alias. ORIENTATION is optional and defines the local basis in which directional material properties are interpreted.

No thickness or area is required because a solid element represents its physical volume directly through the mesh geometry. The Section determines the constitutive law of the elements and the material coordinate system in which an anisotropic law is evaluated.

For isotropic elasticity, changing the material orientation has no effect on the constitutive response because the stiffness is direction independent. For orthotropic or fully anisotropic material data, the orientation is physically significant and must be chosen consistently with the material axes.

The supported Abaqus *SOLID SECTION syntax is identical for ordinary solid regions. FEMaster also accepts a special Abaqus-reader use of *SOLID SECTION for a pure T3D2 truss set; in that case the data value is interpreted as the truss cross-sectional area. This special case is described again under *TRUSS SECTION.

A Section applies one material system and orientation to its target element set. Different material systems or orientations within one geometric component therefore require separate named element sets and Section assignments. A target set containing incompatible element families does not define a uniform Section interpretation.

Parameters

Parameter	Status	Meaning
ELSET	required	Solid element set receiving the Section.
MATERIAL / MAT	required	Material assigned to the region.
ORIENTATION	optional	Named material coordinate system.

FEMaster and Abaqus example

FEMaster

```
*SOLID SECTION, ELSET=SOLID,
 MATERIAL=STEEL,
 ORIENTATION=MAT_AXES
```

Abaqus input

```
*SOLID SECTION, ELSET=SOLID,
 MATERIAL=STEEL,
 ORIENTATION=MAT_AXES
```

8.3 *SHELL SECTION

*SHELL SECTION assigns thickness and constitutive properties to shell elements. Native FEMaster supports two forms:

- TYPE=INTEGRATED, where a material law is integrated through the physical shell thickness;
- TYPE=ABD, where the generalized membrane–bending and transverse-shear stiffnesses are supplied directly.

For TYPE=INTEGRATED, ELSET identifies the shell region and MATERIAL identifies the constitutive law. The data line supplies the physical shell thickness t. The shell uses this thickness to form membrane, bending, shear, mass, and stress-recovery quantities according to the shell formulation.

For a homogeneous linear elastic shell, the familiar generalized relation has the form

$$\begin{bmatrix} \boldsymbol{N} \\ \boldsymbol{M} \end{bmatrix} = \begin{bmatrix} A & B \\ B & D \end{bmatrix} \begin{bmatrix} \boldsymbol{\varepsilon}_0 \\ \boldsymbol{\kappa} \end{bmatrix},$$

with additional transverse-shear resultants governed by a 2×2 shear stiffness. In an integrated Section these generalized stiffnesses arise from the material and thickness. In an ABD Section they are entered directly.

For TYPE=ABD, FEMaster reads 40 numerical values: all 36 entries of the full 6×6 generalized stiffness matrix in row-major order, followed by the four entries of the 2×2 transverse-shear stiffness matrix. The matrix is not assumed to be symmetric from input; enter all values explicitly in the required order.

ORIENTATION selects the local Section/material coordinate system. The ABD matrix is interpreted in this local shell basis. This is critical for laminates and anisotropic panels: an ABD matrix entered in one material direction but assigned with another orientation describes a different structure. CSYSAXIS=1,2,3 selects which axis of the named coordinate system establishes the in-plane reference direction; the default is axis 1.

The supported Abaqus *SHELL SECTION form represents a homogeneous integrated Section. Its data line gives thickness and optionally the number of Simpson integration points. FEMaster currently supports five through-thickness points for this imported form; if the count is omitted, five are used. Layered composite layups and other Abaqus-specific shell-section options are not implied by this syntax.

The shell normal and in-plane Section direction together define the local shell basis in which N_{xx}, N_{yy}, M_{xx}, M_{yy}, top/bottom stresses, and an anisotropic ABD matrix are interpreted. On curved meshes both directions vary with the surface geometry. A 90-degree orientation difference can exchange strong and weak laminate directions while leaving the input syntactically valid.

Parameters

Parameter	Status	Meaning
TYPE	native optional	INTEGRATED (default) or ABD.
ELSET	required	Shell element set.
MATERIAL / MAT	integrated required	Material used by the integrated Section; optional for native ABD input.
THICKNESS	native optional	Nominal thickness parameter; default 1.0.
ORIENTATION	optional	Named Section/material coordinate system.
CSYSAXIS	native optional	Coordinate-system axis 1, 2, or 3 used for the in-plane reference direction; default 1.

Data lines

Entry	Meaning
Integrated FEMaster	One positive shell thickness.
ABD FEMaster	36 row-major generalized-stiffness entries followed by 4 row-major transverse-shear entries.
Abaqus homogeneous form	Thickness and optional integration-point count; the current supported count is 5.

FEMaster and Abaqus example

FEMaster

```
*SHELL SECTION, ELSET=PANEL,
 MATERIAL=ALUMINIUM,
 TYPE=INTEGRATED
2.0
```

Abaqus input

```
*SHELL SECTION, ELSET=PANEL,
 MATERIAL=ALUMINIUM
2.0, 5
```

8.4 *BEAM SECTION

*BEAM SECTION assigns material, profile, and orientation information to a native FEMaster beam element set. MATERIAL selects the constitutive law, PROFILE supplies cross-section constants, and the data line gives a nonzero direction vector used to construct the local beam cross-section axes.

The beam axis itself is determined by the element geometry from node 1 to node 2. The Section orientation vector must therefore provide a second direction that is not parallel to the beam axis. Together they establish the local cross-section basis in which I_y, I_z, I_{yz}, offsets, bending moments, and section forces are interpreted.

The orientation vector belongs to the beam reference geometry. It is not a prescribed nodal rotation and it is not a load direction. For a curved or multi-element beam region, consistency of this vector determines continuity of the local frame along the complete member.

The current supported Abaqus input subset does not provide the native FEMaster beam-profile/beam-section mapping. General Abaqus *BEAM SECTION cards are outside the supported subset unless a reader extension explicitly documents their syntax.

The local orientation determines which bending inertia acts about an applied-load direction. A beam can therefore have the specified area and inertias but a different bending stiffness when an unsymmetric profile or changing member direction produces different local axes.

Parameters

Parameter	Status	Meaning
MATERIAL / MAT	required	Beam material.
ELSET	required	Beam element set.
PROFILE	required	Previously defined native FEMaster Profile.

Data lines

Entry	Meaning
n1x, n1y, n1z	Nonzero direction used to define the initial local cross-section orientation.

FEMaster and Abaqus example

FEMaster

```
*PROFILE, NAME=BEAM_PROFILE
200., 6666.7, 1666.7, 4577.

*BEAM SECTION, ELSET=BEAMS,
 MATERIAL=STEEL, PROFILE=BEAM_PROFILE
0., 1., 0.
```

Abaqus input

The current supported Abaqus input subset has no mapping to FEMaster's native Profile-based *BEAM SECTION definition.

8.5 *TRUSS SECTION

Native *TRUSS SECTION assigns a material and a positive cross-sectional area to a truss element set. A truss carries only axial force, so cross-sectional area is the only geometric Section property required by the current formulation.

For an element of length L, area A, and Young's modulus E, the small-strain axial stiffness scales as

$$k = \frac{EA}{L}.$$

The same area also determines material mass per unit length through ρA and converts axial force to average axial stress through $\sigma = N/A$.

The area must therefore use the same unit system as the geometry and material constants and must be strictly positive.

The supported Abaqus reader does not use a literal *TRUSS SECTION keyword. Instead, a pure T3D2 element set can be assigned through the supported *SOLID SECTION syntax with the cross-sectional area on the data line. The resulting truss mechanics are the same even though the input spelling differs. This mapping is specific to the FEMaster-supported pure T3D2 region and does not describe other Abaqus truss-section workflows.

Parameters

Parameter	Status	Meaning
MATERIAL / MAT	required	Truss material.
ELSET	required	Truss element set.

Data lines

Entry	Meaning
area	Strictly positive cross-sectional area.

FEMaster and Abaqus example

FEMaster

```
*TRUSS SECTION, ELSET=BARS,
 MATERIAL=STEEL
100.0
```

Abaqus input

```
*SOLID SECTION, ELSET=BARS,
 MATERIAL=STEEL
100.0
```

9 Coordinate Systems and Orientations

Coordinate systems define local directions used by materials, Sections, loads, supports, and nodal transformations. They determine the interpretation of anisotropy, beam or shell local axes, cylindrical loading, and boundary-condition components expressed outside the global Cartesian basis.

`*ORIENTATION` and `*TRANSFORM` act on different concepts. `*ORIENTATION` defines a named coordinate system that can be referenced by Sections and other orientation-aware native FEMaster features. Abaqus `*TRANSFORM` assigns a local degree-of-freedom basis to a node set. Neither command moves the geometry; rigid placement of a Part is defined by `*INSTANCE`.

9.1 *ORIENTATION

Native *ORIENTATION creates a named rectangular or cylindrical coordinate system. NAME identifies the system and TYPE selects RECTANGULAR or CYLINDRICAL.

A rectangular coordinate system describes a fixed orthogonal basis. FEMaster accepts one, two, or three supplied direction vectors, corresponding to three, six, or nine scalar values. In most user-written models, two non-collinear directions are the clearest representation: the first vector defines the first local axis and the second vector defines the plane in which the second local axis lies. The final orthogonal triad is then determined from those directions.

For example, the vectors

$$\boldsymbol{a} = (1, 0, 0), \qquad \boldsymbol{b} = (0, 1, 0)$$

define a local basis aligned with the global Cartesian axes. Rotating these vectors rotates the local material or Section basis without changing any node coordinates.

A cylindrical coordinate system defines an axis and corresponding radial/circumferential directions. Unlike a rectangular system, the radial and circumferential directions depend on the position at which the system is evaluated. This represents axisymmetric-like components, cylindrical material directions, and directional quantities that follow a curved geometry.

The supported Abaqus *ORIENTATION form uses DEFINITION=COORDINATES and SYSTEM=RECTANGULAR. Its data contain points a, b, and optionally c. The vector $a - c$ defines the first direction and $b - c$ defines the second in-plane direction. If c is omitted, the global origin is used. The points must define nonzero, non-collinear directions.

A material orientation changes the basis in which directional material constants are interpreted. For orthotropic elasticity, local axes 1, 2, and 3 correspond to the directions associated with E_1, E_2, E_3, G_{12}, G_{13}, G_{23}, and the Poisson-ratio convention of the material definition.

For shell Sections, the selected coordinate-system axis is projected into the shell plane to establish the local in-plane material/Section direction. The shell normal completes the local shell basis. Both directions determine the interpretation of anisotropic stiffness and shell resultants on curved shells.

Parameters

Parameter	Status	Meaning
NAME	required	Coordinate-system name used by Sections and other orientation-aware features.
TYPE	native required	RECTANGULAR or CYLINDRICAL.
DEFINITION	native optional	Accepted legacy parameter.
DEFINITION	Abaqus optional	COORDINATES; default COORDINATES.
SYSTEM	Abaqus optional	RECTANGULAR; default RECTANGULAR.

Data lines

Entry	Meaning
Native rectangular	3, 6, or 9 scalar components defining one to three directions.
Native cylindrical	9 scalar components defining the cylindrical system geometry.
Abaqus rectangular	$a_1, a_2, a_3, b_1, b_2, b_3, c_1, c_2, c_3$; c defaults to the origin.

FEMaster and Abaqus example

FEMaster

```
*ORIENTATION, NAME=MAT_AXES,
 TYPE=RECTANGULAR
1., 0., 0.,
0., 1., 0.
```

Abaqus input

```
*ORIENTATION, NAME=MAT_AXES,
 DEFINITION=COORDINATES,
 SYSTEM=RECTANGULAR
1., 0., 0., 0., 1., 0., 0., 0., 0.
```

9.2 *TRANSFORM

*TRANSFORM is supported in Abaqus input and assigns a local degree-of-freedom basis to a node set. It does not exist as a native FEMaster keyword because native FEMaster loads and supports can reference named orientations directly where a local basis is required.

NSET identifies the nodes that use the transformation. TYPE=R selects a rectangular transform and is the default. The six data values define two directions or points that establish the local rectangular basis. The directions must be nonzero and non-collinear.

TYPE=C selects a cylindrical transform. The two three-component points a and b define the cylindrical axis through

$$\boldsymbol{e}_z \parallel \boldsymbol{b} - \boldsymbol{a}.$$

The points must therefore be distinct. At each transformed node, the local radial and circumferential directions follow from the node position relative to that axis.

The transform affects how nodal vector components are interpreted. For example, a *BOUNDARY or *CLOAD component applied to a transformed node is expressed in the node's local transformed basis rather than directly in the global Cartesian basis. The transform does not rotate or translate the mesh itself.

A node has one effective nodal basis. Overlapping transformed sets with incompatible coordinate systems make that basis ambiguous.

*TRANSFORM differs from *INSTANCE: *INSTANCE changes the actual placement of a Part, whereas *TRANSFORM leaves the geometry unchanged and only changes the local basis used to interpret generalized nodal components.

Parameters

Parameter	Status	Meaning
NSET	required	Node set receiving the local nodal transformation.
TYPE	optional	R for rectangular or C for cylindrical; default R.

Data lines

Entry	Meaning
a1,a2,a3,b1,b2,b3	Two directions/points defining the rectangular basis or cylindrical axis.

FEMaster and Abaqus example

FEMaster

Native FEMaster does not use a separate *TRANSFORM keyword. Loads, supports, Sections, and other supported features reference named *ORIENTATION definitions directly where needed.

Abaqus input

```
*TRANSFORM, NSET=CYLINDER_NODES, TYPE=C
0., 0., 0., 0., 0., 100.
```

10 Fields

Fields store spatially varying data that belong to the finite-element model. They are more general than sets: a set only answers whether an entity belongs to a region, whereas a Field stores one or more numerical values at every addressed location. Typical uses are topology variables, prescribed shell directors, initial conditions, and other data that vary from node to node, element to element, or within an element.

A Field has a *domain* and a fixed number of components. The domain determines what one row refers to. For ordinary nodal and elemental Fields the rows are addressed by the same user-defined node and element identifiers that are used elsewhere in the input deck. For element-local Fields, such as values at local nodes or integration points, an element is named first and the location inside that element is given by a zero-based local index.

User node and element identifiers are independent of these local indices. They are non-negative deck labels, may start at **0**, and do not have to be contiguous. For example, node IDs `0`, `10`, and `1000` are perfectly valid if they are unique in their scope. By contrast, the local node indices of a four-node element are `0`, `1`, `2`, and `3` because they refer to positions in that element's connectivity.

10.1 *FIELD

*FIELD creates and optionally populates a named Field. It is a native FEMaster command; the supported Abaqus input format currently has no generic equivalent. The command is accepted at model level and in the native *LOADCASE contexts documented by the consuming feature.

NAME identifies the Field. TYPE selects its domain, COLS specifies the number of numerical components per location, and FILL defines how entries are initialized before explicit data rows are applied. FILL=ZERO is the default. FILL=NAN initializes unspecified components with NaN so that missing data remain distinguishable from a physical zero.

The supported domains are:

- NODE: one row per addressed node;
- ELEMENT: one row per addressed element;
- ELEMENT_NODAL: values stored separately for the local nodes of an element;
- ELEMENT_IP: values stored at an element's integration points;
- ELEMENT_MP: values stored at material points associated with an integration point.

Aliases without underscores and the short forms IP and MP are accepted where documented. COLS must be positive and is currently limited to 64 components.

For NODE and ELEMENT, every data row begins with the node or element target followed by the Field components. A target may be an ordinary user ID or, for an assembled model, an instance-qualified reference such as INSTANCE.ID. Empty component entries leave the existing value unchanged. This permits sparse updates of a Field that was first initialized with FILL.

For ELEMENT_NODAL, every row starts with an element reference and a zero-based local node index. The local index refers to the position in that element's connectivity, not to the numerical ID of the connected node. Thus, for a four-node element with connectivity 10,20,30,40, local node 0 refers to node 10, local node 1 to node 20, and so on.

ELEMENT_IP uses the same principle for integration points. The local integration-point index is zero based and must exist for the selected element formulation. ELEMENT_MP additionally specifies a zero-based material-point index. This representation allows different element formulations to expose different numbers of local storage positions without inventing global IDs for integration or material points.

Field values may be ordinary floating-point numbers and may also use NAN, +NAN, -NAN, INF, +INF, and -INF. Non-finite values represent partially prescribed or diagnostic Fields only where the consuming feature defines corresponding handling.

User IDs and element-local indices have different meanings. User node and element IDs may include 0 and may be sparse. Element-local node, integration-point, and material-point indices are positional indices beginning at 0.

Parameters

Parameter	Status	Meaning
NAME	required	Unique Field name.
TYPE	required	NODE, ELEMENT, ELEMENT_NODAL, ELEMENT_IP, or ELEMENT_MP.
COLS	required	Number of components per location; must be positive and is currently limited to 64.
FILL	optional	Initial value mode ZERO or NAN; default ZERO.

Data lines

Entry	Meaning
Node/element Field	`target, v1, v2, ....`
Element-nodal Field	`element, local_node, v1, v2, ...`; `local_node` is zero based.
Integration-point Field	`element, local_ip, v1, v2, ...`; `local_ip` is zero based.
Material-point Field	`element, local_ip, local_mp, v1, v2, ...`; both local indices are zero based.

FEMaster and Abaqus example

FEMaster

```
*FIELD, NAME=RHO, TYPE=ELEMENT,
 COLS=1, FILL=ZERO
0, 0.8
10, 1.0

*FIELD, NAME=DIRECTORS,
 TYPE=ELEMENT_NODAL,
 COLS=3, FILL=NAN
10, 0, 0., 0., 1.
10, 1, 0., 0., 1.
```

Abaqus input

The supported Abaqus reader currently has no generic Field keyword. Abaqus model data are imported only where a dedicated supported keyword provides the corresponding information.

10.2 *NORMAL

*NORMAL selects an existing three-component ELEMENT_NODAL Field as the source of shell element-nodal normals or directors. The referenced Field must already exist, must use TYPE=ELEMENT_NODAL, and must contain exactly three components.

This mechanism represents shell orientations that cannot be described by a single normal per geometric node. Because the data are element-nodal, two neighbouring shell elements may prescribe different director vectors at the same geometric node, as occurs at folds, discontinuous shell normals, imported midsurfaces, or other locations where a single averaged nodal direction is not defined.

The vectors supplied in the Field are used as prescribed shell-normal information. Missing entries may be left as NaN when the shell-normal workflow is intended to complete them from the element geometry. The resulting normal direction determines shell top/bottom interpretation, pressure direction, local Section directions, and the sign of bending-related result quantities.

The local node index is the zero-based position inside the element connectivity rather than the connected node's user ID. This distinction allows two shell elements sharing one geometric node to store different element-nodal directors at that location.

Parameters

Parameter	Status	Meaning
FIELD	required	Name of an existing three-component ELEMENT_NODAL Field.

FEMaster and Abaqus example

FEMaster

```
*FIELD, NAME=SHELL_NORMALS,
 TYPE=ELEMENT_NODAL, COLS=3, FILL=NAN
10, 0, 0., 0., 1.
10, 1, 0., 0., 1.
10, 2, 0., 0., 1.
10, 3, 0., 0., 1.

*NORMAL, FIELD=SHELL_NORMALS
```

Abaqus input

The supported Abaqus reader does not expose FEMaster's generic Field-based *NORMAL selector. Direct prescription of element-nodal directors therefore requires native FEMaster input; Abaqus input is limited to the explicitly supported orientation syntax of the relevant model definition.

11 Loads and Boundary Conditions

Loads and boundary conditions define how the finite-element model is forced and supported. Native FEMaster groups reusable definitions in named collectors: `*SUPPORT` fills support collectors, while mechanical and thermal load commands fill load collectors. A native `*LOADCASE` then selects the collectors that participate in that analysis with `*SUPPORTS` and `*LOADS`. The same load definition can therefore be reused in several analyses.

Abaqus input expresses loads and boundary conditions primarily inside a `*STEP`. The supported `*BOUNDARY`, `*CLOAD`, `*DLOAD`, and `*DSLOAD` cards are interpreted according to the active analysis procedure. The side-by-side examples in this chapter show the equivalent FEMaster modelling intent.

Throughout this chapter, remember that a numerical zero is a real physical value. For a native `*SUPPORT`, `0` means that the corresponding displacement or rotation is constrained to zero, while an omitted or `NAN` component means that the degree of freedom remains unconstrained.

11.1 *AMPLITUDE

*AMPLITUDE defines a reusable scalar function that can scale a load as the analysis variable changes. In a transient analysis this variable is physical time. Native FEMaster supports TYPE=LINEAR, STEP, and NEAREST interpolation.

For linear interpolation, a value between two samples (t_i, a_i) and (t_{i+1}, a_{i+1}) is obtained from

$$a(t) = a_i + \frac{t - t_i}{t_{i+1} - t_i} \left(a_{i+1} - a_i\right).$$

STEP holds the preceding sample value between tabulated points. NEAREST uses the closest sample. The amplitude is dimensionless unless the user deliberately assigns another interpretation; it multiplies the referenced load magnitude.

An amplitude does not create a load by itself. It must be referenced by a supported load or boundary-condition keyword. This allows one time history to be reused by several load components.

The supported Abaqus form is *AMPLITUDE, DEFINITION=TABULAR. TIME=STEPTIME and VALUE=RELATIVE are accepted. Up to four time/value pairs may be written on one Abaqus data line.

In a transient analysis, amplitude sample times use the same time unit as *TIME. A model expressed in seconds therefore interprets the amplitude abscissa in seconds.

Parameters

Parameter	Status	Meaning
NAME	required	Unique amplitude name.
TYPE	native optional	LINEAR, STEP, or NEAREST; default LINEAR.
DEFINITION	Abaqus optional	Supported value TABULAR.
TIME	Abaqus optional	Supported value STEPTIME.
VALUE	Abaqus optional	Supported value RELATIVE.

Data lines

Entry	Meaning
Native	One time, value pair per record.
Abaqus	Up to four time, value pairs per physical line.

FEMaster and Abaqus example

FEMaster

```
*AMPLITUDE, NAME=RAMP, TYPE=LINEAR
0.0, 0.0
0.1, 1.0
1.0, 1.0
```

Abaqus input

```
*AMPLITUDE, NAME=RAMP,
 DEFINITION=TABULAR, TIME=STEPTIME
0.0, 0.0, 0.1, 1.0,
1.0, 1.0
```

11.2 *SUPPORT

Native *SUPPORT prescribes generalized nodal displacements and rotations. Every row contains a target node or node set followed by six components in the order

$$(U_x, U_y, U_z, R_x, R_y, R_z).$$

A finite number prescribes that component. 0 therefore creates a homogeneous constraint. An omitted entry or NAN leaves the component free.

SUPPORT_COLLECTOR specifies the named collector receiving the support. A loadcase activates one or more such collectors using *SUPPORTS. This separation represents several support cases, for example a fully fixed case and a second analysis with released directions.

ORIENTATION optionally specifies a local coordinate system. The prescribed components are then expressed in that basis, including cylindrical supports, sliders, or boundaries whose constrained direction is not aligned with the global axes.

Every constrained degree of freedom removes an admissible motion. Constraints without a corresponding physical support increase the model stiffness, while missing support constraints can leave a structural nullspace.

Parameters

Parameter	Status	Meaning
SUPPORT_COLLECTOR	required	Named support collector.
ORIENTATION	optional	Named local basis for the prescribed components.

Data lines

Entry	Meaning
target	Node ID or node set.
Six values	$U_x, U_y, U_z, R_x, R_y, R_z$; omitted or NAN means free.

FEMaster and Abaqus example

FEMaster

```
*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0., 0., 0., 0., 0., 0.
GUIDE, NAN, 0., 0., NAN, NAN, NAN
```

Abaqus input

```
*BOUNDARY
FIXED, 1, 6, 0.
GUIDE, 2, 3, 0.
```

11.3 *BOUNDARY

*BOUNDARY is the supported Abaqus displacement/rotation constraint keyword. TYPE currently supports DISPLACEMENT and defaults to that form.

Each data row contains a target, a first degree of freedom, an optional last degree of freedom, and an optional prescribed value. Abaqus structural DOF numbers are one based:

$$1 \to U_x, \quad 2 \to U_y, \quad 3 \to U_z, \quad 4 \to R_x, \quad 5 \to R_y, \quad 6 \to R_z.$$

If the last DOF is omitted, only the first is constrained. If the value is omitted it defaults to zero.

If the target nodes have an Abaqus *TRANSFORM, the boundary components are interpreted in that local nodal basis. Nonzero prescribed values are currently supported for the documented static procedures. Amplitude-dependent prescribed values are subject to the restrictions described for the corresponding analysis procedure.

Native *SUPPORT presents all six generalized components in one row. Abaqus *BOUNDARY uses DOF ranges, so the same constraint can require several rows.

Parameters

Parameter	Status	Meaning
TYPE	optional	Supported form DISPLACEMENT; default DISPLACEMENT.
AMPLITUDE	optional	Supported subject to the active-procedure restrictions.

Data lines

Entry	Meaning
target, first_dof, last_dof, value	DOF range 1–6; last DOF defaults to first and value defaults to zero.

FEMaster and Abaqus example

FEMaster

```
*SUPPORT, SUPPORT_COLLECTOR=BC
TIP, 1.5, 0., 0., NAN, NAN, NAN
```

Abaqus input

```
*BOUNDARY, TYPE=DISPLACEMENT
TIP, 1, 1, 1.5
TIP, 2, 3, 0.
```

11.4 *CLOAD

*CLOAD applies concentrated nodal force and moment components. Native FEMaster writes all six generalized load components in one row:

$$(F_x, F_y, F_z, M_x, M_y, M_z).$$

Missing components default to zero.

The target may be an individual node or a node set. When a node set is used, the specified vector is applied to *every* node in the set. It is not divided by the number of nodes. A reference node with a structural/distributing *COUPLING instead transfers one definite resultant force or moment to a surface.

ORIENTATION optionally defines a local load basis. AMPLITUDE optionally multiplies the load by a named amplitude during analyses that evaluate time-dependent loading.

Abaqus *CLOAD uses one DOF and one magnitude per row. DOFs 1–3 represent forces and 4–6 represent nodal moments. The supported input accepts real non-follower loading; imaginary and follower concentrated loads are not supported.

When a concentrated nodal load represents a physically distributed load, the idealization can create local stress singularities or mesh-dependent peaks. Surface loading and distributing couplings represent load introduction over a finite area.

Parameters

Parameter	Status	Meaning
LOAD_COLLECTOR	native required	Destination load collector.
ORIENTATION	native optional	Named local load basis.
AMPLITUDE	optional	Named amplitude where supported.
FOLLOWER	Abaqus flag	Currently unsupported.
REAL/IMAGINARY	Abaqus flags	Real loading is supported; imaginary loading is not.

FEMaster and Abaqus example

FEMaster

```
*CLOAD, LOAD_COLLECTOR=TIP_LOAD
TIP, 1000., -250., 0., 0., 0., 0.
```

Abaqus input

```
*CLOAD
TIP, 1, 1000.
TIP, 2, -250.
```

11.5 *DLOAD

*DLOAD has different meanings in native FEMaster and in the supported Abaqus input, so the input dialect matters.

In native FEMaster, *DLOAD applies a distributed vector traction to a surface or surface set. The three components describe traction force per unit area in the selected basis. If ORIENTATION is present, the vector is expressed in that local coordinate system. AMPLITUDE may provide time scaling.

The consistent nodal force generated by a traction $\boldsymbol{t}$ is obtained conceptually from the surface integral

$$\boldsymbol{f}_e = \int_{A_e} N^T \boldsymbol{t} \, \mathrm{d}A,$$

so mesh refinement changes the number of nodal force contributions but not the intended continuous traction magnitude.

In supported Abaqus input, *DLOAD currently represents the GRAV body-acceleration form rather than the native surface traction. Its data contain an element or element-set target, the label GRAV, an acceleration magnitude, and a direction vector. The body acceleration is

$$\boldsymbol{a} = g\,\boldsymbol{d}.$$

Other Abaqus *DLOAD labels are not covered by this supported form.

Supported Abaqus surface pressure and vector traction are expressed through *DSLOAD.

Traction has units of force per unit area. Its total resultant equals the surface integral and therefore coincides numerically with the traction magnitude only when the loaded area is one in the chosen unit system.

Parameters

Parameter	Status	Meaning
LOAD_COLLECTOR	native required	Destination native load collector.
ORIENTATION	native optional	Local traction basis.
AMPLITUDE	optional	Named time scaling where supported.
REAL/IMAGINARY	Abaqus flags	Imaginary distributed loading is unsupported.

FEMaster and Abaqus example

FEMaster

```
*DLOAD, LOAD_COLLECTOR=SHEAR
SIDE, 0., 2.5, 0.
```

Abaqus input

```
*DSLOAD, FOLLOWER=NO
SIDE, TRVEC, 2.5, 0., 1., 0.
```

11.6 *PLOAD

Native *PLOAD applies scalar pressure to a surface or surface set. Unlike a vector traction, pressure acts normal to the selected finite-element surface. The sign therefore depends on the element side and surface orientation.

For pressure p and oriented unit normal $\boldsymbol{n}$, the surface traction is conceptually

$$\boldsymbol{t} = p\boldsymbol{n}.$$

The resulting nodal forces are obtained by surface integration. The load is therefore independent of how many nodes happen to lie on the surface, provided the discretization converges to the same geometry.

AMPLITUDE may scale the pressure where supported. Abaqus input expresses the corresponding supported pressure through *DSLOAD with type P.

The surface side and element connectivity determine the pressure direction. Reversing the load sign can compensate numerically for an opposite orientation, but the same orientation continues to govern contact and shell-result conventions.

Parameters

Parameter	Status	Meaning
LOAD_COLLECTOR	native required	Destination load collector.
AMPLITUDE	optional	Named pressure scaling function.

FEMaster and Abaqus example

FEMaster

```
*PLOAD, LOAD_COLLECTOR=PRESSURE
INNER_FACE, 2.0
```

Abaqus input

```
*DSLOAD
INNER_FACE, P, 2.0
```

11.7 *DSLOAD

*DSLOAD is the supported Abaqus surface-load keyword. Two load types are accepted: P for scalar pressure and TRVEC for vector traction.

For P, the row is

surface, P, p.

No direction components are supplied because the surface normal defines the load direction.

For TRVEC, the row additionally supplies a nonzero direction vector. FEMaster normalizes that vector and multiplies it by the requested magnitude. Thus the magnitude controls the traction norm and the direction entries control orientation rather than requiring the user to provide a pre-scaled vector.

ORIENTATION can define a local basis for TRVEC. AMPLITUDE provides supported time scaling. FOLLOWER defaults to YES; current nonlinear-static support is restricted as documented by the reader, with nonlinear TRVEC requiring FOLLOWER=NO and nonlinear follower pressure not supported. Imaginary loading is not supported.

Parameters

Parameter	Status	Meaning
AMPLITUDE	optional	Named amplitude.
ORIENTATION	optional	Named basis for TRVEC.
FOLLOWER	optional	YES or NO; default YES, subject to nonlinear restrictions.
REAL/IMAGINARY	optional flags	Imaginary loading is unsupported.

Data lines

Entry	Meaning
Pressure	surface, P, magnitude.
Traction	surface, TRVEC, magnitude, dx, dy, dz.

FEMaster and Abaqus example

FEMaster

```
*PLOAD, LOAD_COLLECTOR=LOADS
FACE_A, 5.0
*DLOAD, LOAD_COLLECTOR=LOADS
FACE_B, 0., 3.0, 0.
```

Abaqus input

```
*DSLOAD, FOLLOWER=NO
FACE_A, P, 5.0
FACE_B, TRVEC, 3.0, 0., 1., 0.
```

11.8 *VLOAD

Native *VLOAD applies a distributed three-component body-force or acceleration-type vector to an element or element set. The load is integrated over element volume, so it represents a continuous volumetric loading rather than a set of manually chosen nodal forces.

For mass-proportional acceleration such as gravity, the equivalent force density scales with material density. The intended acceleration vector must use units consistent with the model. In a mm–s system, standard gravity is approximately $9810\,\mathrm{mm/s^2}$.

ORIENTATION optionally supplies a local vector basis and AMPLITUDE optionally provides time scaling. The supported Abaqus equivalent is *DLOAD with GRAV, where magnitude and direction are entered separately.

Parameters

Parameter	Status	Meaning
LOAD_COLLECTOR	native required	Destination load collector.
ORIENTATION	native optional	Named local vector basis.
AMPLITUDE	optional	Named time scaling.

FEMaster and Abaqus example

FEMaster

```
*VLOAD, LOAD_COLLECTOR=GRAVITY
EALL, 0., 0., -9810.
```

Abaqus input

```
*DLOAD
EALL, GRAV, 9810., 0., 0., -1.
```

11.9 *TLOAD

Native *TLOAD creates a thermal load from a scalar nodal temperature Field and a stress-free reference temperature. TEMPERATUREFIELD names the temperature Field and REFERENCETEMPERATURE gives the temperature at which the structure is assumed to be stress free.

At a material point, the relevant temperature difference is conceptually

$$\Delta T = T - T_{\mathrm{ref}}.$$

For isotropic thermal expansion, the free thermal strain is

$$\boldsymbol{\varepsilon}_{\mathrm{th}} = \alpha \Delta T \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}^{T}.$$

Thermal stress develops only to the extent that this free expansion is restrained by compatibility, supports, or spatial temperature gradients.

The temperature Field must use TYPE=NODE and one component. The material used by the affected elements must define thermal expansion if a thermal strain contribution is expected.

The supported Abaqus input currently has no corresponding generic thermal-load import.

Parameters

Parameter	Status	Meaning
LOAD_COLLECTOR	required	Destination load collector.
TEMPERATUREFIELD	required	Scalar nodal temperature Field.
REFERENCETEMPERATURE	required	Stress-free reference temperature.

FEMaster and Abaqus example

FEMaster

```
*FIELD, NAME=TEMPERATURE,
 TYPE=NODE, COLS=1, FILL=ZERO
0, 80.
1, 100.

*TLOAD, LOAD_COLLECTOR=THERMAL,
 TEMPERATUREFIELD=TEMPERATURE,
 REFERENCETEMPERATURE=20.
```

Abaqus input

No corresponding generic thermal-load command is currently supported by the Abaqus reader.

11.10 *INERTIALOAD

The current native keyword is spelled *INERTIALOAD. It defines rigid-body inertial loading over an element set. Every row specifies a reference centre, translational acceleration, angular velocity $\boldsymbol{\omega}$, and angular acceleration $\boldsymbol{\alpha}$.

For position $\boldsymbol{r}$ measured from the reference centre, the rigid-body acceleration field contains the familiar contributions

$$\boldsymbol{a}(\boldsymbol{r}) = \boldsymbol{a}_0 + \boldsymbol{\alpha} \times \boldsymbol{r} + \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \boldsymbol{r}),$$

for the prescribed instantaneous translational, tangential, and centrifugal terms. The corresponding inertial forces depend on the model's mass distribution.

CONSIDER_POINT_MASSES controls whether native *POINTMASS features are included in that mass distribution. When enabled, concentrated equipment and other lumped masses participate in the inertial loading.

Density, concentrated masses, geometry, angular velocity, and acceleration must all use one consistent unit system. Centrifugal acceleration scales with $\omega^2 r$, so a unit error in angular velocity or length can become especially severe.

The spelling *INERTIALOAD matches the currently accepted native keyword exactly.

Parameters

Parameter	Status	Meaning
LOAD_COLLECTOR	required	Destination load collector.
CONSIDER_POINT_MASSES	optional	Boolean; default 0.

Data lines

Entry	Meaning
target	Element set receiving the inertial load.
cx,cy,cz	Reference centre.
ax,ay,az	Translational acceleration of the centre.
wx,wy,wz	Angular velocity.
alphax,alphay,alphaz	Angular acceleration.

FEMaster and Abaqus example

FEMaster

```
*INERTIALOAD, LOAD_COLLECTOR=ROT_INERTIA,
 CONSIDER_POINT_MASSES=1
EALL,
0., 0., 0.,
0., 0., 0.,
0., 0., 10.,
0., 0., 2.
```

Abaqus input

No direct equivalent of the full native *INERTIALOAD definition is currently supported by the Abaqus reader.

12 Constraints

Constraints introduce kinematic relations between degrees of freedom that go beyond ordinary supports. A support prescribes selected degrees of freedom directly. A constraint instead relates the motion of different nodes or regions, for example by connecting a reference node to a surface, tying two independently meshed regions together, defining an idealized joint, or removing rigid-body motion from an otherwise free component.

The distinction between these constraint types is physical, not merely syntactic. A kinematic coupling imposes a rigid relation between selected motions. A structural/distributing coupling transfers generalized motion and loading between a reference point and a distributed region. A Tie creates a permanent compatibility relation between associated geometry. A Connector idealizes a joint between two points. RBM removes the rigid-body nullspace of a selected structural region. Contact, described at the end of this chapter, is unilateral and allows separation.

Constraints are used by the active analysis together with the selected support collectors. The numerical method used to enforce the resulting equations is controlled separately by `*CONSTRAINTMETHOD`; see Chapter 14.

12.1 *COUPLING

*COUPLING relates one master or reference node to a slave region. The master is given as a node set and must contain exactly one node. The slave can be a nonempty node set selected with SLAVE or a nonempty surface set selected with SFSET; exactly one of these two slave definitions must be used.

TYPE=KINEMATIC creates a kinematic coupling. The selected slave degrees of freedom follow the rigid kinematics induced by the reference node. This represents an idealized rigid flange, load-introduction point, or boundary where deformation of the coupled patch is suppressed in the selected directions.

TYPE=STRUCTURAL creates the distributing form. It transfers generalized loads and motions between a reference point and a distributed region without making the complete slave region rigid in the same sense as a kinematic coupling. The connected surface therefore retains deformation modes that a kinematic coupling would suppress.

The data line contains six selector values in the order

$$(U_x, U_y, U_z, R_x, R_y, R_z).$$

A component is active when the supplied value is greater than zero. Missing values are inactive. These six values are therefore *selectors*; they are not prescribed displacement or rotation magnitudes.

For a surface slave, the distribution is based on the selected surface geometry. For a node-set slave, the supplied node region itself defines the coupled region. In either case the reference-node set must contain exactly one node so that the generalized motion and loading have an unambiguous reference point.

Applying identical concentrated nodal forces to all nodes of a surface changes the total load when the mesh is refined. A reference-point load combined with a distributing coupling instead represents a definite resultant force or moment. A kinematic coupling additionally imposes its selected rigid relation and can increase the stiffness of a deformable coupled region.

Parameters

Parameter	Status	Meaning
MASTER	required	Node set containing exactly one master/reference node.
TYPE	required	KINEMATIC or STRUCTURAL.
SLAVE	conditional	Nonempty slave node set; mutually exclusive with SFSET.
SFSET	conditional	Nonempty slave surface set; mutually exclusive with SLAVE.

Data lines

Entry	Meaning
Six DOF selectors	Positive value enables the corresponding $U_x, U_y, U_z, R_x, R_y, R_z$ relation; omitted values are inactive.

FEMaster and Abaqus example

FEMaster

```
*COUPLING, MASTER=REF,
 SFSET=LOADED_FACE, TYPE=STRUCTURAL
1., 1., 1., 0., 0., 0.
```

Abaqus input

The supported Abaqus reader currently does not import the Abaqus coupling keyword family. Coupling relations are available through native FEMaster syntax.

12.2 *TIE

*TIE creates a permanent kinematic connection between slave geometry and master geometry located within a prescribed search distance. The slave may be a node set or a surface set. The master may be a surface set or a line set. FEMaster selects the appropriate supported tie relation from these region types.

DISTANCE defines the geometric search tolerance used to associate slave locations with the master geometry. A distance below the interface separation leaves intended locations unmatched, whereas a distance extending to unrelated geometry admits additional candidates. ADJUST=YES permits the supported initial adjustment of slave geometry to the tie relation; the default is NO.

A Tie represents a bonded or otherwise permanently connected interface. Once the relation has been established, opening or sliding that violates the selected compatibility equations is not permitted. It therefore represents idealized bonded interfaces, mesh transitions, or connections that do not separate. *CONTACT instead represents interfaces that may open, close, or lose contact.

The accuracy of a Tie depends on the geometric relation between the two discretizations. Highly nonmatching or strongly curved meshes and competing nearby master surfaces increase the sensitivity to the search distance.

Tie and contact solve different modelling problems. A Tie is an equality-type connection and remains active. Contact is unilateral: it transmits normal interaction only while active and permits subsequent separation. The stiffness of a tied interface is governed by the surrounding structure rather than an artificial penalty spring; physical compliance of an adhesive layer or joint is not represented by the Tie relation itself.

Parameters

Parameter	Status	Meaning
MASTER	required	Nonempty master surface or line set.
SLAVE	required	Nonempty slave node or surface set.
DISTANCE	required	Search/projection distance used to associate slave and master geometry.
ADJUST	optional	NO (default) or YES.

FEMaster and Abaqus example

FEMaster

```
*TIE, MASTER=MASTER_FACE,
 SLAVE=SLAVE_NODES,
 DISTANCE=0.5, ADJUST=YES
```

Abaqus input

Abaqus *TIE is currently not imported by the FEMaster Abaqus reader, even though the native solver provides Tie constraints.

12.3 *CONNECTOR

*CONNECTOR defines an idealized kinematic joint between two nodes. NSET1 and NSET2 each identify a node set containing exactly one node. COORDINATESYSTEM specifies the local connector axes, and TYPE selects which relative translations and rotations are constrained or released.

The supported connector types are BEAM, HINGE, CYLINDRICAL, TRANSLATOR, JOIN, and JOINRX. They represent different idealized joint kinematics. For example, a hinge is intended to release a rotational degree of freedom about its local hinge axis while constraining the remaining relative motion required by that joint idealization. A translator and a cylindrical connector likewise rely on their local axis system to define the permitted relative motion.

The coordinate system is therefore part of the physical definition, not a cosmetic orientation. Different local connector axes produce different joint kinematics even when the connector type and nodal positions remain unchanged.

Connectors idealize joints without resolving their internal geometry. Bearing deformation, bolt bending, local contact, joint clearance, and detailed connection stress require corresponding beam, shell, solid, spring, or contact representations.

A coupling can reduce a distributed region to a reference node before two reference nodes are connected. This construction makes the idealized joint definition independent of the surface mesh density.

Parameters

Parameter	Status	Meaning
TYPE	required	BEAM, HINGE, CYLINDRICAL, TRANSLATOR, JOIN, or JOINRX.
NSET1	required	First one-node set.
NSET2	required	Second one-node set.
COORDINATESYSTEM	required	Named coordinate system defining the connector axes.

FEMaster and Abaqus example

FEMaster

```
*ORIENTATION, NAME=HINGE_AXES,
 TYPE=RECTANGULAR
1., 0., 0., 0., 1., 0.

*CONNECTOR, TYPE=HINGE,
 NSET1=PIN_A, NSET2=PIN_B,
 COORDINATESYSTEM=HINGE_AXES
```

Abaqus input

Connector translation is currently not part of the supported Abaqus input subset.

12.4 *RBM

*RBM suppresses the rigid-body modes of a selected structural element region without requiring the user to choose arbitrary fixed nodes. The target is specified by ELSET or its alias SET; if omitted, EALL is used.

A free three-dimensional body has six rigid-body modes: three translations and three rotations. These modes have no elastic strain energy and therefore produce a structural nullspace when no physical support removes them. RBM constructs constraint equations that remove the rigid-body motion of the selected region while avoiding the strong local bias that can arise from fixing one arbitrary corner node in several directions.

RBM removes the rigid-body nullspace without introducing a particular physical support point. It does not represent attachment through bearings, bolts, symmetry, contact, or other supports, and it does not complete missing physical boundary conditions.

RBM is also different from inertia relief. RBM changes the admissible kinematics by suppressing rigid motion. Inertia relief retains the free-body interpretation and balances the applied load with an equivalent rigid-body inertial field.

RBM addresses rigid-body motion itself. Other singularities can instead arise from disconnected parts, released mechanisms, missing Sections, unsupported rotational degrees of freedom, or incorrect constraint definitions and remain separate model conditions.

Parameters

Parameter	Status	Meaning
ELSET / SET	optional	Nonempty target structural element set; default EALL.

FEMaster and Abaqus example

FEMaster

```
*RBM, ELSET=FREE_COMPONENT
```

Abaqus input

No direct RBM-suppression keyword is currently imported by the supported Abaqus reader.

12.5 Contact

Contact is a unilateral constraint between two surface regions. In contrast to a Tie, which permanently enforces compatible motion between associated regions, contact permits the surfaces to separate and transmits normal interaction only when the contact condition becomes active.

The current FEMaster contact formulation is frictionless dual-mortar surface-to-surface contact with an augmented-Lagrange normal-contact law. Contact acts over the physical overlap of slave and master facets and is nonlinear because the active contact region changes with deformation. Its response depends on the master/slave assignment, surface orientation, and penalty scale.

12.6 *CONTACT

*CONTACT defines one frictionless contact pair between a nonempty master surface set and a nonempty slave surface set. MASTER and SLAVE name the two regions. PENALTY provides the initial contact penalty stiffness. CLEARANCE shifts the zero-gap condition and defaults to zero. FLIP=YES reverses the master-surface normal used by the contact pair; the default is NO.

For each active slave contact degree of freedom, FEMaster uses an augmented-Lagrange relation of the form

$$p_i = \max\left(0, \lambda_i - \varepsilon g_i\right),$$

where g_i is the current normal gap, λ_i is the augmented-Lagrange multiplier, and ε is the effective penalty stiffness. With the sign convention used by the formulation, the max operation suppresses tensile contact: surfaces may separate freely and normal contact pressure is generated only on the compressive side of the constraint.

The mortar formulation determines interaction from the current physical overlap of slave and master facets rather than simply tying a slave node to one permanently selected master node. This makes the contact response considerably less mesh-position dependent than a simple node-to-node contact law, but it also means that changing overlap topology can introduce non-smooth changes into a nonlinear equilibrium iteration.

The user-supplied PENALTY is a numerical stiffness scale, not a physical material constant. A small value permits greater penetration, whereas a large value can make the global tangent system poorly conditioned. The augmented-Lagrange procedure can increase the effective penalty when additional enforcement is required. The relevant scale is therefore set by the structural stiffness of the contacting bodies and evaluated through the resulting penetration and nonlinear conditioning.

Master/slave assignment. The mortar formulation is not algebraically symmetric with respect to the two labels: the dual interpolation and normal multipliers are associated with the slave side. Similar discretizations can admit either assignment, but changing it between comparable models also changes the discrete formulation. With strongly different meshes, assigning the finer or more detailed side as slave places the dual interpolation on that discretization; mesh sensitivity remains part of the numerical comparison.

Normal orientation. Contact sign depends on the surface orientation. A reversed surface orientation can make a physically separated pair appear active or prevent a penetrating pair from developing reaction. FLIP reverses the master normal for the contact pair but does not change the underlying element connectivity or surface side label.

Penalty magnitude. Penetration decreases as contact enforcement becomes stiffer, while excessive stiffness degrades the conditioning of the equilibrium iterations. The input number alone therefore does not define contact accuracy independently of geometry and structural stiffness.

Analysis type. Contact changes with deformation and belongs to nonlinear static analysis. A linear analysis cannot represent opening, closing, changing overlap, or an evolving unilateral active set.

Parameters

Parameter	Status	Meaning
MASTER	required	Nonempty master surface set.
SLAVE	required	Nonempty slave surface set.
PENALTY	required	Initial augmented-Lagrange penalty stiffness.
CLEARANCE	optional	Contact-clearance offset; default 0.
FLIP	optional	NO (default) or YES; reverses the master-surface normal.

FEMaster and Abaqus example

FEMaster

```
*SURFACE, NAME=MASTER_FACE
MASTER_ELEMENTS, S1
*SURFACE, NAME=SLAVE_FACE
SLAVE_ELEMENTS, S2

*CONTACT, MASTER=MASTER_FACE,
 SLAVE=SLAVE_FACE,
 PENALTY=1.0e8,
 CLEARANCE=0.0, FLIP=NO
```

Abaqus input

The supported Abaqus reader currently does not import the Abaqus contact-definition family. Contact can be added with native FEMaster syntax using surface sets available in the model.

13 Analysis Procedures

An analysis procedure determines which finite-element problem FEMaster solves. The same mesh, materials, Sections, loads, and constraints can be reused for different procedures because the governing equations differ even when the physical model is unchanged.

Native FEMaster uses `*LOADCASE, TYPE=...`. Abaqus input uses `*STEP` followed by one supported procedure keyword. The side-by-side examples in this chapter show equivalent analysis intent. Numerical settings such as constraint treatment, nonlinear convergence, damping, time stepping, and eigensolver controls are described in Chapter 14.

13.1 Analysis families

The principal equations are:

Linear static

$$Ku = f,$$

with the prescribed supports and kinematic constraints applied to the displacement vector u.

Nonlinear static

$$r(u, \lambda) = f_{\text{int}}(u) - \lambda f_{\text{ext}} = 0,$$

solved incrementally with Newton iterations. The load factor λ is prescribed by load control or solved together with u by arc-length control.

Eigenfrequency

$$K\phi_i = \omega_i^2 M\phi_i, \qquad f_i = \frac{\omega_i}{2\pi}.$$

Linear buckling

$$(K + \lambda_i K_G)\,\phi_i = 0,$$

where K_G is the geometric stiffness generated by the reference preload state.

Linear transient dynamics

$$M\ddot{u} + C\dot{u} + Ku = f(t),$$

integrated in time with the implicit Newmark method.

Linear harmonic response

$$\left(K - \omega^2 M + i\omega C\right)\hat{u}(\omega) = \hat{f}(\omega),$$

solved directly for each requested excitation frequency.

13.2 *LOADCASE

Native *LOADCASE begins an analysis definition. TYPE selects one of the supported procedures:

- LINEARSTATIC;
- NONLINEARSTATIC;
- LINEARBUCKLING;
- LINEARSTATICTOPO;
- EIGENFREQ;
- LINEARTRANSIENT;
- LINEARHARMONIC.

NAME is optional and gives the analysis a descriptive identifier.

Commands inside the loadcase select support and load collectors and configure the numerical procedure. Their validity depends on the analysis type: for example, *TIME and *NEWMARK belong to LINEARTRANSIENT, *FREQUENCIES belongs to LINEARHARMONIC, and *NONLINEAR belongs to NONLINEARSTATIC.

A deck may contain several native loadcases, allowing the same structural model to be evaluated under different loads or with different procedures, for example a static preload followed by an eigenfrequency study in a separate run definition.

Each loadcase selects the previously defined collectors that participate in that analysis. A load that exists in the model but is not selected through *LOADS does not contribute to the native loadcase.

Parameters

Parameter	Status	Meaning
TYPE	required	Supported native analysis type listed above.
NAME	optional	Descriptive loadcase name.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARSTATIC,
 NAME=SERVICE_LOAD
*SUPPORTS
BC
*LOADS
SERVICE
*END
```

Abaqus input

```
*STEP, NAME=SERVICE_LOAD
*STATIC
*BOUNDARY
...
*CLOAD
...
*END STEP
```

13.3 *SUPPORTS

*SUPPORTS selects one or more native support collectors for the active loadcase. The selected supports are combined with model constraints such as couplings, ties, connectors, and RBM constraints before the equation system is solved.

Several collectors may be listed together. This supports modular models, for example one collector for permanent fixture constraints and another for a symmetry condition used only in a particular analysis.

The command is supported by the structural analysis families that use nodal constraints, including static, buckling, eigenfrequency, transient, harmonic, and topology-static analyses.

Abaqus input has no separate collector-selection keyword. Its *BOUNDARY cards define the support conditions associated with the Step directly.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=EIGENFREQ
*SUPPORTS
FIXED_SUPPORTS, SYMMETRY_SUPPORTS
*NUMEIGENVALUES
10
*END
```

Abaqus input

```
*STEP
*FREQUENCY
10
*BOUNDARY
FIXED, 1, 3, 0.
*END STEP
```

13.4 *LOADS

*LOADS selects one or more native load collectors for the active loadcase. The resulting external load is the superposition of the loads contained in all selected collectors.

For a linear static analysis,

$$f = \sum_c f_c,$$

where f_c denotes the contribution of one selected collector. For transient or harmonic analysis, amplitudes may make the corresponding load contribution depend on time or excitation frequency according to the supported load definition.

*LOADS is used by linear static, nonlinear static, buckling, transient, and harmonic analyses. A free-vibration eigenfrequency analysis has no applied external-force vector and therefore does not use a load collector.

Abaqus Step-local load cards such as *CLOAD, *DLOAD, and *DSLOAD provide the corresponding Step loading directly.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARSTATIC
*LOADS
GRAVITY, PRESSURE, ACTUATOR
*END
```

Abaqus input

```
*STEP
*STATIC
*DLOAD
EALL, GRAV, 9810., 0., 0., -1.
*DSLOAD
PRESSURE_FACE, P, 2.0
*CLOAD
ACTUATOR, 1, 500.
*END STEP
```

13.5 *END

Native *END closes the current *LOADCASE. It is distinct from *END PART, *END INSTANCE, *END ASSEMBLY, and Abaqus *END STEP.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARSTATIC
...
*END
```

Abaqus input

```
*STEP
*STATIC
...
*END STEP
```

13.6 *STEP

*STEP begins an Abaqus-style analysis Step. The currently supported Abaqus input accepts at most one analysis Step per deck. Exactly one supported procedure keyword—*STATIC, *FREQUENCY, *BUCKLE, *DYNAMIC, or *STEADY STATE DYNAMICS—defines the analysis performed in that Step.

NAME is optional. NLGEOM enables geometric nonlinearity when present without a value or set to YES; NO and OFF disable it. INC specifies the maximum number of increments for nonlinear procedures and defaults to 100. AMPLITUDE selects the supported default Step amplitude behaviour RAMP or STEP. PERTURBATION marks a linear perturbation Step.

Loads and boundary conditions written within the Step belong to that analysis. *END STEP terminates the Step.

Parameters

Parameter	Status	Meaning
NAME	optional	Descriptive Step name.
NLGEOM	optional	Bare/YES enables geometric nonlinearity; NO/OFF disables it.
INC	optional	Positive maximum increment count; default 100.
AMPLITUDE	optional	RAMP or STEP.
PERTURBATION	optional flag	Selects linear perturbation semantics.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=NONLINEARSTATIC
*NONLINEAR, CONTROL=LOAD,
 MAX_INCREMENTS=100
...
*END
```

Abaqus input

```
*STEP, NAME=LARGE_DEFLECTION,
 NLGEOM=YES, INC=100
*STATIC
0.05, 1.0, 1e-5, 0.1
...
*END STEP
```

13.7 *STATIC

Abaqus *STATIC selects static equilibrium. The supported mapping depends on the Step options.

With neither geometric nonlinearity nor Riks path following, the analysis is linear static:

$$Ku = f.$$

A PERTURBATION Step also uses the linear static formulation.

If NLGEOM is enabled or RIKS is present, the procedure uses nonlinear static equilibrium. The nonlinear residual is

$$r(u, \lambda) = f_{\text{int}}(u) - \lambda f_{\text{ext}},$$

and Newton iterations solve the linearized equilibrium equation

$$K_T \Delta u = -r.$$

Without RIKS, λ is advanced by load control. RIKS activates arc-length path following, in which displacement and load factor are solved together under an additional path constraint. This allows equilibrium paths to continue through limit points where simple monotonic load control may fail.

DIRECT requests fixed nonlinear load increments rather than adaptive stepping where supported.

For ordinary nonlinear static input, the data positions are initial increment, Step period, minimum increment, and maximum increment. The values are converted to the normalized load path represented by the FEMaster nonlinear analysis. For RIKS, the first four entries analogously define initial, period, minimum, and maximum arc-length scales. Abaqus Riks termination controls in the later data positions are currently not supported.

Parameters

Parameter	Status	Meaning
RIKS	optional flag	Select arc-length path following.
DIRECT	optional flag	Request fixed nonlinear load increments.

Data lines

Entry	Meaning
Ordinary static	Optional initial, period, minimum, maximum.
Riks	First four values define the supported path-increment controls; unsupported later termination fields must be omitted.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=NONLINEARSTATIC
*NONLINEAR, CONTROL=ARC_LENGTH,
 INITIAL_INCREMENT=0.05,
 MINIMUM_INCREMENT=1e-5,
 MAXIMUM_INCREMENT=0.1,
 MAX_INCREMENTS=200
*END
```

Abaqus input

```
*STEP, NLGEOM=YES, INC=200
*STATIC, RIKS
0.05, 1.0, 1e-5, 0.1
*END STEP
```

13.8 *FREQUENCY

Abaqus *FREQUENCY selects free-vibration eigenfrequency extraction. The constrained generalized eigenproblem is

$$K\phi_i = \lambda_i M\phi_i, \qquad \lambda_i = \omega_i^2, \qquad f_i = \frac{\sqrt{\lambda_i}}{2\pi}.$$

The data line begins with the positive number of requested modes.

The mode shapes ϕ_i describe relative deformation patterns; their arbitrary scale is not a physical vibration amplitude. Frequencies are determined by both stiffness and mass, so density, concentrated masses, Sections, and support conditions must all be physically meaningful.

EIGENSOLVER accepts LANCZOS or SUBSPACE as supported Abaqus syntax. FEMaster's own eigensolution settings determine the numerical backend used for the calculation.

Data lines

Entry	Meaning
First value	Positive number of requested eigenpairs.
Remaining accepted positions	Additional Abaqus spectral controls not used by the currently supported mapping.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=EIGENFREQ
*SUPPORTS
BC
*NUMEIGENVALUES
12
*END
```

Abaqus input

```
*STEP, PERTURBATION
*FREQUENCY, EIGENSOLVER=LANCZOS
12
*BOUNDARY
FIXED, 1, 6, 0.
*END STEP
```

13.9 *BUCKLE

Abaqus *BUCKLE selects linear eigenvalue buckling. The analysis first establishes the reference stress state caused by the selected preload. That stress state produces a geometric stiffness matrix K_G. Buckling factors then follow from

$$(K + \lambda_i K_G)\,\phi_i = 0.$$

Depending on the sign convention used for K_G, the equivalent generalized eigenproblem may be written with $-K_G$ on the right-hand side.

The eigenvalue λ_i is a multiplier on the reference preload, not an absolute force. If the preload corresponds to 10 kN and the first factor is 3.2, the linearized critical load associated with that mode is 32 kN. This interpretation is valid only within the assumptions of linear eigenvalue buckling; imperfections, material nonlinearity, contact changes, and nonlinear prebuckling deformation are not represented by the eigenvalue alone.

The first data value is the positive number of requested buckling modes. EIGENSOLVER accepts LANCZOS or SUBSPACE as supported Abaqus syntax.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARBUCKLING
*SUPPORTS
BC
*LOADS
PRELOAD
*NUMEIGENVALUES
5
*END
```

Abaqus input

```
*STEP, PERTURBATION
*BUCKLE
5
*BOUNDARY
...
*CLOAD
...
*END STEP
```

13.10 *DYNAMIC

Abaqus *DYNAMIC, DIRECT selects FEMaster's linear implicit transient analysis. The structural equation is

$$M\ddot{u}(t) + C\dot{u}(t) + Ku(t) = f(t).$$

The current supported mapping uses a fixed time increment and therefore requires DIRECT. Geometric nonlinearity is not supported by this linear transient procedure.

The first two data values are the fixed time increment Δt and the positive Step period T. The analysis runs from $t = 0$ to $t = T$ with the specified increment. Loads may vary with time through amplitudes.

Native FEMaster exposes the same analysis through LINEARTRANSIENT, *TIME, *NEWMARK, optional *DAMPING, and output-cadence controls.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARTRANSIENT
*SUPPORTS
BC
*LOADS
DYNAMIC_LOAD
*TIME
0., 1., 0.01
*NEWMARK
0.25, 0.5
*END
```

Abaqus input

```
*STEP
*DYNAMIC, DIRECT
0.01, 1.0
*BOUNDARY
...
*CLOAD, AMPLITUDE=RAMP
...
*END STEP
```

13.11 *STEADY STATE DYNAMICS

Abaqus *STEADY STATE DYNAMICS, DIRECT selects direct linear harmonic response. For harmonic excitation at angular frequency $\omega = 2\pi f$, FEMaster solves the complex system

$$\left(K - \omega^2 M + i\omega C\right) \hat{u} = \hat{f}.$$

The result $\hat{u}$ contains amplitude and phase information. Resonances appear when the excitation frequency approaches a natural frequency and damping is small.

DIRECT is required. INTERVAL currently supports RANGE. FREQUENCYSCALE may be LINEAR or LOGARITHMIC.

Each data record contains lower frequency, optional upper frequency, optional point count, bias, and scale factor. If the upper frequency is omitted or zero, only the lower frequency is solved. For a range, the upper frequency must be at least as large as the lower. The default point count is 20. Bias values other than 1 and scale factors other than 1 are not supported. A logarithmic range requires a positive lower frequency.

Native FEMaster specifies a linear frequency sweep with *FREQUENCIES, SCALE=LINEAR. Multiple or explicitly listed frequencies can be represented according to the native frequency controls documented in the next chapter.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARHARMONIC
*SUPPORTS
BC
*LOADS
HARMONIC_FORCE
*FREQUENCIES, SCALE=LINEAR
10., 1000., 100
*END
```

Abaqus input

```
*STEP
*STEADY STATE DYNAMICS, DIRECT,
 FREQUENCYSCALE=LINEAR
10., 1000., 100, 1., 1.
*CLOAD
...
*END STEP
```

13.12 *END STEP

*END STEP closes the active Abaqus Step. A supported procedure must have been selected within the Step.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARSTATIC
...
*END
```

Abaqus input

```
*STEP
*STATIC
...
*END STEP
```

14 Solver and Numerical Controls

The commands in this chapter determine how an analysis is solved. They do not change the mesh or material model; instead they control the linear algebra, constraint treatment, nonlinear path following, time integration, damping, frequency sampling, eigenvalue extraction, and selected static balancing procedures.

These settings can strongly affect robustness and computational cost, but they do not correct an inconsistent physical model. A mechanism, missing Section, disconnected mesh, or inadequate support definition remains a model-level cause of singularity regardless of the numerical regularization.

14.1 *SOLVER

*SOLVER selects the sparse linear solution strategy for supported native loadcases. DEVICE may be CPU or GPU; the default is CPU. METHOD may be DIRECT or INDIRECT; the default is DIRECT.

A direct method factorizes the sparse equation matrix and then solves by substitutions. Its convergence does not depend on an iterative stopping process, while its factorization memory can become substantial for large three-dimensional models.

An indirect method solves the linear system iteratively. For a system

$$Ax = b,$$

the method improves an approximate x until the residual

$$r = b - Ax$$

is sufficiently small. Iterative methods can reduce memory use for suitable large systems, but their efficiency is strongly affected by conditioning, matrix definiteness, scaling, and the available preconditioning strategy.

The chosen constraint formulation matters because it changes the algebraic character of the system. Nullspace and elimination methods lead to reduced structural systems and are suitable for direct or supported iterative solution. Lagrange multipliers create the indefinite saddle-point system

$$\begin{bmatrix} K & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} u \\ \lambda \end{bmatrix} = \begin{bmatrix} f \\ d \end{bmatrix},$$

which requires a linear solver that can handle indefinite matrices and is therefore restricted to supported direct configurations.

The actual CPU/GPU libraries available depend on the FEMaster build. Selecting GPU in a build without the required GPU backend cannot create GPU support by itself.

The default CPU/direct configuration avoids iterative convergence and backend-specific GPU requirements. Indirect or GPU configurations exchange this baseline for different memory, scaling, and hardware characteristics; their numerical behaviour still depends on the physical model and constraint system.

Parameters

Parameter	Status	Meaning
DEVICE	optional	CPU (default) or GPU.
METHOD	optional	DIRECT (default) or INDIRECT.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARSTATIC
*SOLVER, DEVICE=CPU, METHOD=DIRECT
...
*END
```

Abaqus input

The supported Abaqus input does not translate Abaqus solver-selection controls to native *SOLVER settings.

14.2 *CONSTRAINTMETHOD

`*CONSTRAINTMETHOD` selects how equations of the form

$$Cu = d$$

are imposed on the structural system. Supported input values are `NULLSPACE`, `LAGRANGE`, and `ELIMINATION`. Not every analysis supports every method; in particular, nonlinear and harmonic procedures may impose stricter restrictions than linear static analysis.

Nullspace projection

The nullspace method writes every admissible displacement vector as

$$u = u_p + Tq,$$

where

$$Cu_p = d, \qquad CT = 0.$$

The columns of T span all homogeneous motions that satisfy the constraints, while u_p is one particular displacement satisfying the prescribed inhomogeneous values.

For a linear static system

$$Ku = f,$$

substitution gives the reduced equation

$$T^T K T q = T^T (f - K u_p).$$

Thus constrained directions are removed before the linear solve. The same projection applies naturally to mass and damping operators in dynamic analyses:

$$K_r = T^T K T, \qquad M_r = T^T M T, \qquad C_r = T^T C T.$$

Nullspace projection handles general linear constraint equations while keeping the reduced system in structural form. Couplings, ties, and other multi-point constraints can therefore be represented without requiring a favourable substitution ordering.

Lagrange multipliers

The Lagrange method retains the original displacement vector and adds one multiplier per constraint equation:

$$\begin{bmatrix} K & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} u \\ \lambda \end{bmatrix} = \begin{bmatrix} f \\ d \end{bmatrix}.$$

The multipliers λ represent generalized constraint reactions. The advantage is the direct enforcement of the equations without eliminating displacement variables. The disadvantage is the larger indefinite saddle-point system.

This method requires a supported direct backend. Its additional unknowns provide multiplier reactions and preserve the original displacement variables.

Elimination

Elimination chooses dependent degrees of freedom and expresses them in terms of independent degrees of freedom, again producing an affine relation

$$u = u_p + Tq.$$

Unlike a numerical nullspace basis, the transformation is constructed from explicit substitutions. This can be very efficient for constraints with a clear master/slave structure and an acyclic dependency graph.

For dense or strongly coupled general equations, nullspace projection avoids the favourable substitution ordering required by efficient elimination.

Parameters

Parameter	Status	Meaning
`TYPE`	required	`NULLSPACE`, `LAGRANGE`, or `ELIMINATION`.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARSTATIC
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*SOLVER, METHOD=INDIRECT
...
*END
```

Abaqus input

The supported Abaqus input does not expose FEMaster's native constraint-method selection.

14.3 *NONLINEAR

*NONLINEAR configures a native NONLINEARSTATIC analysis. It controls the equilibrium path, increment size, adaptive growth and cutback, Newton iteration limits, convergence tolerance, and optional weak-row regularization.

At a prescribed load factor λ, equilibrium is

$$r(u, \lambda) = f_{\text{int}}(u) - \lambda f_{\text{ext}} = 0.$$

Newton's method linearizes the residual at iteration k:

$$K_T(u_k)\Delta u_k = -r(u_k, \lambda), \qquad u_{k+1} = u_k + \Delta u_k.$$

The tangent stiffness K_T contains the current material, geometric, constraint, and contact contributions appropriate to the model.

Load control

CONTROL=LOAD is the default. The load factor is advanced by a prescribed increment $\Delta\lambda$, and Newton iterations solve only for the displacement correction. Load control is efficient on monotonic equilibrium paths but cannot generally pass a limit point where the physical equilibrium curve turns backward in load factor.

Arc-length control

CONTROL=ARC_LENGTH solves displacement and load factor together under an additional path constraint. In generic form the constraint limits a combined displacement/load increment,

$$\Delta u^T \Delta u + \psi^2 \, \Delta\lambda^2 \, \|f_{\text{ext}}\|^2 = \Delta s^2,$$

where Δs is the arc-length scale and ψ is controlled by ARC_LENGTH_PSI. This allows the solution to continue through snap-through or other limit points for which simple load control would stall.

Increment control

INITIAL_INCREMENT defines the first normalized load or arc-length step. MINIMUM_INCREMENT and MAXIMUM_INCREMENT bound adaptive changes. MAX_INCREMENTS limits the number of accepted increments. The legacy INCREMENTS=n shorthand sets the initial scale to $1/n$; an explicitly supplied INITIAL_INCREMENT takes precedence.

With ADAPTIVE=ON, quickly converged increments can grow by GROWTH_FACTOR, while rejected or slowly converged attempts are reduced by CUTBACK_FACTOR. The default thresholds are FAST_ITERATIONS=6, SLOW_ITERATIONS=10, GROWTH_FACTOR=1.5, and CUTBACK_FACTOR=0.5. MAXIMUM_CUTBACKS limits repeated reductions of one attempted path segment.

Adaptive stepping does not alter the equilibrium equations. It changes only how aggressively the solution path is traversed. Persistently small increments can result from contact transitions, material stiffness, element distortion, unstable mechanisms, or an inconsistent tangent; cutback only changes the attempted path discretization.

Newton convergence

MAXITER limits Newton iterations per attempted increment; the default is 20. TOL, default 10^{-8}, controls the equilibrium convergence criterion. A small residual tolerance does not guarantee an accurate finite-element model: mesh discretization, constitutive assumptions, and load idealization remain independent error sources.

Weak-row regularization

`REGULARIZE_ZERO_ROWS=ON` enables diagonal stabilization for tangent rows whose stiffness is effectively zero. `REGULARIZATION_ALPHA`, default 10^{-4}, controls the scale of that stabilization relative to representative nonzero tangent stiffness.

This option stabilizes temporary zero-stiffness directions that can occur in nonlinear models, for example before an otherwise meaningful mechanism engages. It does not resolve a genuinely unconstrained rigid-body mode or disconnected component. A material change of the physical response with increasing regularization indicates sensitivity to the underlying model rather than only numerical stabilization.

Parameters

Parameter	Status	Meaning
`CONTROL`	optional	`LOAD` (default) or `ARC_LENGTH`.
`INCREMENTS`	optional	Legacy count mapped to initial scale $1/n$.
`MAX_INCREMENTS`	optional	Maximum accepted nonlinear increments.
`INITIAL_INCREMENT`	optional	Initial load/arc-length step scale.
`MINIMUM_INCREMENT`	optional	Lower step-size bound.
`MAXIMUM_INCREMENT`	optional	Upper step-size bound.
`ARC_LENGTH_PSI`	optional	Arc-length load weighting; default 1.0.
`ADAPTIVE`	optional	Adaptive growth/cutback; default `ON`.
`GROWTH_FACTOR`	optional	Fast-convergence growth factor; default 1.5.
`CUTBACK_FACTOR`	optional	Rejection/slow-convergence reduction factor; default 0.5.
`FAST_ITERATIONS`	optional	Fast convergence threshold; default 6.
`SLOW_ITERATIONS`	optional	Slow convergence threshold; default 10.
`MAXIMUM_CUTBACKS`	optional	Cutback limit; default 20.
`MAXITER`	optional	Newton iteration limit; default 20.
`TOL`	optional	Equilibrium convergence tolerance; default 10^{-8}.
`REGULARIZE_ZERO_ROWS`	optional	Enable weak-row regularization; default `ON`.
`REGULARIZATION_ALPHA`	optional	Regularization scale; default 10^{-4}.

FEMaster and Abaqus example

FEMaster

```
*NONLINEAR, CONTROL=ARC_LENGTH,
 MAX_INCREMENTS=200,
 INITIAL_INCREMENT=0.05,
 MINIMUM_INCREMENT=1e-5,
 MAXIMUM_INCREMENT=0.1,
 MAXITER=40, TOL=1e-8,
 ADAPTIVE=ON
```

Abaqus input

```
*STEP, NLGEOM=YES, INC=200
*STATIC, RIKS
0.05, 1.0, 1e-5, 0.1
```

14.4 *TIME

*TIME defines the time interval and fixed integration step for LINEARTRANSIENT. Two forms are accepted:

$$t_{\text{start}},\ t_{\text{end}}, \Delta t$$

or the compact form

$$t_{\text{end}}, \Delta t,$$

which uses $t_{\text{start}} = 0$.

The time increment must resolve the highest response frequencies that matter to the analysis. Implicit time integration may remain numerically stable for a large Δt while still giving an inaccurate transient response. Stability and accuracy are not the same criterion.

The ratio between Δt and the shortest relevant period T_{min} determines the temporal resolution. Phase and peak-amplitude accuracy generally require many points per period.

FEMaster and Abaqus example

FEMaster

```
*TIME
0.0, 2.0, 0.005
```

Abaqus input

```
*DYNAMIC, DIRECT
0.005, 2.0
```

14.5 *NEWMARK

*NEWMARK sets the β and γ parameters of the implicit Newmark method. Defaults are

$$\beta = 0.25, \qquad \gamma = 0.5,$$

the constant-average-acceleration scheme.

Newmark updates displacement and velocity according to

$$u_{n+1} = u_n + \Delta t\, v_n + \Delta t^2 \left[\left(\frac{1}{2} - \beta \right) a_n + \beta a_{n+1} \right],$$

$$v_{n+1} = v_n + \Delta t \left[(1-\gamma) a_n + \gamma a_{n+1} \right].$$

For the default parameters the linear method is unconditionally stable for the standard undamped problem and introduces no algorithmic high-frequency damping. Accuracy still requires a sufficiently small time step.

Data lines

Entry	Meaning
beta, gamma	Newmark integration parameters.

FEMaster and Abaqus example

FEMaster

```
*NEWMARK
0.25, 0.5
```

Abaqus input

The supported Abaqus *DYNAMIC, DIRECT mapping uses FEMaster's transient integration but does not expose a separate imported Newmark-parameter card.

14.6 *DAMPING

`*DAMPING, TYPE=RAYLEIGH` defines proportional damping for transient or harmonic analysis:

$$C = \alpha M + \beta K.$$

The mass-proportional coefficient α primarily affects low frequencies, while the stiffness-proportional coefficient β increasingly affects high frequencies.

For a mode with circular frequency ω_i, the approximate modal damping ratio is

$$\zeta_i = \frac{1}{2}\left(\frac{\alpha}{\omega_i} + \beta\omega_i\right).$$

This relation determines α and β from specified damping ratios at two frequencies.

Rayleigh damping is frequency dependent. Coefficients fitted at two frequencies produce different damping ratios elsewhere in the spectrum, especially for modes far below or above the fitting range.

Parameters

Parameter	Status	Meaning
`TYPE`	required	Currently `RAYLEIGH`.

Data lines

Entry	Meaning
`alpha, beta`	Mass- and stiffness-proportional Rayleigh coefficients.

FEMaster and Abaqus example

FEMaster

```
*DAMPING, TYPE=RAYLEIGH
0.0, 1.0e-5
```

Abaqus input

The supported Abaqus reader currently does not import a separate Step damping card.

14.7 *FREQUENCIES

*FREQUENCIES defines the excitation-frequency sweep for LINEARHARMONIC. The native command currently supports linear spacing; SCALE may be omitted or set to LINEAR.

The data line contains start frequency f_0, end frequency f_1, and integer point count n. For $n > 1$,

$$f_i = f_0 + i\frac{f_1 - f_0}{n - 1}, \qquad i = 0, \ldots, n - 1.$$

For $n = 1$, the single solved frequency is f_0.

Frequency resolution determines which parts of the response curve are sampled. A coarse linear sweep can miss a narrow lightly damped resonance even though the endpoints cover the correct range; additional points near a resonance resolve its peak amplitude and phase more closely.

Parameters

Parameter	Status	Meaning
SCALE	optional	Currently LINEAR.

FEMaster and Abaqus example

FEMaster

```
*FREQUENCIES, SCALE=LINEAR
10., 1000., 100
```

Abaqus input

```
*STEADY STATE DYNAMICS, DIRECT,
 FREQUENCYSCALE=LINEAR
10., 1000., 100, 1., 1.
```

14.8 *NUMEIGENVALUES

*NUMEIGENVALUES sets the number of requested eigenpairs for EIGENFREQ or LINEARBUCKLING. The data value must be a positive integer.

The requested value is a result count, not a frequency or buckling-factor cutoff. A request for ten modes therefore returns ten eigenpairs but does not independently define the covered spectral range, particularly in the presence of multiplicities or closely spaced eigenvalues.

FEMaster and Abaqus example

FEMaster

```
*NUMEIGENVALUES
20
```

Abaqus input

```
*FREQUENCY
20
```

14.9 *SIGMA

*SIGMA sets the spectral shift used by native LINEARBUCKLING. The shift guides the eigensolver toward the targeted part of the buckling spectrum.

For the reduced buckling problem written as

$$A\phi = \lambda(-B)\phi,$$

with elastic stiffness A and geometric stiffness contribution B, shift-invert methods become most effective when the shift is placed near, but appropriately below, the eigenvalue range of interest.

SIGMA=0 requests the automatic shift strategy. FEMaster estimates a scale from the pre-buckling solution where possible and falls back to conservative stiffness/geometric-stiffness estimates if necessary. A manual shift directly targets a known approximate spectral range.

FEMaster and Abaqus example

FEMaster

```
*SIGMA
0.0
```

Abaqus input

The supported Abaqus *BUCKLE mapping does not expose native FEMaster's explicit *SIGMA control.

14.10 *WRITEEVERY

*WRITEEVERY controls how often transient results are written. It does not change the integration step or the computed transient trajectory.

With TYPE=STEPS, the data value is the number of integration steps between written states. With TYPE=TIME, it is a physical time interval. Writing less frequently can greatly reduce output size when the integration time step is small.

Output sampling must still be fine enough for the quantity being inspected. A transient can be integrated accurately but appear to miss a short peak if results are written too sparsely.

Parameters

Parameter	Status	Meaning
TYPE	optional	STEPS (default) or TIME.

FEMaster and Abaqus example

FEMaster

```
*WRITEEVERY, TYPE=STEPS
10
```

Abaqus input

Abaqus output-request cards currently do not control FEMaster's transient write cadence.

14.11 *INITIALVELOCITY

*INITIALVELOCITY assigns initial generalized nodal velocity for LINEARTRANSIENT from an existing six-component nodal Field. The Field must use TYPE=NODE and component order

$$(V_x, V_y, V_z, \Omega_x, \Omega_y, \Omega_z),$$

corresponding to the translational and rotational nodal DOFs.

FIELD names the Field that completely supplies the initial velocity state. Initial displacement is taken from the analysis/model state defined by the current procedure.

Parameters

Parameter	Status	Meaning
FIELD	required	Existing six-component NODE Field.

FEMaster and Abaqus example

FEMaster

```
*FIELD, NAME=V0, TYPE=NODE,
 COLS=6, FILL=ZERO
0, 1., 0., 0., 0., 0.
*INITIALVELOCITY, FIELD=V0
```

Abaqus input

No initial-velocity mapping is currently supported by the Abaqus reader.

14.12 *INERTIARELIEF

*INERTIARELIEF enables inertia relief for a linear static analysis of an intentionally free or insufficiently supported body.

For a free body under a nonzero resultant external force or moment, the ordinary static equation has no physical equilibrium because the body accelerates. Inertia relief determines the rigid-body translational and angular acceleration associated with the applied load and model mass properties, forms the corresponding inertial forces, and balances the external resultant. The remaining static solution represents elastic deformation relative to the accelerating body.

This represents free-flight or free-body loadcases without adding an artificial fixed support. It does not represent real bearings, bolts, fixtures, or other physical supports.

CONSIDER_POINT_MASSES controls whether *POINTMASS features contribute to the mass and inertia used for the balancing field; the default is true.

Parameters

Parameter	Status	Meaning
CONSIDER_POINT_MASSES	Optional	Boolean; default 1.

FEMaster and Abaqus example

FEMaster

```
*INERTIARELIEF, CONSIDER_POINT_MASSES=1
```

Abaqus input

No inertia-relief procedure is currently imported by the supported Abaqus reader.

14.13 *REBALANCELOADS

*REBALANCELOADS adjusts the active external load system of a linear static analysis so that its resultant force and moment vanish. It is intended for a load that is theoretically self-equilibrated but contains a small numerical imbalance because of discretization, rounding, approximate surface selection, or equivalent-load construction.

This differs physically from inertia relief. *REBALANCELOADS assumes a theoretically self-equilibrated applied load and corrects its numerical residual. *INERTIARELIEF accepts a genuinely unbalanced free-body load and represents the rigid-body acceleration needed to balance it.

Rebalancing a load with a physical resultant removes that resultant and thereby defines a different loadcase; it does not supply support to an unsupported model.

FEMaster and Abaqus example

FEMaster

```
*REBALANCELOADS
```

Abaqus input

No corresponding load-rebalancing input card is currently supported by the Abaqus reader.

15 Output, Diagnostics, and Results

FEMaster writes analysis results in its own result formats and provides several diagnostic commands for inspecting matrices, constraints, and the assembled model. These diagnostics do not change the physical problem; they are intended for verification, comparison, and troubleshooting.

The native `.res` format preserves FEMaster Field domains and is the most complete textual result source. FRD output provides a CalculiX/CGX-compatible visualization format for supported nodal quantities. Additional writer formats may be available depending on the executable configuration.

Abaqus output-request cards such as `*NODE FILE` are accepted where documented so that otherwise supported Abaqus decks do not fail solely because they contain common output requests. These cards currently do not control FEMaster's result writers.

15.1 *OVERVIEW

`*OVERVIEW` prints a diagnostic summary of the current model without changing the analysis.

The overview is intended to answer basic model-verification questions before numerical results are interpreted: which Parts and Instances exist, how many nodes and elements are present, which named sets and surfaces are available, which materials and Sections have been defined, and which loads, supports, and constraints are present.

In models with a Part/Instance hierarchy or many named regions, the overview exposes missing Instances, empty sets, unexpected entity counts, and omitted property assignments independently of the linear solver.

A structurally plausible overview establishes only the reported model organization. It does not establish the physical consistency of material units, element orientations, shell normals, surface sides, load directions, or boundary conditions.

FEMaster and Abaqus example

FEMaster

```
*OVERVIEW
```

Abaqus input

No Abaqus input keyword maps directly to FEMaster's native `*OVERVIEW` diagnostic.

15.2 *REQUESTSTIFFNESS

*REQUESTSTIFFNESS requests stiffness-matrix output from supported native analyses. It is available for linear static, nonlinear static, and linear buckling loadcases. FILE optionally specifies the output filename; otherwise FEMaster generates a deterministic default name.

For linear static analysis, the exported matrix exposes symmetry, rank, sparsity, condition-related behaviour, and differences from another finite-element implementation. For nonlinear static analysis, the relevant stiffness is the tangent operator associated with the state at which the output is produced. In buckling analysis, the elastic stiffness can be exported separately from the geometric stiffness.

Matrix output is a diagnostic and verification feature distinct from the ordinary result Field output used for engineering post-processing.

Parameters

Parameter	Status	Meaning
FILE	optional	Stiffness-matrix output filename.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARSTATIC
*REQUESTSTIFFNESS, FILE=K_service.txt
...
*END
```

Abaqus input

Abaqus matrix-output controls are currently not translated to FEMaster's native matrix diagnostics.

15.3 *REQUESTSTGEOM

*REQUESTSTGEOM requests geometric-stiffness output for LINEARBUCKLING. FILE optionally specifies the output filename or base name.

Linear buckling uses a stress-dependent geometric stiffness K_G generated by the reference preload state. Exporting K_G exposes the destabilizing contribution of the applied preload and, together with *REQUESTSTIFFNESS, permits independent inspection of the generalized buckling problem.

The command changes output only; it does not modify the preload or buckling eigenproblem.

Parameters

Parameter	Status	Meaning
FILE	optional	Geometric-stiffness output filename/base.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARBUCKLING
*REQUESTSTIFFNESS, FILE=K.txt
*REQUESTSTGEOM, FILE=KG.txt
...
*END
```

Abaqus input

No corresponding geometric-stiffness export card is currently translated from Abaqus input.

15.4 *CONSTRAINTSUMMARY

*CONSTRAINTSUMMARY enables diagnostic reporting of the constraint equations used by the active native loadcase.

When several constraint sources act simultaneously, support collectors, RBM suppression, couplings, ties, connectors, and other kinematic relations can all contribute equations. The summary reports their number and grouping.

In an unexpectedly singular or over-constrained model, the constraint groups can expose a duplicate coupling or a support that constrains a reference node twice more directly than the final sparse matrix.

*CONSTRAINTSUMMARY does not choose the enforcement method. *CONSTRAINTMETHOD separately determines whether the equations are handled by nullspace projection, elimination, or Lagrange multipliers where supported.

FEMaster and Abaqus example

FEMaster

```
*LOADCASE, TYPE=LINEARSTATIC
*CONSTRAINTSUMMARY
*CONSTRAINTMETHOD, TYPE=NULLSPACE
...
*END
```

Abaqus input

No Abaqus output-request card maps to this FEMaster constraint diagnostic.

15.5 Abaqus output-request keywords

The supported Abaqus reader accepts four common file/print request keywords: *NODE FILE, *EL FILE, *NODE PRINT, and *EL PRINT. Their requested variable names are read so that the deck remains syntactically acceptable, but the cards do not filter or configure FEMaster result output.

Acceptance of an output card does not imply that FEMaster reproduces the corresponding Abaqus output-file semantics. Post-processing is based on FEMaster's documented result files and Fields.

15.6 *NODE FILE

`*NODE FILE` is accepted in supported Abaqus input but does not create an Abaqus-style node file. Requested variables such as `U` or `RF` do not change which FEMaster result Fields are produced.

FEMaster and Abaqus example

FEMaster

Native FEMaster has no `*NODE FILE` input keyword. Nodal results are written by the enabled FEMaster result writers.

Abaqus input

```
*NODE FILE
U, RF
```

15.7 *EL FILE

*EL FILE is accepted and ignored in the same sense as *NODE FILE. It does not create a separate Abaqus-style element file and does not select a subset of FEMaster element Fields.

FEMaster and Abaqus example

FEMaster

Native FEMaster has no *EL FILE keyword.

Abaqus input

```
*EL FILE
S, E
```

15.8 *NODE PRINT

*NODE PRINT is accepted so that imported Abaqus decks may retain common print requests, but it does not generate an Abaqus-formatted nodal print table.

FEMaster and Abaqus example

FEMaster

No native *NODE PRINT keyword is used for FEMaster result selection.

Abaqus input

```
*NODE PRINT
U, RF
```

15.9 *EL PRINT

*EL PRINT is the element counterpart of *NODE PRINT. The keyword is accepted but does not configure FEMaster result output.

FEMaster and Abaqus example

FEMaster

No native *EL PRINT keyword is used for FEMaster result selection.

Abaqus input

```
*EL PRINT
S, E
```

15.10 Native .res files

The native .res file is organized into loadcase and Field blocks. A loadcase header has the form

Syntax

```
LC <id>
```

and is followed by the Fields produced by that analysis.

Indexed Field storage

All standard result Fields associated with model entities are written in indexed form. Every row therefore contains one or more explicit index columns followed by the Field values. The header states both counts:

Syntax

```
FIELD, NAME=<name>, TYPE=<type>,
INDEX_COLS=<nindex>, VALUE_COLS=<nvalue>, ROWS=<rows>
<index-1> ... <index-n> <value-1> ... <value-m>
...
END FIELD
```

The indices identify the corresponding model entity rather than an implicit row number. Results therefore remain traceable to the user-visible node and element labels even when the model contains Parts and multiple Instances.

Entities belonging to the implicit default Instance retain their unqualified local identifier, for example 8. Entities belonging to an explicitly named Instance use a qualified identifier of the form

Syntax

```
<instance>.<local-id>
```

such as TRUSS_A.8 or TRUSS_B.10. The same convention is used for node and element identifiers.

The number and meaning of the index columns depend on the Field domain:

Field type	Index columns	Meaning
NODE	1	node identifier
ELEMENT	1	element identifier
ELEMENT_NODAL	2	element identifier, local node index
ELEMENT_IP	2	element identifier, local integration-point index
ELEMENT_MP	2	element identifier, local material-point index

Local node, integration-point, and material-point indices are zero based. User node and element identifiers are non-negative labels and may themselves also start at 0; these are separate concepts.

A typical nodal Field therefore looks like:

Syntax

```
FIELD, NAME=DISPLACEMENT, TYPE=NODE,
INDEX_COLS=1, VALUE_COLS=6, ROWS=4
               1    0.000000e+00    0.000000e+00 ...
      TRUSS_A.8    4.761905e-02   -4.761905e-02 ...
       TRUSS_B.1    0.000000e+00    0.000000e+00 ...
      TRUSS_B.10    4.761905e-02    0.000000e+00 ...
END FIELD
```

For ELEMENT_NODAL, ELEMENT_IP, and ELEMENT_MP, the first index is the semantic element identifier and the second is the zero-based local position within that element. This keeps the external result format independent of any internal element ordering.

Common result quantities

Typical Fields depend on analysis type:

Analysis	Typical results
Linear static	Displacement, strain, stress, shell top/bottom stresses and resultants where applicable, external forces, reactions, section forces, and related recovered quantities.
Nonlinear static	Accepted-increment displacement/stress/load-factor states plus final displacement, strain, stress, internal/external forces, and reactions.
Eigenfrequency	Mode shapes, eigenvalues, eigenfrequencies/frequencies, and participation-related quantities where available.
Linear buckling	Buckling mode shapes and buckling factors.
Linear transient	Displacement, velocity, and acceleration at written time frames.
Linear harmonic	Real and imaginary displacement, stress, and strain amplitudes for the requested excitation frequencies.

Generalized nodal motion and force vectors use the six-component nodal order Ux, Uy, Uz, Rx, Ry, Rz. Stress and strain tensors instead use the engineering order

$$(x, y, z, yz, zx, xy).$$

The component meaning of raw result data therefore depends on whether the Field contains generalized nodal quantities or stress/strain tensors.

Harmonic results

A complex harmonic quantity is written as separate real and imaginary components:

$$\hat{q} = q_{\text{real}} + i q_{\text{imag}}.$$

Amplitude and phase can be reconstructed from

$$|\hat{q}| = \sqrt{q_{\text{real}}^2 + q_{\text{imag}}^2}, \qquad \varphi = \operatorname{atan2}(q_{\text{imag}}, q_{\text{real}}).$$

Harmonic results distinguish the real part at a chosen phase, the complex amplitude magnitude, and signed components. These quantities are not interchangeable.

15.11 FRD visualization output

FRD provides a CalculiX/CGX-compatible representation of the mesh and supported nodal results. It is intended primarily for visualization. Data that do not naturally map to FRD nodal blocks—for example some element-local, integration-point, material-point, scalar-history, or participation quantities—remain available in the native result representation.

Typical mappings include:

FEMaster result	FRD block	Content
Displacement / mode / buckling vector	`DISP`	Translational and supported generalized displacement components.
Velocity	`VELO`	Nodal velocity components.
Acceleration	`ACCE`	Nodal acceleration components.
Reaction force	`FORC`	Generalized reaction-force components.
Stress	`STRESS`	Nodal recovered stress components.
Strain	`TOSTRAIN`	Nodal recovered strain components.
Shell resultants	`SHR`	Membrane, bending, and transverse-shear resultants.

FRD requires numeric node identifiers and therefore cannot use the qualified string identifiers employed by `.res`. FEMaster assigns every written node the deterministic FRD identifier

$$n_{\text{FRD}} = 10^8 \, i_{\text{instance}} + n_{\text{local}},$$

where i_{instance} is the numeric Instance identifier and n_{local} is the original Part-local node ID. The local node ID must be smaller than 10^8. The same FRD node numbering is used consistently for coordinates, element connectivity, and nodal result blocks. Thus identifiers such as `100000001` and `200000001` represent the same local node label in different Instances; they are output identifiers and do not replace the semantic `INSTANCE.local-id` notation used in `.res`.

User node and element IDs in FEMaster may include `0`. FRD numbering is an output-format concern and does not impose a positive-ID restriction on the FEMaster input model.

Higher-order element connectivity is reordered where necessary to satisfy FRD conventions. For shell resultants, an explicitly assigned Section orientation defines the local output basis. Without an explicit orientation, FEMaster uses a deterministic tangent basis derived from the shell geometry.

Transient FRD frames use physical time. Eigenfrequency and buckling frames use the corresponding spectral value where supported by the writer.

16 Advanced Model Features

This chapter collects native FEMaster features that extend the basic mesh–material–load workflow. `*POINTMASS` adds concentrated mass, rotary inertia, and optional nodal spring stiffness. The topology-static commands use element Fields to scale stiffness and rotate material directions without changing mesh connectivity.

These capabilities are currently native FEMaster features; the supported Abaqus input subset does not provide direct equivalents unless stated otherwise.

16.1 *POINTMASS

*POINTMASS assigns concentrated mass and optional generalized spring properties to a named node set. NSET identifies the target nodes.

The first value is one translational point mass m. It contributes equally to the three translational inertia directions. The next three values are rotary inertias

$$(I_x, I_y, I_z)$$

associated with the nodal rotational degrees of freedom. Translational and rotational spring constants may then be supplied. Missing values default to zero, so a pure lumped mass requires only the first scalar.

For one target node, the idealized diagonal generalized mass contribution has the form

$$M_p = \mathrm{diag}(m, m, m, I_x, I_y, I_z),$$

while the optional generalized spring contribution is

$$K_p = \mathrm{diag}(k_x, k_y, k_z, k_{rx}, k_{ry}, k_{rz}).$$

The exact active DOFs still depend on the surrounding structural model.

Point masses represent equipment, payloads, actuators, engines, sensors, or other concentrated inertia in eigenfrequency, transient, inertial-load, or inertia-relief calculations without explicitly meshing the attached body.

If the target set contains several nodes, the specified feature is applied at every target node. The mass value therefore represents mass *per target node*, not the total mass of the whole set. A total payload distributed over several nodes requires corresponding per-node values or a reference-node representation.

Spring constants must use consistent units. Translational stiffness has force/length units, while rotational stiffness has moment/radian units. Rotary inertia has mass-length^2 units.

Concentrated inertia placed away from a structure's centre of rotation contributes to rotational dynamics through both translational mass and rotary inertia. Omitting the rotary component changes the high-frequency response of a rigid payload idealization.

Parameters

Parameter	Status	Meaning
NSET	required	Target node set.

Data lines

Entry	Meaning
mass	Concentrated translational mass; default 0.
Ix, Iy, Iz	Rotary inertia components; default 0.
kx, ky, kz	Translational spring constants; default 0.
krx, kry, krz	Rotational spring constants; default 0.

FEMaster and Abaqus example

FEMaster

```
*POINTMASS, NSET=PAYLOAD
0.025,
1.2e-3, 1.2e-3, 2.0e-3,
1000., 1000., 0.,
0., 0., 0.
```

Abaqus input

The supported Abaqus reader currently does not map concentrated-mass or spring cards to native `*POINTMASS`.

16.2 Topology-weighted linear statics

`LINEARSTATICTOPO` extends linear static analysis with element-wise density and optional material-orientation Fields. It is intended for topology/sizing workflows in which design variables are supplied externally or updated between solver runs.

The density Field scales each element's constitutive stiffness by

$$s_e = \rho_e^p,$$

where ρ_e is the value selected by `*TOPODENSITY` and p is set by `*TOPOEXPONENT`. Thus an element with $\rho = 1$ retains its full stiffness, while intermediate density is penalized according to the exponent.

If an orientation Field is supplied, its three components are interpreted as element-wise Euler rotation angles. The resulting additional rotation is applied to the material coordinate system for the topology-aware solid response. The Field therefore changes material orientation, not element geometry.

The analysis also produces topology-related output such as density, compliance-related quantities, density sensitivity, volume, and orientation sensitivity where applicable.

16.3 *TOPODENSITY

*TOPODENSITY selects the scalar element Field ρ_e used by an active LINEARSTATICTOPO loadcase. FIELD is required; NAME is accepted as an alternative spelling. The referenced Field must use TYPE=ELEMENT and exactly one component.

The stiffness multiplier is

$$\rho_e^p.$$

The command does not automatically force values into the interval $[0, 1]$. An optimization based on physical density fractions therefore depends on an external workflow that supplies values consistent with that interpretation. Negative density values or inappropriate exponents can produce nonphysical stiffness scaling.

For a SIMP-style topology problem, $0 < \rho_{\min} \leq \rho_e \leq 1$ is commonly used to avoid exactly singular void elements, but the choice of minimum density belongs to the optimization strategy surrounding the FEM solve.

Compliance sensitivity contains factors proportional to $1/\rho_e$. Exactly zero density therefore introduces singular sensitivity behaviour in gradient-based topology workflows even though the associated zero-stiffness element has a direct conceptual interpretation.

Parameters

Parameter	Status	Meaning
FIELD / NAME	required	Scalar ELEMENT Field containing element densities.

FEMaster and Abaqus example

FEMaster

```
*FIELD, NAME=RHO, TYPE=ELEMENT,
 COLS=1, FILL=ZERO
0, 0.8
1, 1.0

*LOADCASE, TYPE=LINEARSTATICTOPO
*TOPODENSITY, FIELD=RHO
*TOPOEXPONENT
3.0
*END
```

Abaqus input

Topology-density Fields and LINEARSTATICTOPO are native FEMaster capabilities.

16.4 *TOPOORIENT

*TOPOORIENT selects a three-component ELEMENT Field containing additional material-orientation angles for LINEARSTATICTOPO. FIELD is required and NAME is accepted as an alias.

The three Field components are Euler rotation angles

$$(\theta_x, \theta_y, \theta_z)$$

in **radians**. They define the additional material rotation

$$R(\theta_x, \theta_y, \theta_z) = R_x(\theta_x)\, R_y(\theta_y)\, R_z(\theta_z).$$

This rotation acts in addition to the Section's base material orientation. It is therefore intended for optimization or parameter studies in which the preferred anisotropic material direction varies from element to element.

An orientation Field changes only directional constitutive response. It does not rotate the mesh and has no effect on a fully isotropic material law because isotropic stiffness is invariant under rotation.

The rotation order is significant because finite rotations about different axes do not commute. An external optimization or preprocessing tool that generates *TOPOORIENT values must therefore use the same $R_x R_y R_z$ convention and radians. For example, a second-component value of 0.2 means a rotation of 0.2 rad, not 0.2°.

Parameters

Parameter	Status	Meaning
FIELD / NAME	required	Three-component ELEMENT Field of Euler angles $(\theta_x, \theta_y, \theta_z)$ in radians.

FEMaster and Abaqus example

FEMaster

```
*FIELD, NAME=ORIENT,
 TYPE=ELEMENT, COLS=3, FILL=ZERO
0, 0.0, 0.0, 0.0
1, 0.0, 0.2, 0.0

*LOADCASE, TYPE=LINEARSTATICTOPO
*TOPODENSITY, FIELD=RHO
*TOPOORIENT, FIELD=ORIENT
*END
```

Abaqus input

No topology-orientation Field mapping is currently supported by the Abaqus reader.

16.5 *TOPOEXPONENT

*TOPOEXPONENT sets the scalar penalization exponent p used by LINEARSTATICTOPO:

$$K_e(\rho_e) = \rho_e^p K_{e,0}$$

for the element stiffness contribution represented by the topology scaling.

With $p = 1$, stiffness varies linearly with density. Values $p > 1$ penalize intermediate densities increasingly strongly and are commonly used in SIMP-style topology optimization to drive the design toward a more nearly discrete solid/void distribution.

The exponent also affects compliance sensitivity. If an element compliance contribution before the density derivative is denoted by c_e, the density sensitivity scales conceptually with

$$\frac{\partial C}{\partial \rho_e} \propto -p\, \rho_e^{p-1} c_e.$$

The exact reported Field follows FEMaster's topology result definitions, while the sign and penalization dependence follow this standard stiffness-scaling idea.

Data lines

Entry	Meaning
exponent	Scalar density penalization exponent p.

FEMaster and Abaqus example

FEMaster

```
*TOPOEXPONENT
3.0
```

Abaqus input

No corresponding topology penalization keyword is currently supported in Abaqus input.

17 Complete Examples

The examples in this chapter combine the individual keyword pages into complete modelling workflows. Each section describes one physical problem and presents native FEMaster and supported Abaqus input side by side. The examples intentionally keep explicit sets, properties, boundary conditions, loads, and analysis controls so that the relation between the two input styles remains visible.

Native FEMaster defines reusable support and load collectors and activates them inside a `*LOADCASE`. Abaqus input places supported `*BOUNDARY` and load cards inside a `*STEP`. Where the syntax differs, the examples use the exact FEMaster-supported representation rather than assuming that every feature of the full Abaqus product is available.

Several examples intentionally use node or element ID `0`. This demonstrates that zero is a valid FEMaster user ID; user IDs need not begin at one.

17.1 Linear static truss

The first model is a two-node axial bar in a consistent N–mm unit system. Node 0 is fixed and node 1 is guided so only axial translation remains free. A 1000 N force acts in the global x-direction. With

$$E = 210000 \text{ MPa}, \qquad L = 1000 \text{ mm}, \qquad A = 100 \text{ mm}^2,$$

the analytical displacement is

$$u_x = \frac{FL}{EA} = \frac{1000 \cdot 1000}{210000 \cdot 100} \approx 0.047619 \text{ mm}.$$

The case provides both a syntax template and a basic stiffness/reaction verification model.

FEMaster

```
*HEADING
Linear static axial truss

*NODE
0,    0., 0., 0.
1, 1000., 0., 0.
*ELEMENT, TYPE=T3, ELSET=BAR
0, 0, 1
*NSET, NAME=FIXED
0
*NSET, NAME=TIP
1

*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*TRUSS SECTION, ELSET=BAR,
 MATERIAL=STEEL
100.

*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0., 0., 0., NAN, NAN, NAN
TIP, NAN, 0., 0., NAN, NAN, NAN
*CLOAD, LOAD_COLLECTOR=AXIAL
TIP, 1000., 0., 0., 0., 0., 0.

*LOADCASE, TYPE=LINEARSTATIC,
 NAME=AXIAL_TENSION
*SUPPORTS
BC
*LOADS
AXIAL
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*END
```

Abaqus input

```
*HEADING
Linear static axial truss

*NODE
0,    0., 0., 0.
1, 1000., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
0, 0, 1
*NSET, NSET=FIXED
0
*NSET, NSET=TIP
1

*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*SOLID SECTION, ELSET=BAR,
 MATERIAL=STEEL
100.

*STEP, NAME=AXIAL_TENSION
*STATIC
*BOUNDARY
FIXED, 1, 3, 0.
TIP, 2, 3, 0.
*CLOAD
TIP, 1, 1000.
*END STEP
```

The axial reaction at the fixed node balances the applied 1000 N load to numerical precision. The corresponding axial stress is approximately

$$\sigma = \frac{F}{A} = 10 \text{ MPa}.$$

17.2 Reusable Part and translated Instance

This example demonstrates the optional Part/Assembly hierarchy. The material is defined once at model level. The reusable rod topology and Section assignment are defined inside a Part. One Instance translates the complete Part by 500 mm in x. Assembly-level node sets then identify the local endpoint IDs on that particular Instance.

The Part coordinates remain local to the reusable component. The Instance placement determines the actual position of that copy in the assembled structure. If a second Instance were added with a different placement, both copies could still use local node IDs 0 and 1 without manual renumbering.

FEMaster

```
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3

*PART, NAME=ROD
*NODE
0,   0., 0., 0.
1, 500., 0., 0.
*ELEMENT, TYPE=T3, ELSET=BAR
0, 0, 1
*TRUSS SECTION, ELSET=BAR,
 MATERIAL=STEEL
50.
*END PART

*ASSEMBLY
*INSTANCE, NAME=ROD_A, PART=ROD
500., 0., 0.
*END INSTANCE
*NSET, NSET=LEFT, INSTANCE=ROD_A
0
*NSET, NSET=RIGHT, INSTANCE=ROD_A
1
*END ASSEMBLY

*SUPPORT, SUPPORT_COLLECTOR=BC
LEFT, 0., 0., 0., NAN, NAN, NAN
RIGHT, NAN, 0., 0., NAN, NAN, NAN
*CLOAD, LOAD_COLLECTOR=P
RIGHT, 500., 0., 0., 0., 0., 0.

*LOADCASE, TYPE=LINEARSTATIC
*SUPPORTS
BC
*LOADS
P
*END
```

Abaqus input

```
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3

*PART, NAME=ROD
*NODE
0,   0., 0., 0.
1, 500., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
0, 0, 1
*SOLID SECTION, ELSET=BAR,
 MATERIAL=STEEL
50.
*END PART

*ASSEMBLY, NAME=ASSEMBLY
*INSTANCE, NAME=ROD_A, PART=ROD
500., 0., 0.
*END INSTANCE
*NSET, NSET=LEFT, INSTANCE=ROD_A
0
*NSET, NSET=RIGHT, INSTANCE=ROD_A
1
*END ASSEMBLY

*STEP
*STATIC
*BOUNDARY
LEFT, 1, 3, 0.
RIGHT, 2, 3, 0.
*CLOAD
RIGHT, 1, 500.
*END STEP
```

The left endpoint of the placed rod lies at $x = 500$ mm and the right endpoint at $x = 1000$ mm. No change to the Part's local coordinates was required.

17.3 Homogeneous shell cantilever

A rectangular shell panel is clamped along one edge and loaded by two equal transverse nodal forces at the free edge. The native model uses the current four-node finite-rotation shell `MITC4FRT`. The Abaqus label `S4` is read as the same FEMaster FRT shell family. Both Sections represent a homogeneous 2 mm shell.

This example is intentionally simple enough that shell normals and load direction can be checked visually. Nodes are ordered counter-clockwise when viewed from the positive shell-normal side. The free-edge load acts in negative global z.

FEMaster

```
*NODE
0,    0.,   0., 0.
1, 1000.,   0., 0.
2, 1000., 500., 0.
3,    0., 500., 0.
*ELEMENT, TYPE=MITC4FRT, ELSET=PANEL
0, 0, 1, 2, 3
*NSET, NAME=ROOT
0, 3
*NSET, NAME=TIP
1, 2

*MATERIAL, NAME=ALUMINIUM
*ELASTIC
70000., 0.33
*SHELL SECTION, ELSET=PANEL,
 MATERIAL=ALUMINIUM, TYPE=INTEGRATED
2.0

*SUPPORT, SUPPORT_COLLECTOR=BC
ROOT, 0., 0., 0., 0., 0., 0.
*CLOAD, LOAD_COLLECTOR=P
TIP, 0., 0., -50., 0., 0., 0.

*LOADCASE, TYPE=LINEARSTATIC
*SUPPORTS
BC
*LOADS
P
*END
```

Abaqus input

```
*NODE
0,    0.,   0., 0.
1, 1000.,   0., 0.
2, 1000., 500., 0.
3,    0., 500., 0.
*ELEMENT, TYPE=S4, ELSET=PANEL
0, 0, 1, 2, 3
*NSET, NSET=ROOT
0, 3
*NSET, NSET=TIP
1, 2

*MATERIAL, NAME=ALUMINIUM
*ELASTIC
70000., 0.33
*SHELL SECTION, ELSET=PANEL,
 MATERIAL=ALUMINIUM
2.0, 5

*STEP
*STATIC
*BOUNDARY
ROOT, 1, 6, 0.
*CLOAD
TIP, 3, -50.
*END STEP
```

Because `*CLOAD` applied to a node set acts on every node in that set, the total transverse force is -100 N in both decks. A total force of -50 N corresponds to -25 N at each free-edge node or to one -50 N reference-node load transferred through a distributing coupling.

17.4 Solid cube under pressure

The solid example demonstrates an element-face surface and scalar pressure loading. The native deck uses `*PLOAD`; Abaqus input uses `*DSLOAD` with type `P`. In both cases the load direction follows the selected finite-element surface orientation rather than an independently supplied global vector.

FEMaster

```
*NODE
0, 0., 0., 0.
1, 1., 0., 0.
2, 1., 1., 0.
3, 0., 1., 0.
4, 0., 0., 1.
5, 1., 0., 1.
6, 1., 1., 1.
7, 0., 1., 1.
*ELEMENT, TYPE=C3D8, ELSET=CUBE
0, 0, 1, 2, 3, 4, 5, 6, 7
*NSET, NAME=BASE
0, 1, 2, 3
*SURFACE, NAME=LOADED_FACE
CUBE, S2

*MATERIAL, NAME=MAT
*ELASTIC
10000., 0.3
*SOLID SECTION, ELSET=CUBE, MATERIAL=MAT

*SUPPORT, SUPPORT_COLLECTOR=BC
BASE, 0., 0., 0., NAN, NAN, NAN
*PLOAD, LOAD_COLLECTOR=P
LOADED_FACE, 1.0

*LOADCASE, TYPE=LINEARSTATIC
*SUPPORTS
BC
*LOADS
P
*END
```

Abaqus input

```
*NODE
0, 0., 0., 0.
1, 1., 0., 0.
2, 1., 1., 0.
3, 0., 1., 0.
4, 0., 0., 1.
5, 1., 0., 1.
6, 1., 1., 1.
7, 0., 1., 1.
*ELEMENT, TYPE=C3D8, ELSET=CUBE
0, 0, 1, 2, 3, 4, 5, 6, 7
*NSET, NSET=BASE
0, 1, 2, 3
*SURFACE, NAME=LOADED_FACE,
 TYPE=ELEMENT
CUBE, S2

*MATERIAL, NAME=MAT
*ELASTIC
10000., 0.3
*SOLID SECTION, ELSET=CUBE, MATERIAL=MAT

*STEP
*STATIC
*BOUNDARY
BASE, 1, 3, 0.
*DSLOAD
LOADED_FACE, P, 1.0
*END STEP
```

For `C3D8`, the physical face represented by `S2` and its normal direction determine the pressure sign. The surface side is part of the model definition.

17.5 Eigenfrequency analysis

The modal example uses a two-element truss with distributed material density and requests the lowest five eigenpairs. Only axial motion is left free. The common example intentionally uses distributed density rather than `*POINTMASS`, because the current Abaqus input subset does not provide an equivalent concentrated-mass mapping.

The natural frequencies are governed by

$$K\phi_i = \omega_i^2 M\phi_i.$$

Changing density therefore changes the frequencies even though the static stiffness is unchanged.

FEMaster

```
*NODE
0,    0., 0., 0.
1,  500., 0., 0.
2, 1000., 0., 0.
*ELEMENT, TYPE=T3, ELSET=BAR
0, 0, 1
1, 1, 2
*NSET, NAME=FIXED
0
*NSET, NAME=GUIDED
1, 2
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*DENSITY
7.85e-9
*TRUSS SECTION, ELSET=BAR, MATERIAL=STEEL
100.
*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0., 0., 0., NAN, NAN, NAN
GUIDED, NAN, 0., 0., NAN, NAN, NAN

*LOADCASE, TYPE=EIGENFREQ
*SUPPORTS
BC
*NUMEIGENVALUES
5
*END
```

Abaqus input

```
*NODE
0,    0., 0., 0.
1,  500., 0., 0.
2, 1000., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
0, 0, 1
1, 1, 2
*NSET, NSET=FIXED
0
*NSET, NSET=GUIDED
1, 2
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*DENSITY
7.85e-9
*SOLID SECTION, ELSET=BAR, MATERIAL=STEEL
100.

*STEP, PERTURBATION
*FREQUENCY
5
*BOUNDARY
FIXED, 1, 3, 0.
GUIDED, 2, 3, 0.
*END STEP
```

The first mode shapes identify whether the corresponding numerical frequencies belong to structural deformation or to an unintended mechanism. A plausible frequency attached to an unintended mechanism is not a valid modal result.

17.6 Linear buckling of a compressed member

A linear buckling analysis uses a reference preload to construct geometric stiffness and then extracts load multipliers. The example below uses a slender truss only to demonstrate the input structure; a flexural column requires a beam, shell, or solid model with transverse/bending degrees of freedom.

The buckling factors satisfy a generalized eigenproblem involving the elastic stiffness and preload-dependent geometric stiffness. They multiply the reference load rather than representing absolute force directly.

FEMaster

```
*NODE
0, 0., 0., 0.
1, 1000., 0., 0.
*ELEMENT, TYPE=T3, ELSET=MEMBER
0, 0, 1
*NSET, NAME=LEFT
0
*NSET, NAME=RIGHT
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*TRUSS SECTION, ELSET=MEMBER, MATERIAL=STEEL
100.
*SUPPORT, SUPPORT_COLLECTOR=BC
LEFT, 0., 0., 0., NAN, NAN, NAN
RIGHT, NAN, 0., 0., NAN, NAN, NAN
*CLOAD, LOAD_COLLECTOR=PRELOAD
RIGHT, -1000., 0., 0., 0., 0., 0.

*LOADCASE, TYPE=LINEARBUCKLING
*SUPPORTS
BC
*LOADS
PRELOAD
*NUMEIGENVALUES
5
*SIGMA
0.0
*END
```

Abaqus input

```
*NODE
0, 0., 0., 0.
1, 1000., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=MEMBER
0, 0, 1
*NSET, NSET=LEFT
0
*NSET, NSET=RIGHT
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*SOLID SECTION, ELSET=MEMBER, MATERIAL=STEEL
100.

*STEP, PERTURBATION
*BUCKLE
5
*BOUNDARY
LEFT, 1, 3, 0.
RIGHT, 2, 3, 0.
*CLOAD
RIGHT, 1, -1000.
*END STEP
```

The model above is deliberately a syntax example rather than a flexural-buckling benchmark: an axial truss does not possess beam bending stiffness. A column model based on flexural buckling requires `B33`, shell, or solid elements and boundary conditions that admit the relevant buckling mode.

17.7 Arc-length snap-through

A shallow two-bar arch illustrates geometrically nonlinear path following. The native deck selects `ARC_LENGTH`; Abaqus input selects `*STATIC, RIKS`. The centre node is constrained to remain in the $x = 0$ plane and to have zero out-of-plane motion, while vertical displacement remains free.

Arc-length control permits the equilibrium path to pass a limit point at which load factor no longer increases monotonically with displacement.

FEMaster

```
*NODE
0, -1000.,   0., 0.
1,     0., 100., 0.
2,  1000.,   0., 0.
*ELEMENT, TYPE=T3, ELSET=BARS
0, 0, 1
1, 1, 2
*NSET, NAME=ENDS
0, 2
*NSET, NAME=APEX
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*TRUSS SECTION, ELSET=BARS, MATERIAL=STEEL
100.
*SUPPORT, SUPPORT_COLLECTOR=BC
ENDS, 0., 0., 0., NAN, NAN, NAN
APEX, 0., NAN, 0., NAN, NAN, NAN
*CLOAD, LOAD_COLLECTOR=P
APEX, 0., -20000., 0., 0., 0., 0.

*LOADCASE, TYPE=NONLINEARSTATIC
*SUPPORTS
BC
*LOADS
P
*NONLINEAR, CONTROL=ARC_LENGTH,
 MAX_INCREMENTS=200,
 INITIAL_INCREMENT=0.05,
 MINIMUM_INCREMENT=1e-5,
 MAXIMUM_INCREMENT=0.1,
 MAXITER=40, TOL=1e-8
*END
```

Abaqus input

```
*NODE
0, -1000.,   0., 0.
1,     0., 100., 0.
2,  1000.,   0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BARS
0, 0, 1
1, 1, 2
*NSET, NSET=ENDS
0, 2
*NSET, NSET=APEX
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*SOLID SECTION, ELSET=BARS, MATERIAL=STEEL
100.

*STEP, NLGEOM=YES, INC=200
*STATIC, RIKS
0.05, 1.0, 1e-5, 0.1
*BOUNDARY
ENDS, 1, 3, 0.
APEX, 1, 1, 0.
APEX, 3, 3, 0.
*CLOAD
APEX, 2, -20000.
*END STEP
```

For nonlinear verification, plot load factor against apex displacement. A converged sequence of increments is much more informative than the final state alone because the characteristic feature of snap-through is the shape of the equilibrium path.

17.8 Linear transient ramp load

The transient model applies an axial load with a tabular amplitude. Native FEMaster defines the integration interval and Newmark parameters directly. Abaqus `*DYNAMIC, DIRECT` supplies the fixed time increment and Step period. The default Newmark parameters are $\beta = 0.25$ and $\gamma = 0.5$.

FEMaster

```
*NODE
0,    0., 0., 0.
1, 1000., 0., 0.
*ELEMENT, TYPE=T3, ELSET=BAR
0, 0, 1
*NSET, NAME=FIXED
0
*NSET, NAME=TIP
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*DENSITY
7.85e-9
*TRUSS SECTION, ELSET=BAR, MATERIAL=STEEL
100.
*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0., 0., 0., NAN, NAN, NAN
TIP, NAN, 0., 0., NAN, NAN, NAN
*AMPLITUDE, NAME=RAMP
0., 0.
0.1, 1.
1., 1.
*CLOAD, LOAD_COLLECTOR=P, AMPLITUDE=RAMP
TIP, 1000., 0., 0., 0., 0., 0.

*LOADCASE, TYPE=LINEARTRANSIENT
*SUPPORTS
BC
*LOADS
P
*TIME
0., 1., 0.01
*NEWMARK
0.25, 0.5
*WRITEEVERY, TYPE=STEPS
10
*END
```

Abaqus input

```
*NODE
0,    0., 0., 0.
1, 1000., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
0, 0, 1
*NSET, NSET=FIXED
0
*NSET, NSET=TIP
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*DENSITY
7.85e-9
*SOLID SECTION, ELSET=BAR, MATERIAL=STEEL
100.
*AMPLITUDE, NAME=RAMP
0., 0., 0.1, 1., 1., 1.

*STEP
*DYNAMIC, DIRECT
0.01, 1.0
*BOUNDARY
FIXED, 1, 3, 0.
TIP, 2, 3, 0.
*CLOAD, AMPLITUDE=RAMP
TIP, 1, 1000.
*NODE FILE
U
*END STEP
```

The `*NODE FILE` card on the Abaqus side is accepted but does not control FEMaster output. The native `*WRITEEVERY` setting determines the requested transient write cadence for the FEMaster form.

17.9 Linear harmonic frequency sweep

The harmonic example evaluates a real axial harmonic force from 10 to 1000 frequency units at 100 linearly spaced points. The dynamic stiffness is

$$K_{\mathrm{dyn}}(\omega) = K - \omega^2 M + i\omega C.$$

Rayleigh damping is added on the native side with a small stiffness-proportional coefficient. The current Abaqus input subset does not import an equivalent damping card in this example, so the two decks are identical in geometry and forcing but not in damping unless the native damping line is removed.

FEMaster

```
*NODE
0, 0., 0., 0.
1, 1000., 0., 0.
*ELEMENT, TYPE=T3, ELSET=BAR
0, 0, 1
*NSET, NAME=FIXED
0
*NSET, NAME=TIP
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*DENSITY
7.85e-9
*TRUSS SECTION, ELSET=BAR, MATERIAL=STEEL
100.
*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0., 0., 0., NAN, NAN, NAN
TIP, NAN, 0., 0., NAN, NAN, NAN
*CLOAD, LOAD_COLLECTOR=HARMONIC
TIP, 1., 0., 0., 0., 0., 0.

*LOADCASE, TYPE=LINEARHARMONIC
*SUPPORTS
BC
*LOADS
HARMONIC
*FREQUENCIES, SCALE=LINEAR
10., 1000., 100
*DAMPING, TYPE=RAYLEIGH
0., 1e-5
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*END
```

Abaqus input

```
*NODE
0, 0., 0., 0.
1, 1000., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
0, 0, 1
*NSET, NSET=FIXED
0
*NSET, NSET=TIP
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*DENSITY
7.85e-9
*SOLID SECTION, ELSET=BAR, MATERIAL=STEEL
100.

*STEP
*STEADY STATE DYNAMICS, DIRECT,
 FREQUENCYSCALE=LINEAR
10., 1000., 100, 1., 1.
*BOUNDARY
FIXED, 1, 3, 0.
TIP, 2, 3, 0.
*CLOAD, REAL
TIP, 1, 1.
*END STEP
```

When comparing the two harmonic decks numerically, remove the native `*DAMPING` line or otherwise ensure that both calculations use the same damping model. Harmonic response can change dramatically near resonance even for small damping differences.

17.10 Using the examples as templates

The examples follow the dependency order of the model: geometry and element connectivity precede regions, materials, and Sections; loads and constraints depend on those definitions; numerical solver controls act on the resulting analysis system.

The supported Abaqus input consists of the syntax documented in this manual. A keyword or option that exists in Abaqus itself is not automatically supported by FEMaster, and native FEMaster collector and Field keywords are part of an Abaqus deck only where such mixed use is explicitly documented.

The side-by-side layout is a modelling translation guide: the left and right columns express the same physical finite-element problem wherever both input styles support it. When one side lacks an equivalent feature, the text states the difference explicitly rather than hiding it behind an internal implementation detail.

18 Keyword Index

This chapter provides a compact index of the input keywords accepted by the current FEMaster and supported Abaqus syntaxes. Every keyword has one canonical explanation in the preceding thematic chapters. A check mark means that the spelling is accepted in that input style; a dash means that the same keyword is not available there, although an equivalent modelling operation may be shown on the detailed page.

Command names are displayed in the conventional spaced form used in the manual, for example `*END PART`, `*SOLID SECTION`, and `*STEADY STATE DYNAMICS`. `*INERTIALOAD` is shown with the exact native spelling currently accepted by FEMaster.

Table 18.1: Input keywords and their canonical documentation locations.

Keyword	FEMaster	Abaqus	Canonical topic
`*HEADING`	✓	✓	Input Format
`*MODEL`	✓	–	Model Definition
`*PART`	✓	✓	Model Definition
`*END PART`	✓	✓	Model Definition
`*ASSEMBLY`	✓	✓	Model Definition
`*END ASSEMBLY`	✓	✓	Model Definition
`*INSTANCE`	✓	✓	Model Definition
`*END INSTANCE`	✓	✓	Model Definition
`*NODE`	✓	✓	Model Definition
`*ELEMENT`	✓	✓	Model Definition and Elements
`*NSET`	✓	✓	Regions and Selections
`*ELSET`	✓	✓	Regions and Selections
`*SURFACE`	✓	✓	Regions and Selections
`*SFSET`	✓	–	Regions and Selections
`*MATERIAL`	✓	✓	Materials
`*ELASTIC`	✓	✓	Materials
`*HYPERELASTIC`	✓	✓	Materials
`*DENSITY`	✓	✓	Materials
`*THERMALEXPANSION`	✓	–	Materials
`*EXPANSION`	–	✓	Materials
`*PROFILE`	✓	–	Sections and Element Properties
`*SOLID SECTION`	✓	✓	Sections and Element Properties
`*SHELL SECTION`	✓	✓	Sections and Element Properties
`*BEAM SECTION`	✓	–	Sections and Element Properties
`*TRUSS SECTION`	✓	–	Sections and Element Properties
`*ORIENTATION`	✓	✓	Coordinate Systems and Orientations
`*TRANSFORM`	–	✓	Coordinate Systems and Orientations
`*FIELD`	✓	–	Fields
`*NORMAL`	✓	–	Fields
`*AMPLITUDE`	✓	✓	Loads and Boundary Conditions
`*SUPPORT`	✓	–	Loads and Boundary Conditions
`*BOUNDARY`	–	✓	Loads and Boundary Conditions

Keyword	FEMaster	Abaqus	Canonical topic
*CLOAD	✓	✓	Loads and Boundary Conditions
*DLOAD	✓	✓	Loads and Boundary Conditions; semantics differ by dialect
*PLOAD	✓	–	Loads and Boundary Conditions
*DSLOAD	–	✓	Loads and Boundary Conditions
*VLOAD	✓	–	Loads and Boundary Conditions
*TLOAD	✓	–	Loads and Boundary Conditions
*INERTIALOAD	✓	–	Loads and Boundary Conditions
*COUPLING	✓	–	Constraints
*TIE	✓	–	Constraints
*CONNECTOR	✓	–	Constraints
*RBM	✓	–	Constraints
*CONTACT	✓	–	Constraints: Contact
*LOADCASE	✓	–	Analysis Procedures
*SUPPORTS	✓	–	Analysis Procedures
*LOADS	✓	–	Analysis Procedures
*END	✓	–	Analysis Procedures
*STEP	–	✓	Analysis Procedures
*STATIC	–	✓	Analysis Procedures
*FREQUENCY	–	✓	Analysis Procedures
*BUCKLE	–	✓	Analysis Procedures
*DYNAMIC	–	✓	Analysis Procedures
*STEADY STATE DYNAMICS	–	✓	Analysis Procedures
*END STEP	–	✓	Analysis Procedures
*SOLVER	✓	–	Solver and Numerical Controls
*CONSTRAINTMETHOD	✓	–	Solver and Numerical Controls
*NONLINEAR	✓	–	Solver and Numerical Controls
*TIME	✓	–	Solver and Numerical Controls
*NEWMARK	✓	–	Solver and Numerical Controls
*DAMPING	✓	–	Solver and Numerical Controls
*FREQUENCIES	✓	–	Solver and Numerical Controls
*NUMEIGENVALUES	✓	–	Solver and Numerical Controls
*SIGMA	✓	–	Solver and Numerical Controls
*WRITEEVERY	✓	–	Solver and Numerical Controls
*INITIALVELOCITY	✓	–	Solver and Numerical Controls
*INERTIARELIEF	✓	–	Solver and Numerical Controls
*REBALANCELOADS	✓	–	Solver and Numerical Controls
*OVERVIEW	✓	–	Output, Diagnostics, and Results
*REQUESTSTIFFNESS	✓	–	Output, Diagnostics, and Results
*REQUESTSTGEOM	✓	–	Output, Diagnostics, and Results
*CONSTRAINTSUMMARY	✓	–	Output, Diagnostics, and Results
*NODE FILE	–	✓	Output; accepted without controlling FEMaster writers
*EL FILE	–	✓	Output; accepted without controlling FEMaster writers
*NODE PRINT	–	✓	Output; accepted without controlling FEMaster writers
*EL PRINT	–	✓	Output; accepted without controlling FEMaster writers
*POINTMASS	✓	–	Advanced Model Features
*TOPODENSITY	✓	–	Advanced Model Features
*TOPOORIENT	✓	–	Advanced Model Features
*TOPOEXPONENT	✓	–	Advanced Model Features

18.1 Supported element formulations

The *ELEMENT keyword selects one of the formulations documented in Chapter 5. The following table provides a compact element-type index. The Abaqus column lists the spelling accepted in Abaqus input when it differs from the native FEMaster name.

FEMaster formulation	Abaqus input	Family
C3D4	C3D4	4-node linear tetrahedral solid
C3D5	C3D5	5-node pyramid input / transition solid
C3D6	C3D6	6-node linear wedge solid
C3D8	C3D8	8-node full-integration hexahedral solid
C3D8R	C3D8R	8-node reduced-integration hexahedral solid
C3D10	C3D10	10-node quadratic tetrahedral solid
C3D15	C3D15	15-node quadratic wedge solid
C3D20	C3D20	20-node quadratic hexahedral solid
C3D20R	C3D20R	20-node reduced-integration quadratic hexahedral solid
S3	–	Legacy native 3-node shell
MITC3FRT	S3, S3R	Current finite-rotation 3-node shell
S4	–	Legacy native 4-node shell
MITC4	–	Legacy native 4-node MITC shell
MITC4FRT	S4, S4R	Current finite-rotation 4-node MITC shell
S6	–	Legacy native 6-node shell
MITC6FRT	S6, S6R	Current finite-rotation 6-node shell
S8	–	Legacy native 8-node shell
MITC8	–	Legacy native 8-node MITC shell
MITC8FRT	S8, S8R	Current finite-rotation 8-node MITC shell
QSPT	–	Native quadrilateral shear-panel element
B33	B33	2-node 3D beam
T3	T3D2	2-node 3D truss

The Abaqus shell mapping is intentionally explicit. For example, Abaqus S4 and S4R select FEMaster's MITC4FRT; they do not select the legacy native S4. Detailed topology figures, formulation notes, and element-selection guidance are given in the Elements chapter.

18.2 Coverage notes

A check mark in the Abaqus column means that the documented subset of that keyword is accepted by FEMaster. It does not imply support for every option available in the complete Abaqus product.

Likewise, identical names do not always imply identical data semantics. *DLOAD is the clearest example: native FEMaster uses it for vector surface traction, while supported Abaqus *DLOAD currently covers GRAV body acceleration. The detailed keyword page is authoritative whenever the two input styles differ.

The four Abaqus output cards *NODE FILE, *EL FILE, *NODE PRINT, and *EL PRINT are listed because they are accepted input keywords. They do not currently configure FEMaster output selection.

The index, detailed thematic documentation, and applicable complete examples together represent every added input keyword or element formulation. This keeps the manual aligned with the syntax users can actually run.

A Mathematical Derivations

This appendix derives the topology-related quantities written by FEMaster. The main reference chapters describe the DSL commands, while this appendix gives the mathematical background for compliance, density sensitivities, and orientation sensitivities.

A.1 Element stiffness

For a linear elastic finite element, the unmodified element stiffness is

$$K_{e,0} = \int_{\Omega_e} B^T D B \, d\Omega.$$

B is the strain-displacement matrix, D is the constitutive matrix, and Ω_e is the physical element domain. With an isoparametric mapping from natural coordinates $r = (\xi, \eta, \zeta)$, the integral is evaluated as

$$K_{e,0} = \int_{\Omega'_e} B(r)^T D B(r) \, |\det J(r)| \, d\xi \, d\eta \, d\zeta.$$

For numerical integration with quadrature points r_q and weights w_q, this becomes

$$K_{e,0} \approx \sum_{q=1}^{Q} w_q B(r_q)^T D B(r_q) \, |\det J(r_q)|.$$

A.2 Compliance

For a linear static analysis,

$$Ku = f.$$

The external work measure commonly used for topology optimization is the compliance

$$C = f^T u.$$

Using the equilibrium equation $f = Ku$, this can also be written as

$$C = u^T K u.$$

Because the global stiffness matrix is assembled from element stiffness matrices, compliance can be decomposed into element contributions:

$$C = \sum_{e=1}^{n_e} u_e^T K_e u_e.$$

This expression is the basis for element-wise topology sensitivities. u_e is the element displacement vector extracted from the global solution.

A.3 Density penalization

In a density-based topology formulation, an element density variable ρ_e scales the stiffness contribution. With a penalization exponent p,

$$K_e(\rho_e) = \rho_e^p K_{e,0}.$$

The exponent p makes intermediate densities less efficient and therefore encourages designs closer to a solid-void distribution.

The element compliance contribution is

$$C_e(\rho_e) = u_e^T K_e(\rho_e) u_e = \rho_e^p u_e^T K_{e,0} u_e.$$

If u_e is held fixed for the local sensitivity expression, the derivative is

$$\frac{\partial C_e}{\partial \rho_e} = p\rho_e^{p-1} u_e^T K_{e,0} u_e.$$

Depending on the optimization convention, the objective derivative may be reported with the opposite sign because minimizing compliance through the equilibrium equation often uses the adjoint result

$$\frac{dC}{d\rho_e} = -u_e^T \frac{\partial K_e}{\partial \rho_e} u_e = -p\rho_e^{p-1} u_e^T K_{e,0} u_e.$$

The sign convention is therefore defined together with the optimizer interface and the Field name written by the analysis.

A.4 Orientation-dependent stiffness

For orientation-based topology or material steering, the constitutive matrix is rotated before the element stiffness is evaluated. Let the material orientation be represented by three angles

$$\alpha = (\alpha_1, \alpha_2, \alpha_3).$$

The elementary rotation matrices are

$$R_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha_1 & -\sin\alpha_1 \\ 0 & \sin\alpha_1 & \cos\alpha_1 \end{bmatrix},$$

$$R_2 = \begin{bmatrix} \cos\alpha_2 & 0 & \sin\alpha_2 \\ 0 & 1 & 0 \\ -\sin\alpha_2 & 0 & \cos\alpha_2 \end{bmatrix}, \qquad R_3 = \begin{bmatrix} \cos\alpha_3 & -\sin\alpha_3 & 0 \\ \sin\alpha_3 & \cos\alpha_3 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

For the convention used here, the combined rotation is

$$R = R_1 R_2 R_3.$$

The multiplication order must be used consistently. Different conventions can describe the same physical orientation with different angle triples.

A.5 Voigt transformation

The strain vector uses engineering shear strains,

$$\varepsilon = \begin{bmatrix} \varepsilon_{xx} & \varepsilon_{yy} & \varepsilon_{zz} & 2\varepsilon_{yz} & 2\varepsilon_{xz} & 2\varepsilon_{xy} \end{bmatrix}^T,$$

and the stress vector is

$$\sigma = \begin{bmatrix} \sigma_{xx} & \sigma_{yy} & \sigma_{zz} & \sigma_{yz} & \sigma_{xz} & \sigma_{xy} \end{bmatrix}^T.$$

The strain transformation is written as

$$\varepsilon_m = T(R)\varepsilon_g,$$

where T is the 6×6 Voigt transformation matrix built from the entries of R. The rotated constitutive matrix used in global strain coordinates is

$$D_{\mathrm{rot}} = T^T D T.$$

The orientation-dependent element stiffness is therefore

$$K_e(\rho_e, \alpha_e) = \rho_e^p \int_{\Omega_e} B^T D_{\mathrm{rot}}(\alpha_e) B \, d\Omega.$$

With quadrature,

$$K_e(\rho_e, \alpha_e) \approx \rho_e^p \sum_{q=1}^{Q} w_q B(r_q)^T T(\alpha_e)^T D T(\alpha_e) B(r_q) |\det J(r_q)|.$$

A.6 Orientation sensitivity

For one element, the compliance contribution is

$$C_e = \rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T T^T D T \varepsilon_q |\det J_q|, \qquad \varepsilon_q = B(r_q) u_e.$$

Only T depends on the orientation angles, so the derivative with respect to angle α_j is

$$\frac{\partial C_e}{\partial \alpha_j} = \rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T \left(\frac{\partial T^T}{\partial \alpha_j} D T + T^T D \frac{\partial T}{\partial \alpha_j} \right) \varepsilon_q |\det J_q|.$$

If D is symmetric, the scalar expression can also be evaluated as

$$\frac{\partial C_e}{\partial \alpha_j} = 2\rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T T^T D \frac{\partial T}{\partial \alpha_j} \varepsilon_q |\det J_q|,$$

provided the same transformation convention is used.

A.7 Rotation derivatives

The derivatives of the elementary rotation matrices are

$$\frac{\partial R_1}{\partial \alpha_1} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -\sin\alpha_1 & -\cos\alpha_1 \\ 0 & \cos\alpha_1 & -\sin\alpha_1 \end{bmatrix},$$

$$\frac{\partial R_2}{\partial \alpha_2} = \begin{bmatrix} -\sin\alpha_2 & 0 & \cos\alpha_2 \\ 0 & 0 & 0 \\ -\cos\alpha_2 & 0 & -\sin\alpha_2 \end{bmatrix},$$

$$\frac{\partial R_3}{\partial \alpha_3} = \begin{bmatrix} -\sin\alpha_3 & -\cos\alpha_3 & 0 \\ \cos\alpha_3 & -\sin\alpha_3 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

For $R = R_1 R_2 R_3$, the total rotation derivatives are

$$\frac{\partial R}{\partial \alpha_1} = \frac{\partial R_1}{\partial \alpha_1} R_2 R_3,$$

$$\frac{\partial R}{\partial \alpha_2} = R_1 \frac{\partial R_2}{\partial \alpha_2} R_3,$$
$$\frac{\partial R}{\partial \alpha_3} = R_1 R_2 \frac{\partial R_3}{\partial \alpha_3}.$$

The derivative $\partial T / \partial \alpha_j$ is obtained by differentiating every entry of $T(R)$ with the product rule. For example, an entry $R_{ab} R_{cd}$ contributes

$$\frac{\partial}{\partial \alpha_j} \left(R_{ab} R_{cd} \right) = \frac{\partial R_{ab}}{\partial \alpha_j} R_{cd} + R_{ab} \frac{\partial R_{cd}}{\partial \alpha_j}.$$

This product-rule structure is the key point of the orientation derivative: once R, $\partial R / \partial \alpha_j$, T, and $\partial T / \partial \alpha_j$ are known, the sensitivity follows directly from the quadrature expression above.